

Programmable Logic Devices

Prof. Claudio Passerone

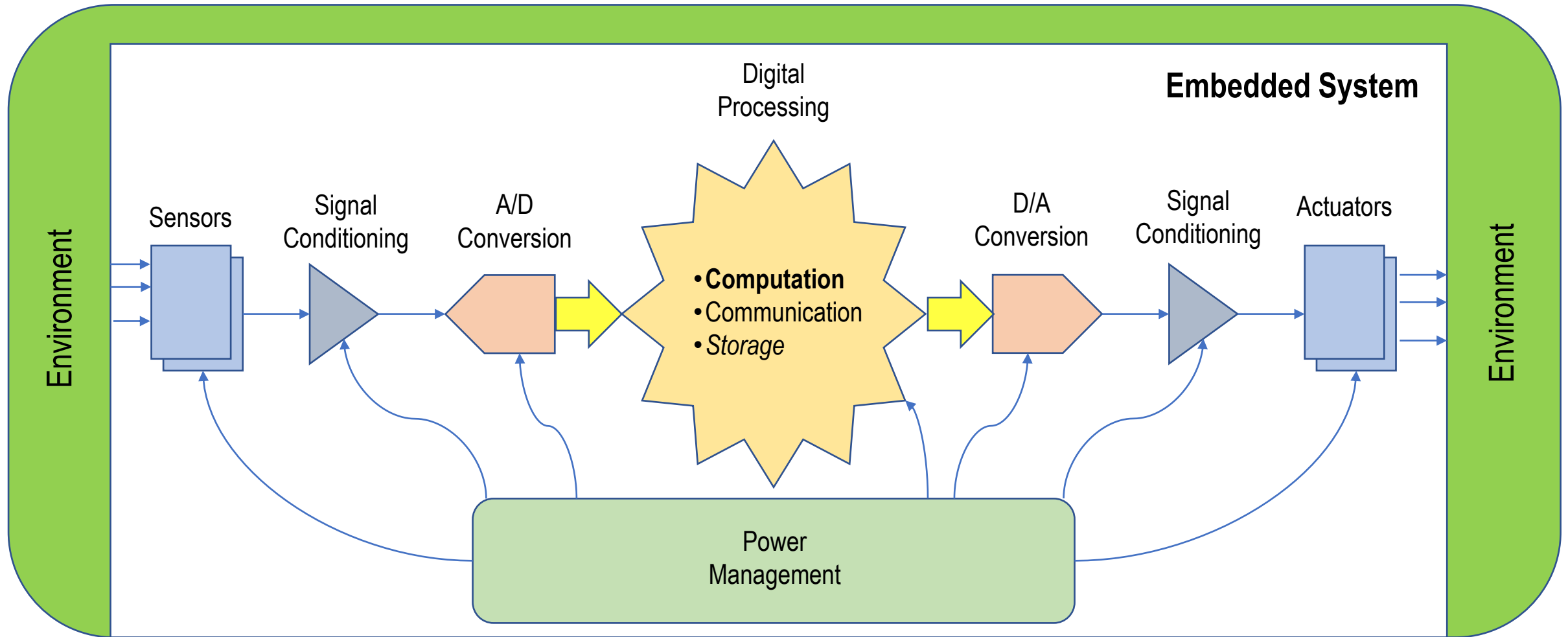
(Electronics for Embedded Systems)

Outline

- Introduction
- CPLDs
 - ROM, PLA, PAL, Macrocell, Macroblock, CPLD
 - Interconnections
 - Input/Output
- FPGAs
 - Logic Element, Logic Array Block, Device architecture
 - Additional resources
- Design flow
- Place&Route

Introduction

- Typical architecture of an Embedded System



Introduction

- Custom digital hardware
 - Full custom IC
 - Best performance and power consumption
 - Very high NRE costs (masks, design)
 - Long time-to-market
 - Good for demanding very high volume designs
 - Gate arrays
 - Very good performance and power
 - High NRE costs (masks)
 - Only higher layers masks, not the full IC like in full custom
 - Long time-to-market
 - Good for high volume designs

Introduction

- Custom digital hardware
 - Programmable Logic
 - Good performance and power (compared to SW)
 - Low NRE costs (compared to IC)
 - No masks, the user buys an already manufactured integrated circuit
 - Higher per-unit costs
 - Good for low volume designs and prototyping

Introduction

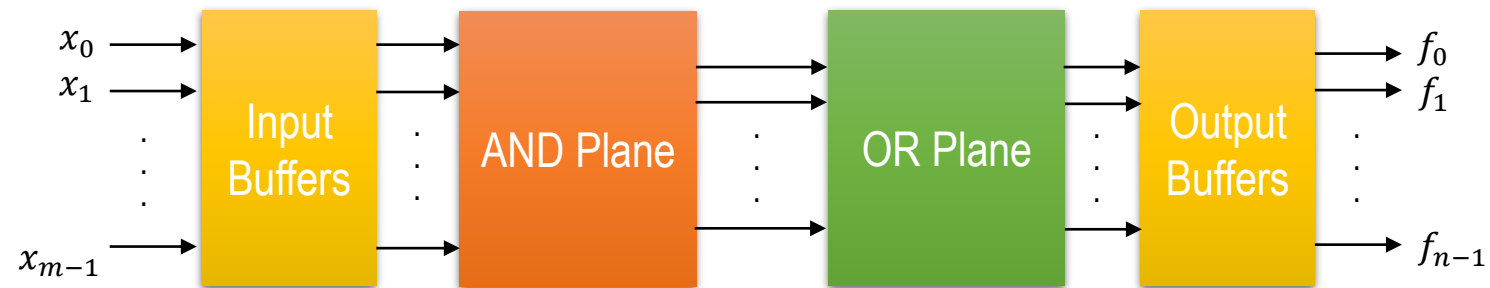
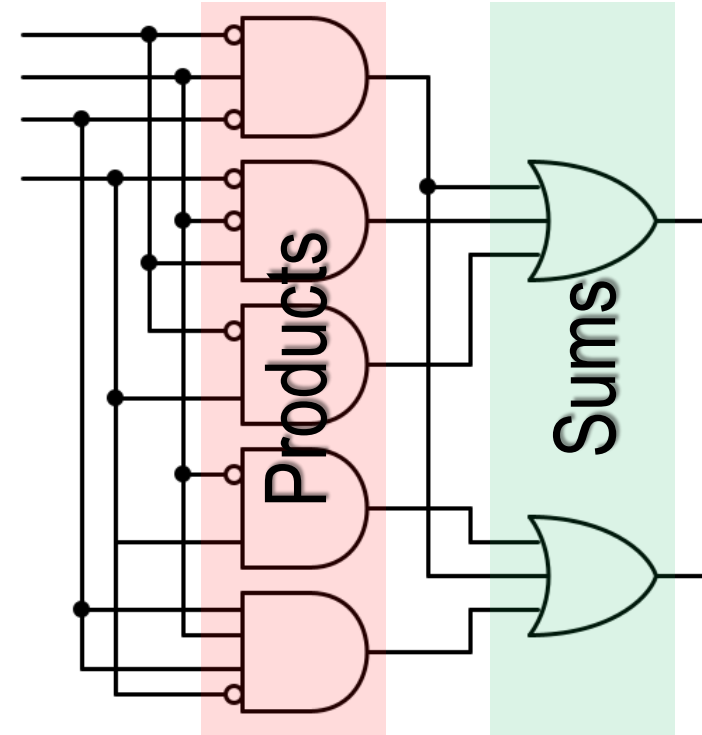
- What is a programmable logic device (PLD)?
 - It is an integrated circuit (hardware device)
 - Contains logic elements of various complexities
 - Allows the designer to configure and connect them in different ways
 - Configuration after manufacturing
 - It implements a hardware digital circuit
 - The function of the circuit is configured by the user of the PLD
 - Masks do not depend on the implemented function
 - The same device can be used for different applications
 - Can be produced in large volumes
 - NRE costs (for the PLD manufacturer) spread over many different customers
 - Designers of PLDs use full custom, users of PLDs don't

Introduction

- PLDs can be configured to implement a logic circuit
 - Combinational or sequential
 - The final circuit is hardware, not software
 - But it is usually described using a high level “programming” language
 - VHDL, Verilog, SystemC, Chisel, HLS
 - Designers define
 - Internal logic functions
 - Interconnections and pins
- Technologies used to configure PLDs
 - Fuse, anti-fuse (one time programmable)
 - Floating gate (reprogrammable)
 - SRAM (reprogrammable, reconfigurable at run-time, faster than floating gate technology)

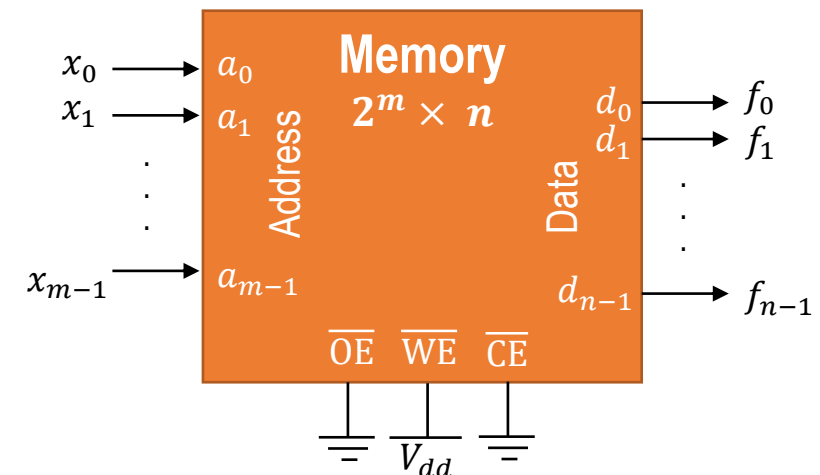
Introduction

- Expressing a combinational logic circuit
 - Boolean expressions
 - Truth tables
- Canonical forms
 - Sum of Products (SoP)
 - Two-levels logic implementation
 - AND Plane for products
 - OR Plane for sums
 - Minimization
 - Karnaugh maps
 - Other algorithms (multilevel logic, multiple functions, ...)



ROM based PLD

- Memories can implement combinational logic functions
 - Address pins $\rightarrow$ logic inputs
 - Data pins $\rightarrow$ logic outputs
 - Content $\rightarrow$ truth table of the function
- If the content is changed, the function changes
 - A $(2^m \times n)$ memory can implement any set of n combinational logic functions of m shared inputs
 - Inputs are common to the n logic functions
 - OTPROM, EPROM, E²PROM (permanent), SRAM (volatile)
 - Not ROM, otherwise make it full custom
 - Not NAND Flash, because of the sequential read access
 - Not DRAM, because of the complex access protocol
- When used as a circuit, the memory is always read
 - Written only before its use, to configure the circuit



ROM based PLD

- Examples

- Using an 8 x 2 memory (8 locations, $m = 3$ address lines, $n = 2$ outputs)
 - Full adder
- Using a 16 x 1 memory (16 locations, $m = 4$ address lines, $n = 1$ output)
 - Computing parity
 - Multiplexer (with multiple pin assignments)
- Using a 16 x 2 memory (16 locations, $m = 4$ address lines, $n = 2$ outputs)
 - Full adder (3 address lines used, 2 outputs, see LUT examples)
 - Encoder (4 inputs, 2 outputs, as an exercise)

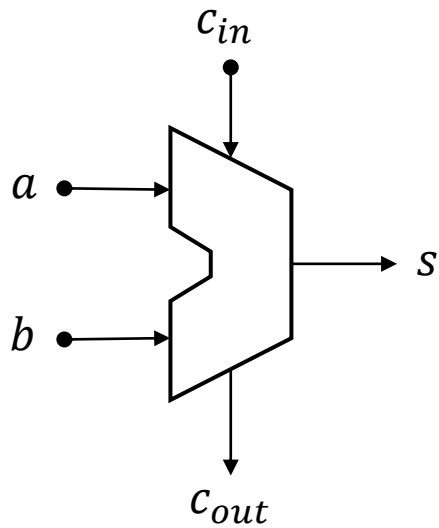
- Advantages

- Very simple to implement
- Exploits fuse and floating gate technologies
- Don't need to optimize the function

ROM based PLD

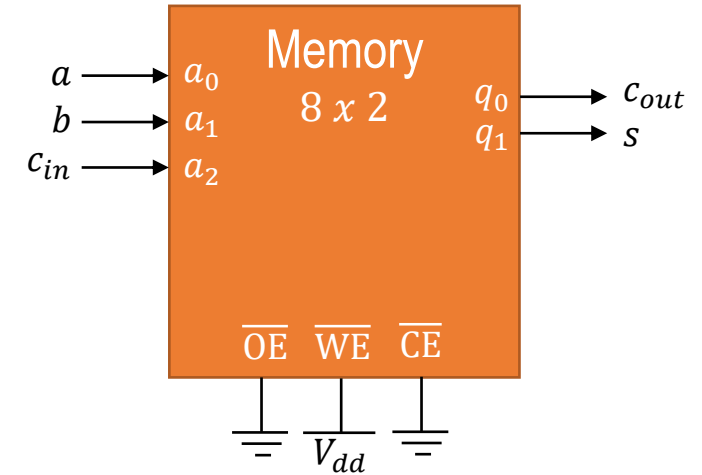
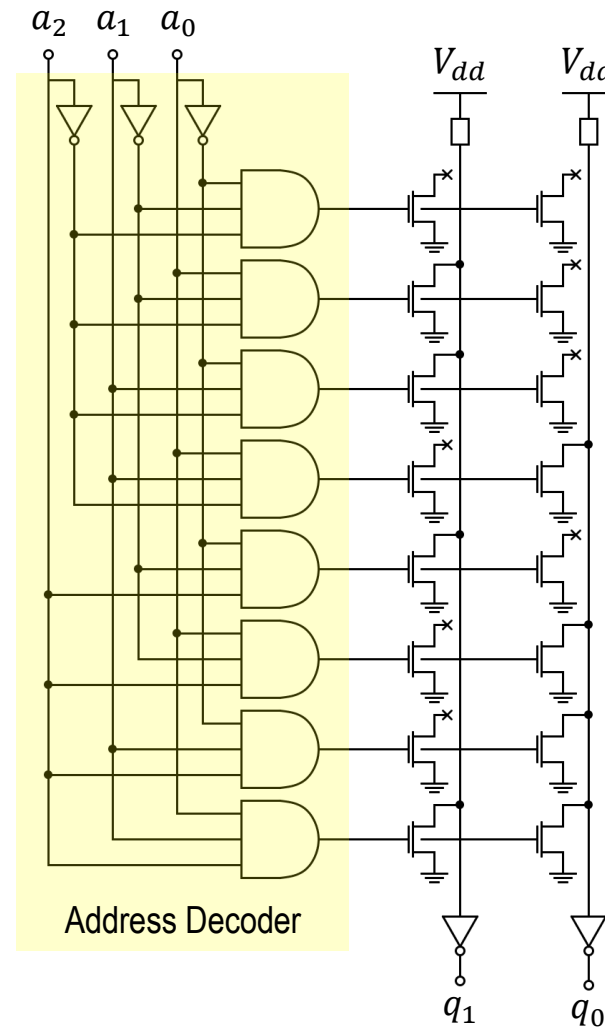
- Full Adder

- Using an 8 x 2 memory
- Memory content depends on pin assignment



Truth Table

c_{in}	b	a	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

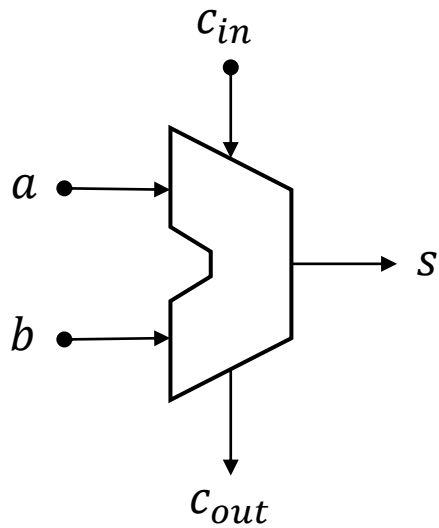


Memory content

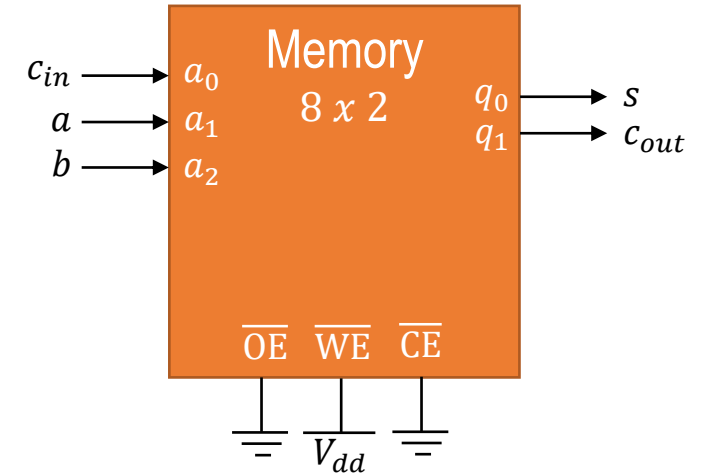
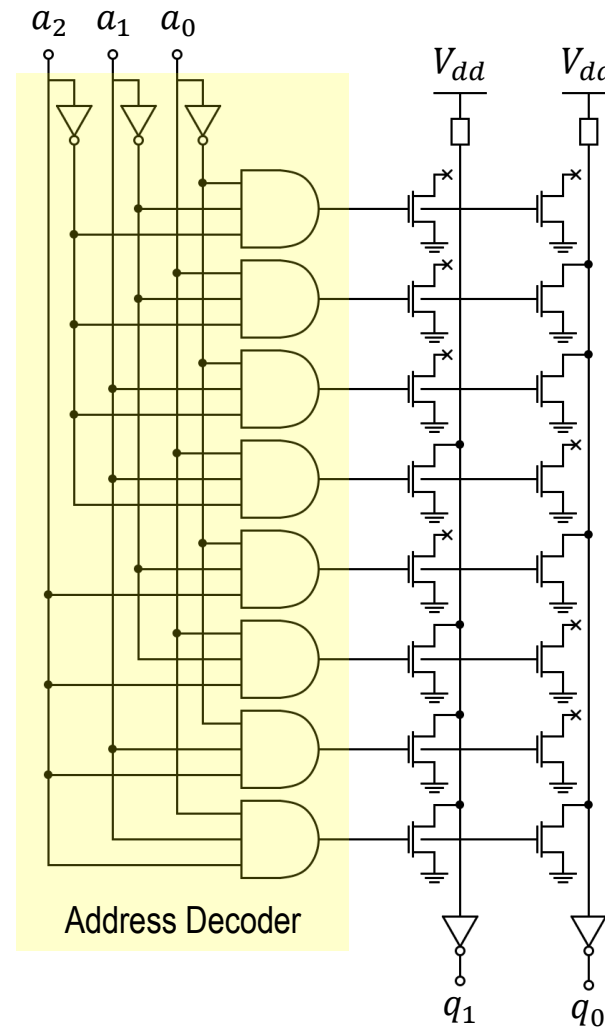
Address	Content
0	0
1	2
2	2
3	1
4	2
5	1
6	1
7	3

ROM based PLD

- Full Adder (change pin assignment)
 - Using an 8 x 2 memory
 - Memory content depends on pin assignment



Truth Table				
b	a	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

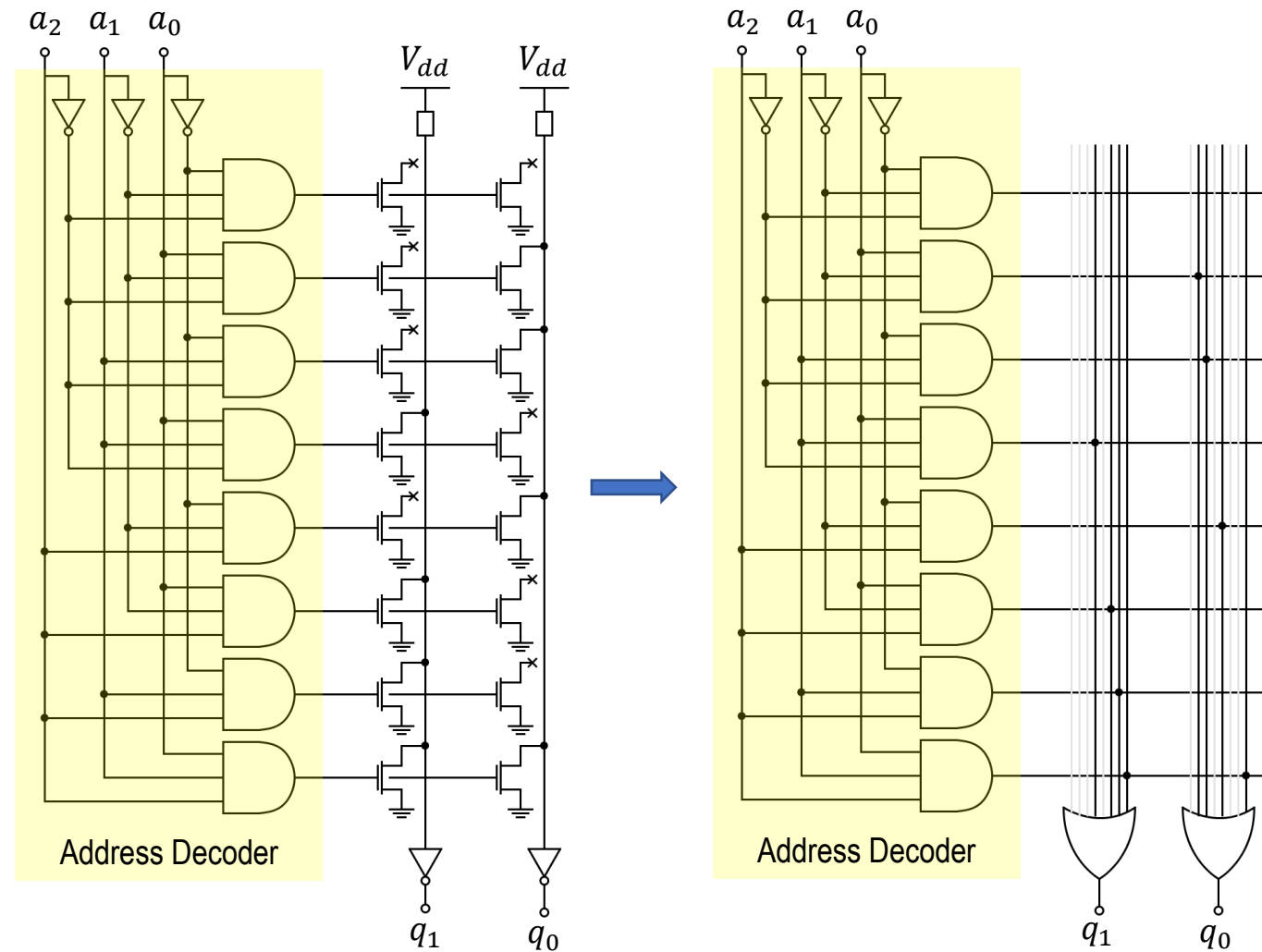


Memory content

Address	Content
0	0
1	1
2	1
3	2
4	1
5	2
6	2
7	3

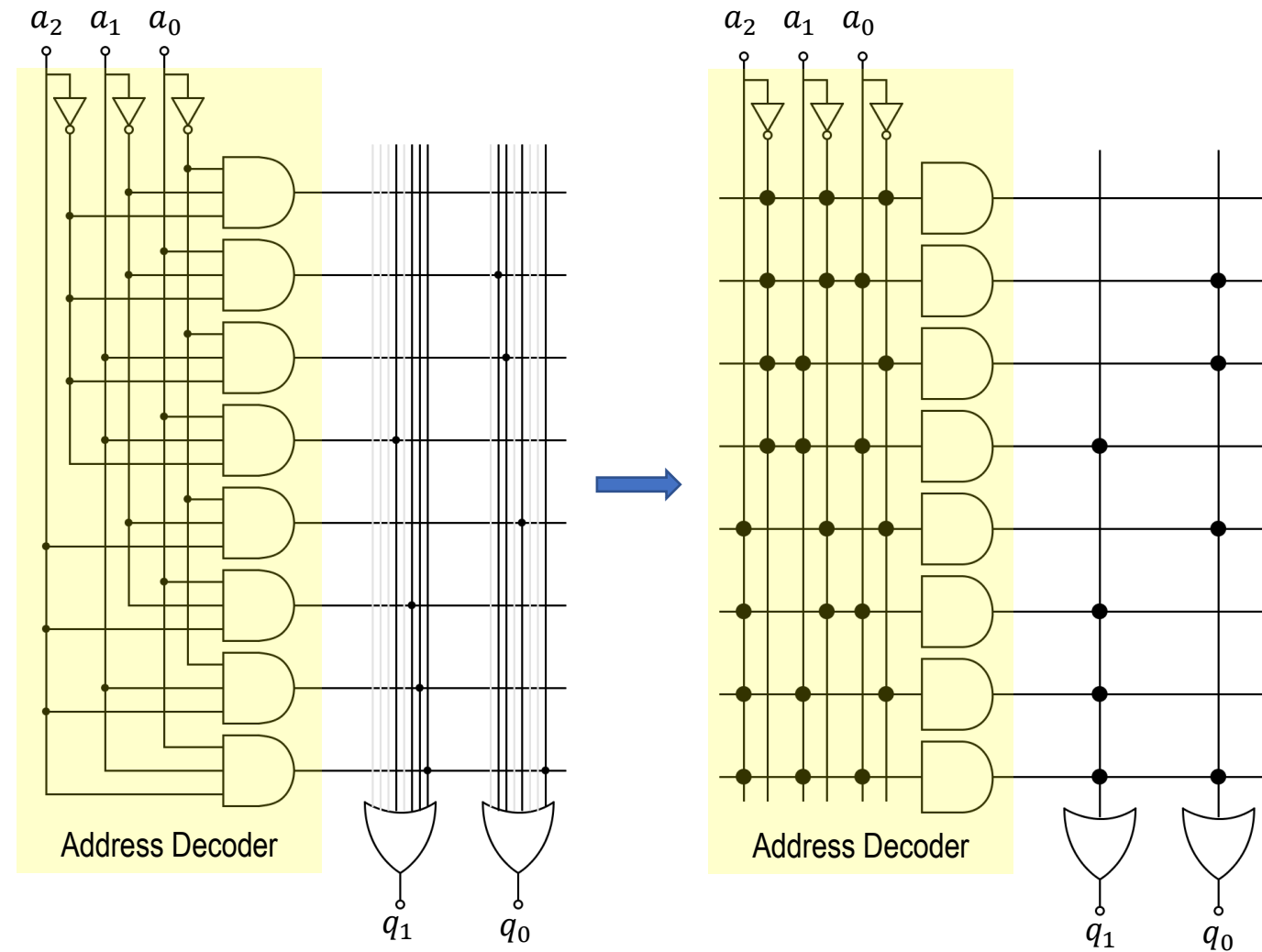
ROM based PLD

- Canonical form
 - It is a Sum of Products (SoP)
 - AND Plane is the Address Decoder
 - Computes all products with all literals
 - Also known as *minterms*
 - A *minterm* is a product term where each variable appears once, affirmed or complemented, such that the result is logic 1
 - OR Plane is the Wired OR
 - Pulled-up Bit Lines
 - Parallel transistors with Source grounded
 - Output inverter
 - Alternative schematic diagram
 - Used to represent many kind of PLDs in the following slides



ROM based PLD

- Canonical form
 - It is a Sum of Products (SoP)
 - AND Plane is the Address Decoder
 - Computes all products with all literals
 - Also known as *minterms*
 - A *minterm* is a product term where each variable appears once, affirmed or complemented, such that the result is logic 1
 - OR Plane is the Wired OR
 - Pulled-up Bit Lines
 - Parallel transistors with Source grounded
 - Output inverter
 - Alternative schematic diagram
 - Used to represent many kind of PLDs in the following slides

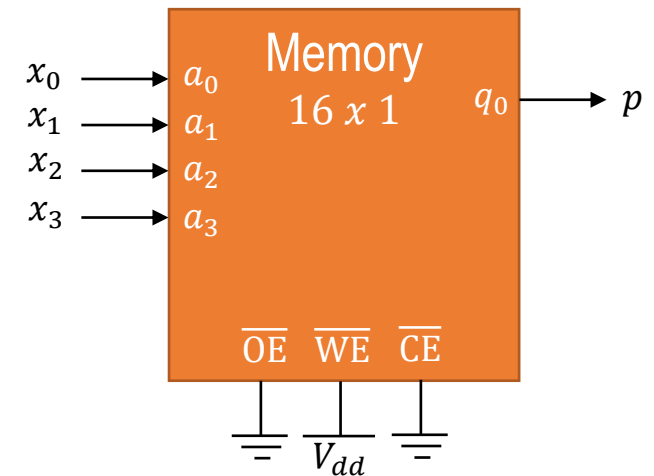


ROM based PLD

- Computing parity
 - Even parity
 - Total number of bits at 1 should be even
 - Odd parity
 - Total number of bits at 1 should be odd
 - Complement the memory content
- Because of the nature of the function the memory content does not depend on pin assignment

Truth Table

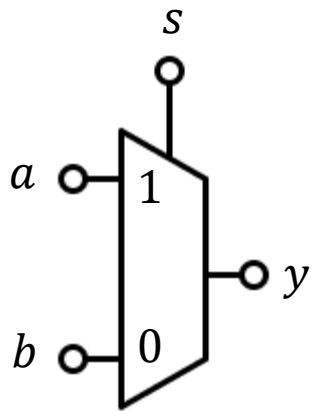
x_3	x_2	x_1	x_0	p
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



ROM based PLD

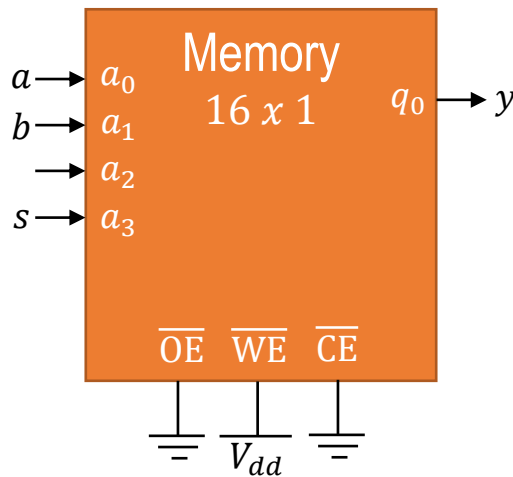
- Multiplexer

- 3 inputs, 1 output
- Using a 16 x 1 memory
 - One input unused
 - Make it *don't care*
 - But don't leave it floating!



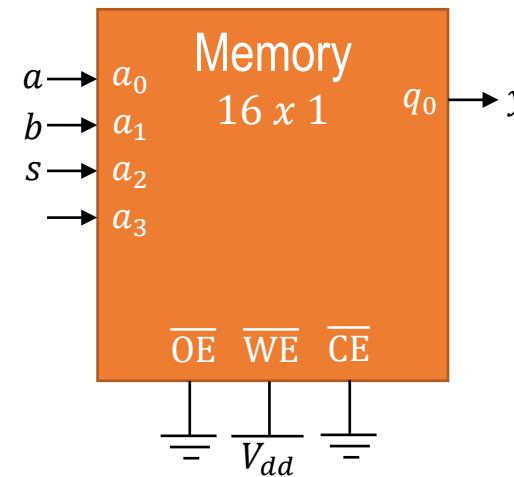
Truth Table

<i>b</i>	<i>a</i>	<i>s</i>	<i>y</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



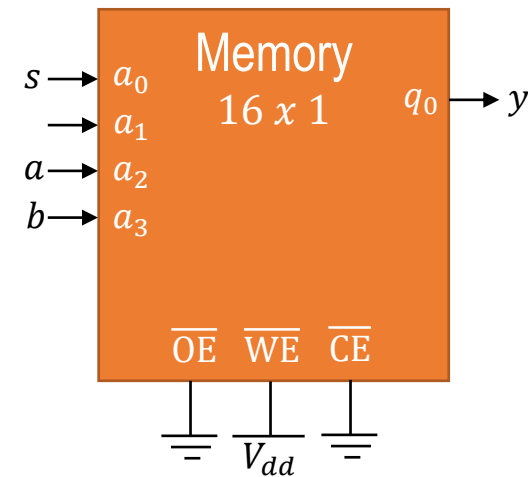
Memory content

<i>a</i> ₃	<i>a</i> ₂	<i>a</i> ₁	<i>a</i> ₀	<i>y</i>
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



Memory content

<i>a</i> ₃	<i>a</i> ₂	<i>a</i> ₁	<i>a</i> ₀	<i>y</i>
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



Memory content

<i>a</i> ₃	<i>a</i> ₂	<i>a</i> ₁	<i>a</i> ₀	<i>y</i>
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

ROM based PLD

- Disadvantages
 - Size of memory exponential on the number of inputs
 - 4 input, 1 output → 16 bits (2 bytes)
 - 10 input, 1 output → 1024 bits (128 bytes)
 - 16 input, 8 output → 524288 bits (64 Kbytes)
 - 32 input, 8 output → 4 Gbytes
 - A 16 bit adder is even larger than this!
 - Slow
 - Particularly with many inputs
 - Address decoders become large and use many levels of logic

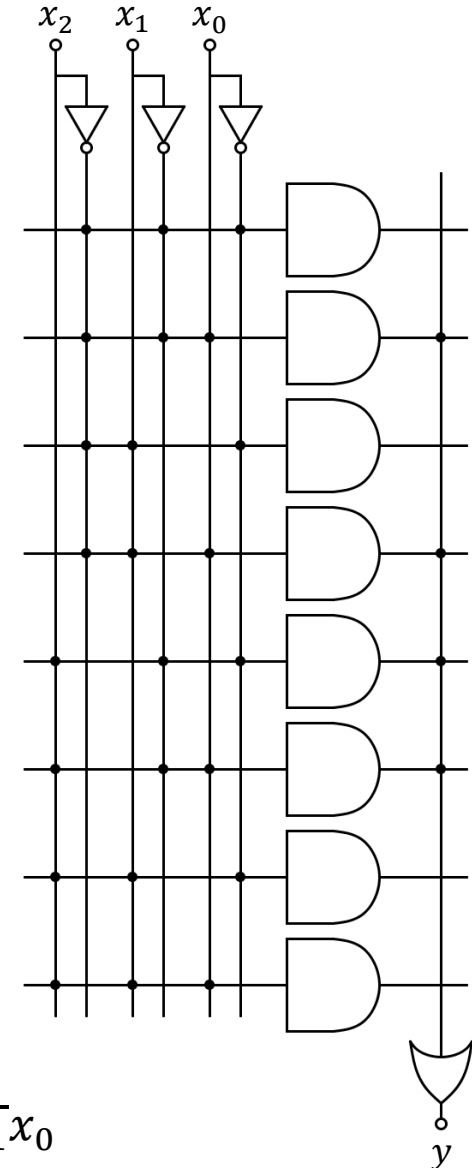
ROM based PLD

- Internal structure
 - The address decoder implements all possible minterms
 - Only minterms whose output is logic 1 for the specific function to implement are connected to the Bit Line
 - Implements a sum of product canonical form
 - Example: 3-input 1-output

Truth Table

x_2	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$y = \overline{x_2}\overline{x_1}x_0 + \overline{x_2}x_1x_0 + x_2\overline{x_1}\overline{x_0} + x_2\overline{x_1}x_0$$



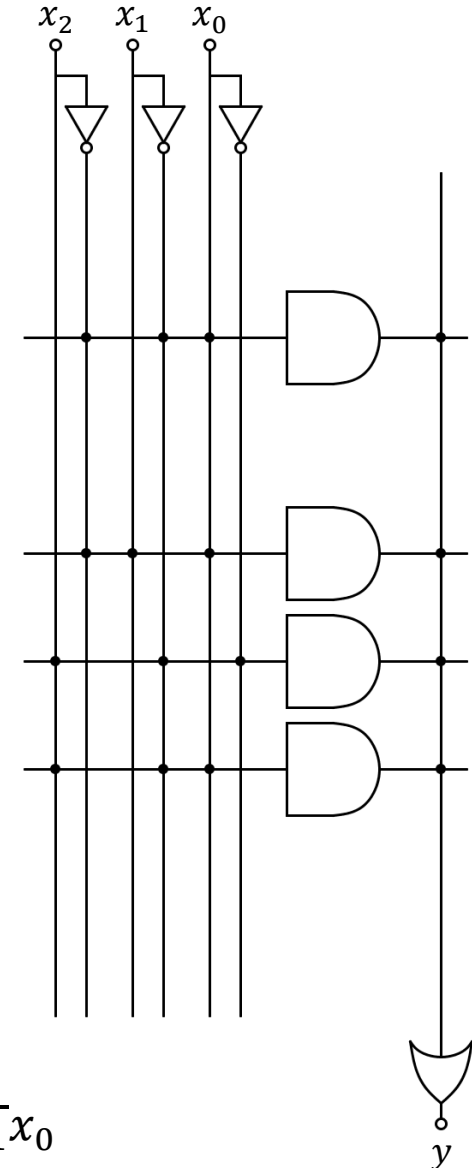
ROM based PLD

- Eliminate minterms whose output is logic 0
 - They don't affect the output
 - Reduced area → reduced cost
 - Reduced parasitic capacitances → improved performance
 - Same example as before

Truth Table

x_2	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$y = \overline{x_2}\overline{x_1}x_0 + \overline{x_2}x_1x_0 + x_2\overline{x_1}x_0 + x_2\overline{x_1}x_0$$



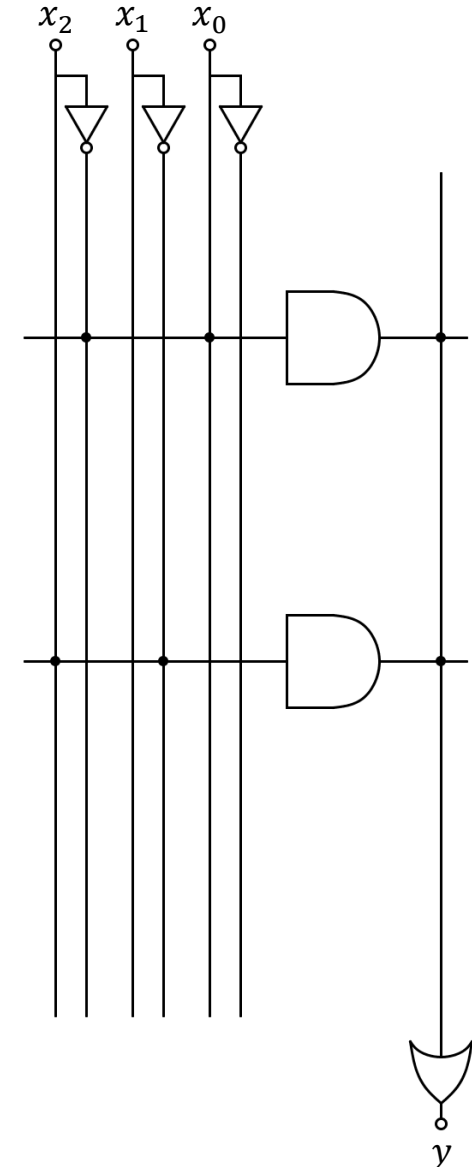
ROM based PLD

- Merge minterms that are part of the same cube (implicant)
 - Use standard boolean optimization techniques
 - Karnaugh maps
 - Quine-McCluskey algorithm
 - Reduced area → reduced cost
 - Better performance

Truth Table

x_2	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$y = \overline{x_2}x_0 + x_2\overline{x_1}$$



ROM based PLD

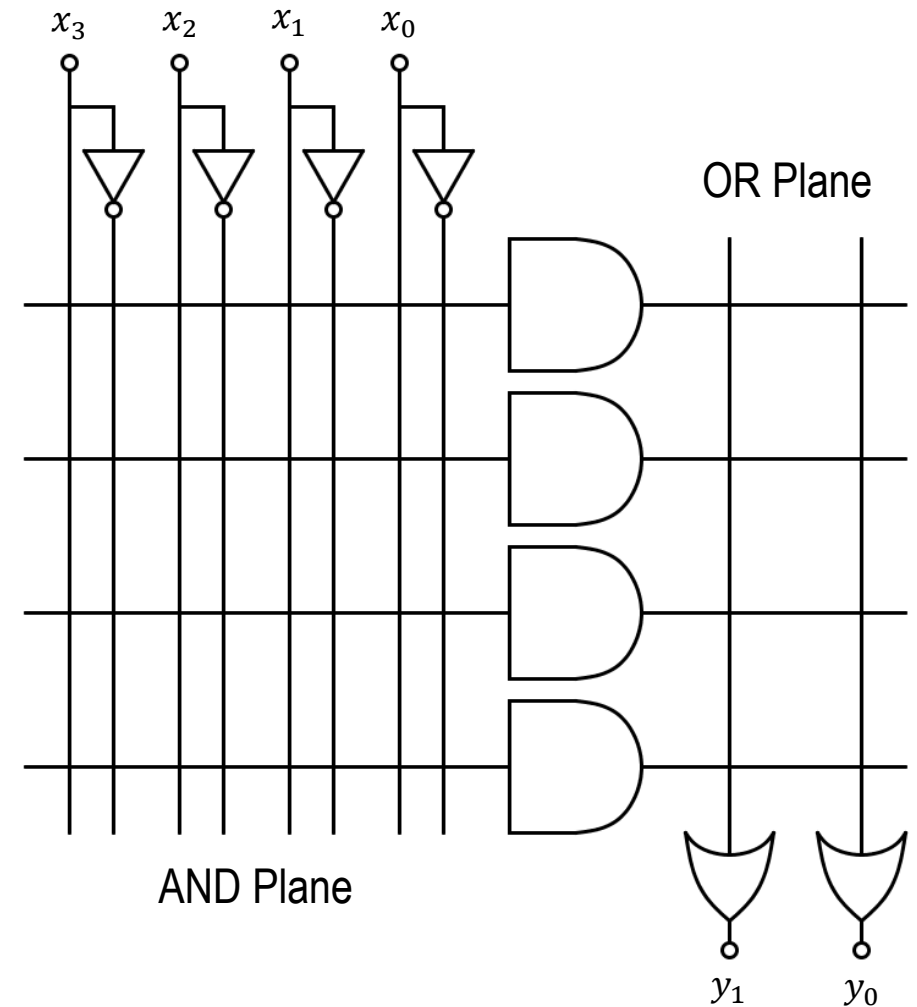
- After optimizations, the memory is not general anymore
 - Lesson learned
 - The full address decoder is not needed for most logic functions
 - Logic optimizations allow reducing the number of products
 - But in a ROM the address decoder is fixed
- How to make it general?
 - Make connections to AND gates configurable, as in the OR plane
 - Use fuses or floating gate technologies (same as in OR plane)
 - The number of AND gates becomes independent of the number of inputs (address lines)
 - ROM has a fixed AND plane and a programmable OR plane
 - Always computes all minterms
 - Number of AND gates = 2^m
 - Where m is the number of inputs

Programmable Logic Array (PLA)

- Programmable Logic Array (PLA)
 - Programmable AND and OR planes
 - An m -input n -output PLA with k AND gates (each with m inputs), with $k < 2^m$, can implement any set of n combinational logic functions of m inputs, provided that the total number of implicants (cubes) is not larger than k , the number of available AND gates
 - Not all possible m -inputs n -outputs functions can be implemented using a PLA
 - A ROM has $k = 2^m$ AND gates, so it can implement all functions
 - PLA size linearly depends on the number of inputs
 - ROM size is exponential on the number of inputs
 - The number of AND gates k does not depend on the number of inputs m
 - PLA size linearly depends on the number of AND gates
 - The number of AND gates k is decided by the manufacturer
 - Based on other constraints, like performance, costs, etc.

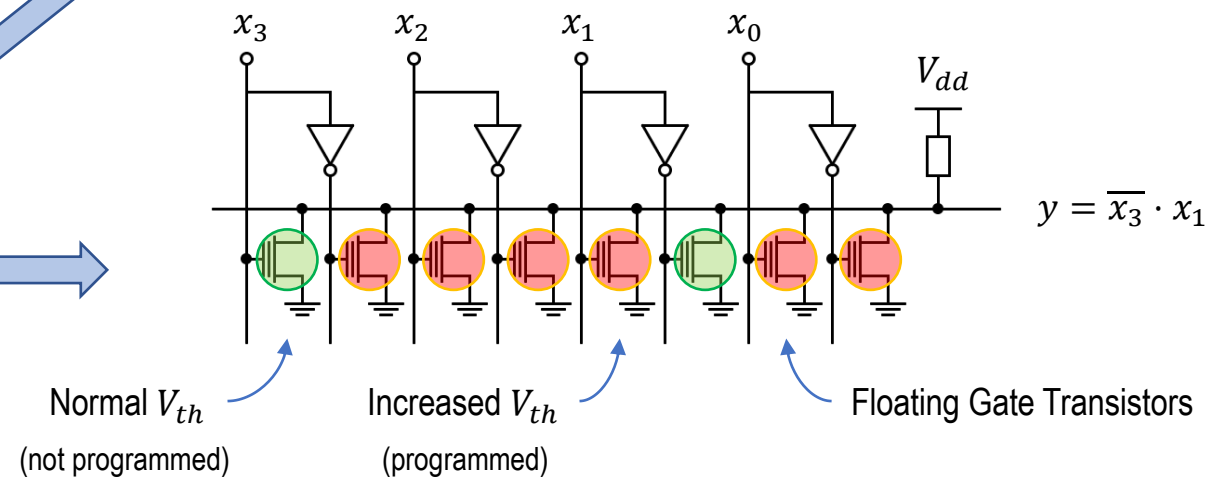
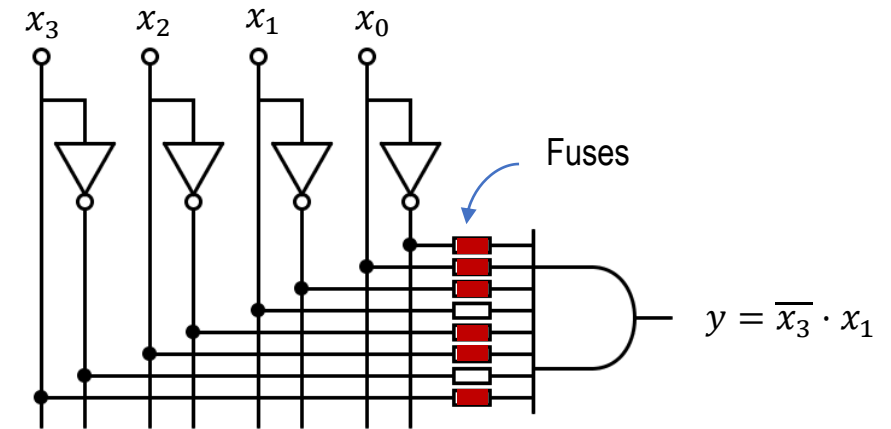
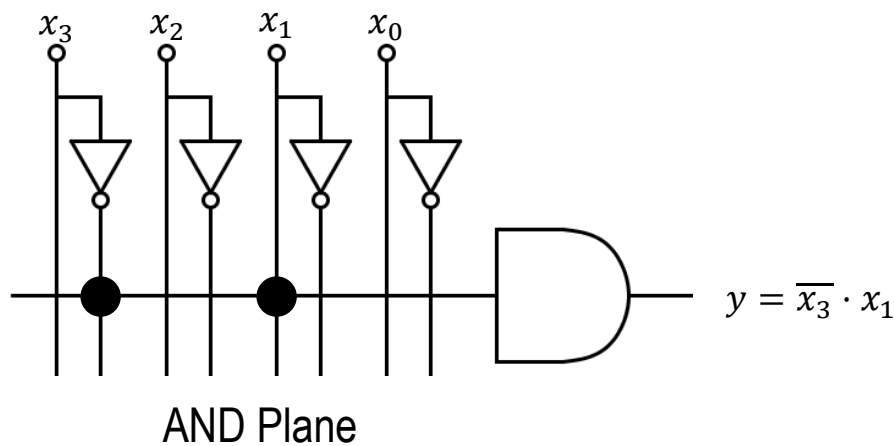
Programmable Logic Array (PLA)

- PLA structure example
 - 4 inputs ($m = 4$)
 - Each input affirmed and complemented
 - 4 AND gates ($k = 4$)
 - It is $k < 2^m$ ($4 < 2^4 = 16$)
 - The number k of AND gates does not depend on the number m of inputs
 - It could be other values
 - For instance 3, 5 or 6
 - More than 16 is useless in this case
 - 2 outputs ($n = 2$)
 - All intersections are programmable interconnections



Programmable Logic Array (PLA)

- PLA structure example
 - All intersections are programmable interconnections
 - OR Plane → Wired OR
 - Similar to memories
 - Fuses or floating gate transistors
 - AND Plane
 - Fuses or floating gate transistors

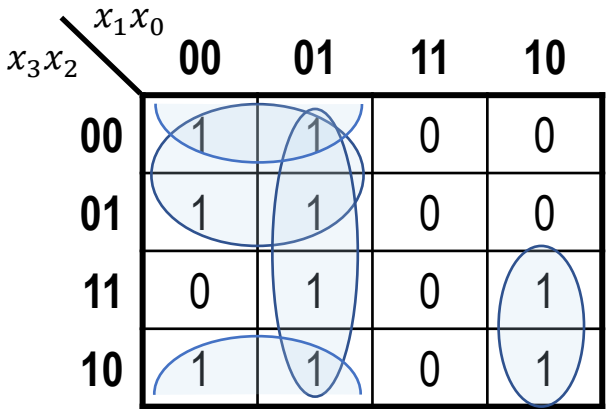


Programmable Logic Array (PLA)

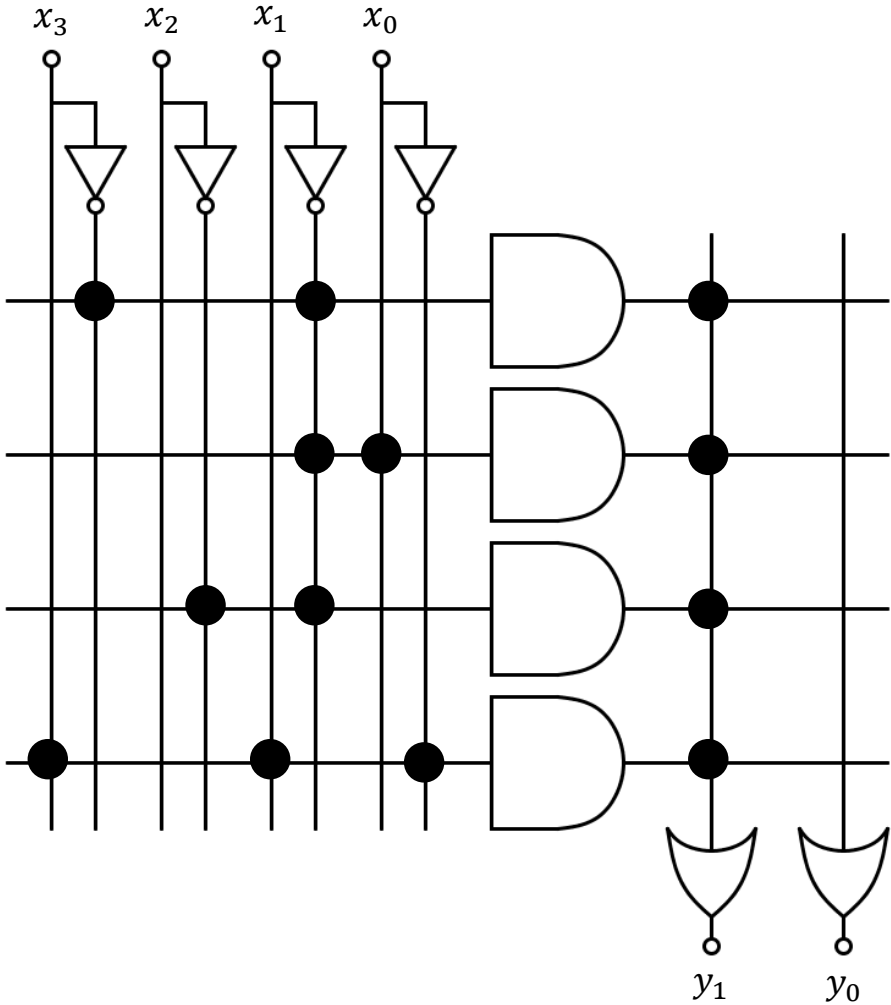
- PLA Example 1
 - 1 function, 9 minterms, 4 implicants
 - It fits, wastes one output

Truth Table

x_3	x_2	x_1	x_0	y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



$$y = \overline{x_3} \overline{x_1} + \overline{x_1} x_0 + \overline{x_2} \overline{x_1} + x_3 x_1 \overline{x_0}$$



Programmable Logic Array (PLA)

• PLA Example 2

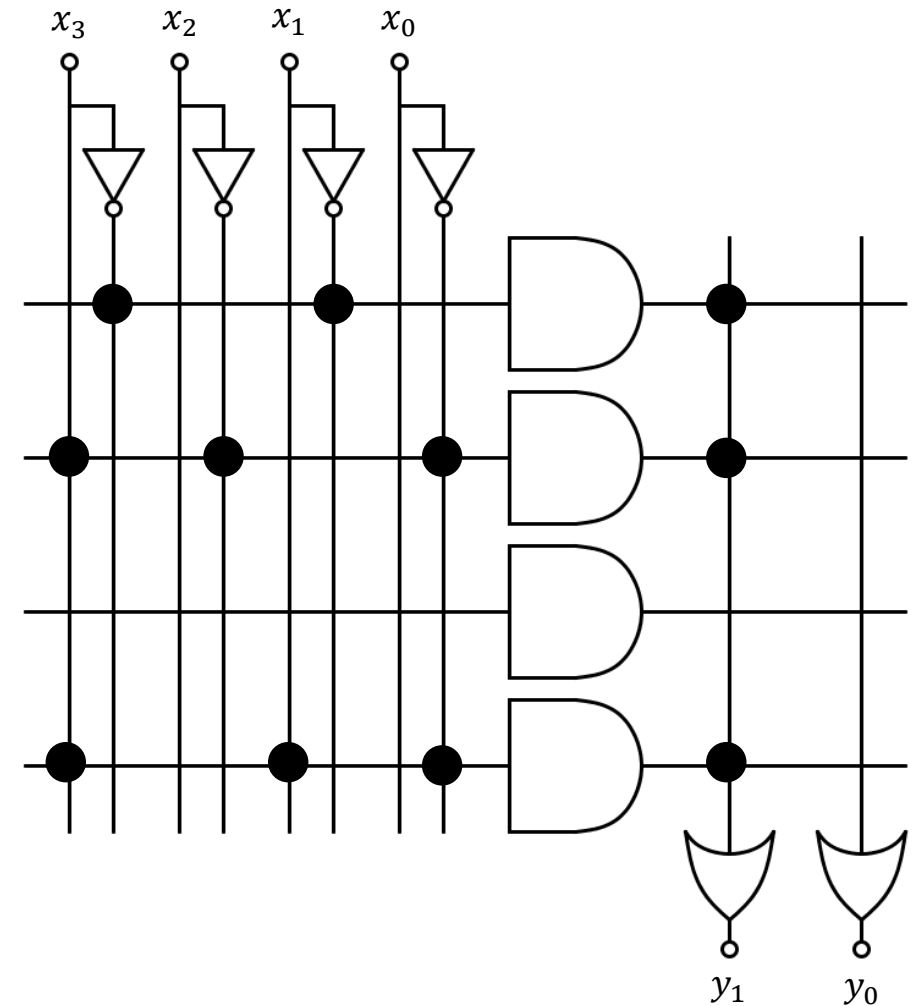
- 1 function, 7 minterms, 3 implicants
 - It fits, wastes one output and one AND

Truth Table

x_3	x_2	x_1	x_0	y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

		$x_1 x_0$			
		00	01	11	10
$x_3 x_2$	00	1	1	0	0
	01	1	1	0	0
	11	0	0	0	1
	10	1	0	0	1

$$y = \overline{x_3} \overline{x_1} + x_3 x_1 \overline{x_0} + x_3 \overline{x_2} \overline{x_0}$$



Programmable Logic Array (PLA)

• PLA Example 3

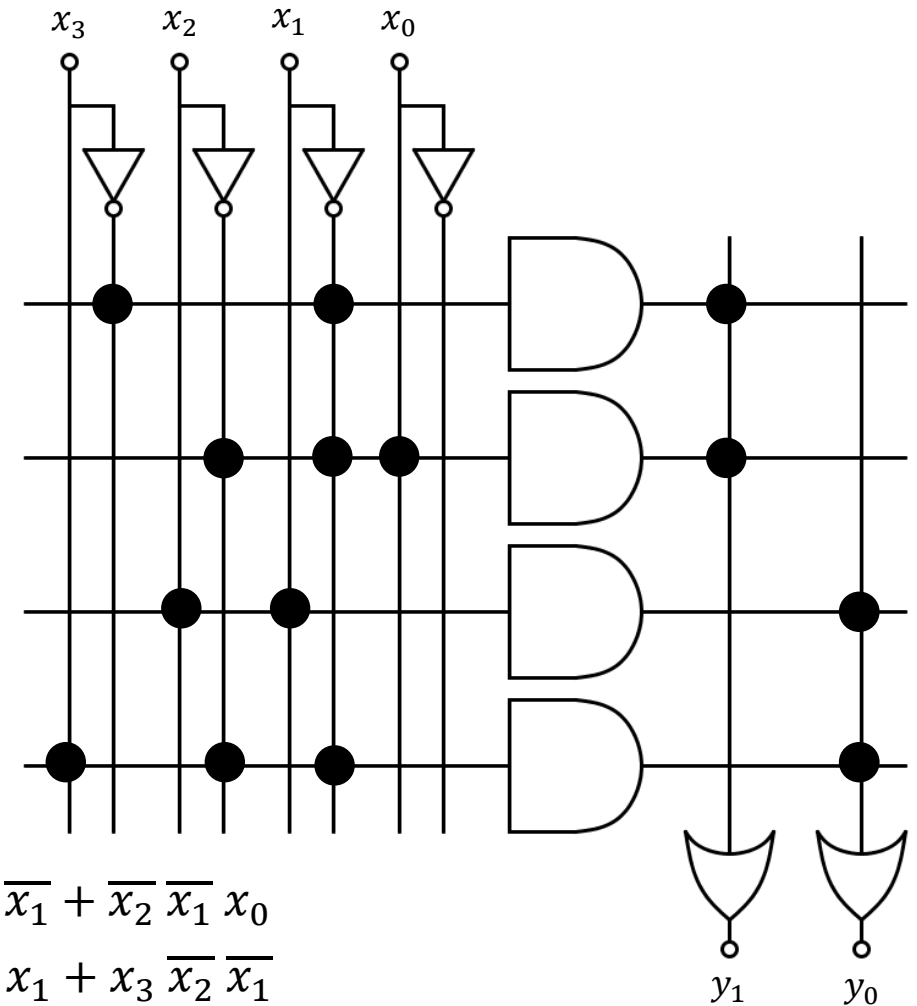
- 2 functions, 10 minterms, 2+2=4 distinct implicants
 - They fit

Truth Table

x_3	x_2	x_1	x_0	y_1	y_0
0	0	0	0	1	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	1
1	1	1	1	0	1

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	0	0	0	0
10	0	1	0	0

$x_3x_2 \backslash x_1x_0$	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	0	1	1
10	1	1	0	0



Programmable Logic Array (PLA)

- PLA Example 4

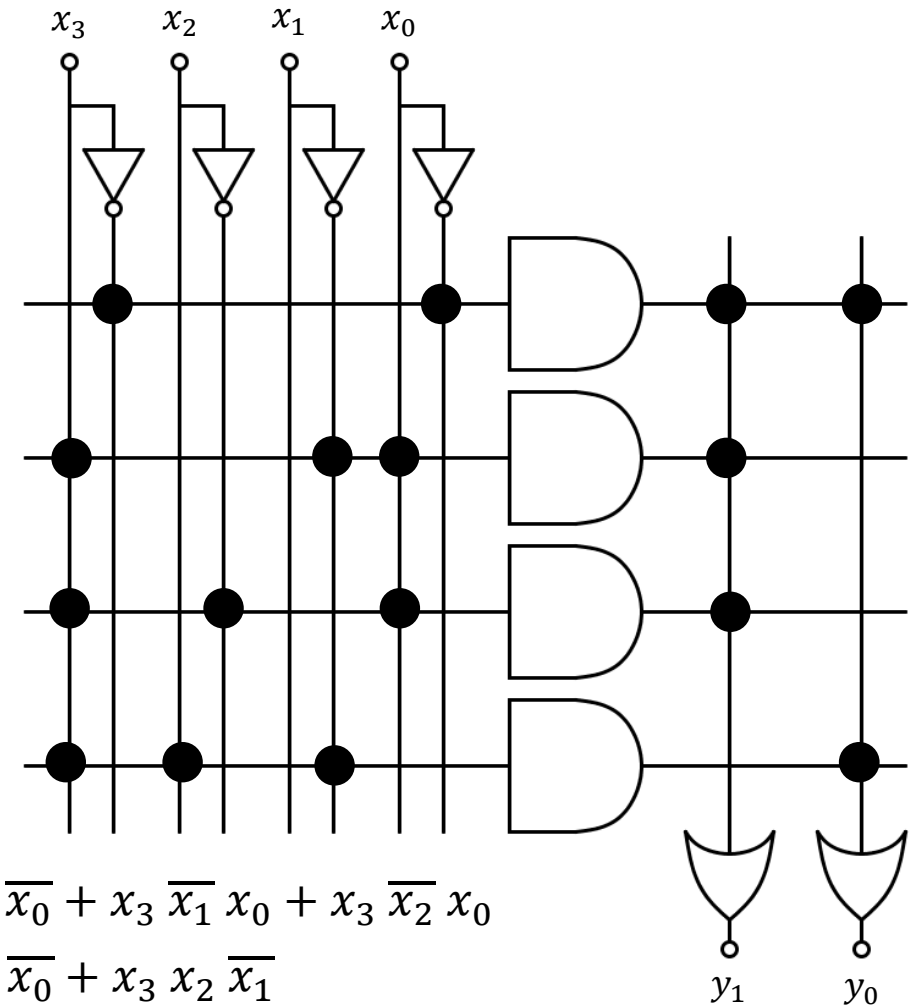
- 2 functions, 8 minterms, 3+2=5 but 4 distinct implicants
 - They fit

Truth Table

x_3	x_2	x_1	x_0	y_1	y_0
0	0	0	0	1	1
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	1
1	1	1	0	0	0
1	1	1	1	0	0

$x_3x_2 \backslash x_1x_0$				
	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	0	1	0	0
10	0	1	1	0

$x_3x_2 \backslash x_1x_0$				
	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	1	0	0
10	0	0	0	0



Programmable Logic Array (PLA)

• PLA Example 5

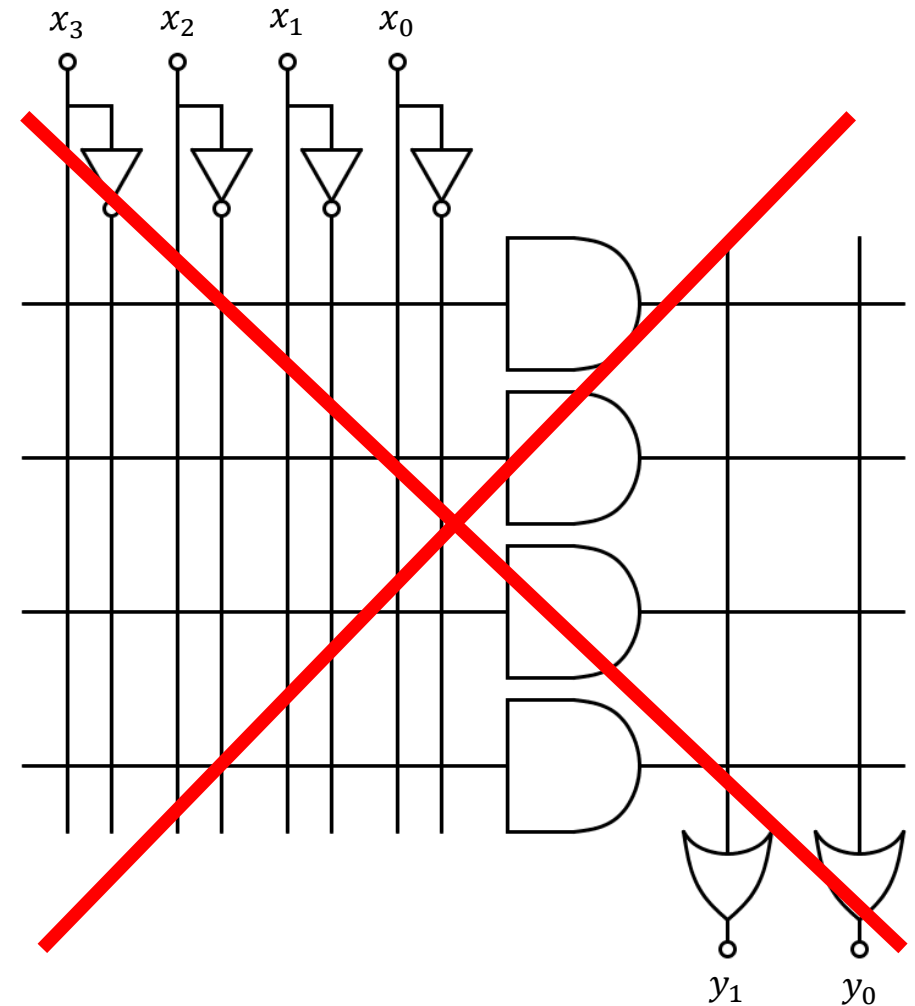
- 1 function, 7 minterms, 5 implicants
 - It doesn't fit, it can be implemented in a ROM

Truth Table

x_3	x_2	x_1	x_0	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

		$x_1 x_0$			
		00	01	11	10
$x_3 x_2$	00	1	0	0	0
	01	0	1	1	0
	11	1	0	1	0
	10	0	0	1	1

$$y = \overline{x_3} \overline{x_2} \overline{x_1} \overline{x_0} + x_3 x_2 \overline{x_1} \overline{x_0} + \overline{x_3} x_2 x_0 + x_2 x_1 x_0 + x_3 \overline{x_2} x_1$$



Programmable Logic Array (PLA)

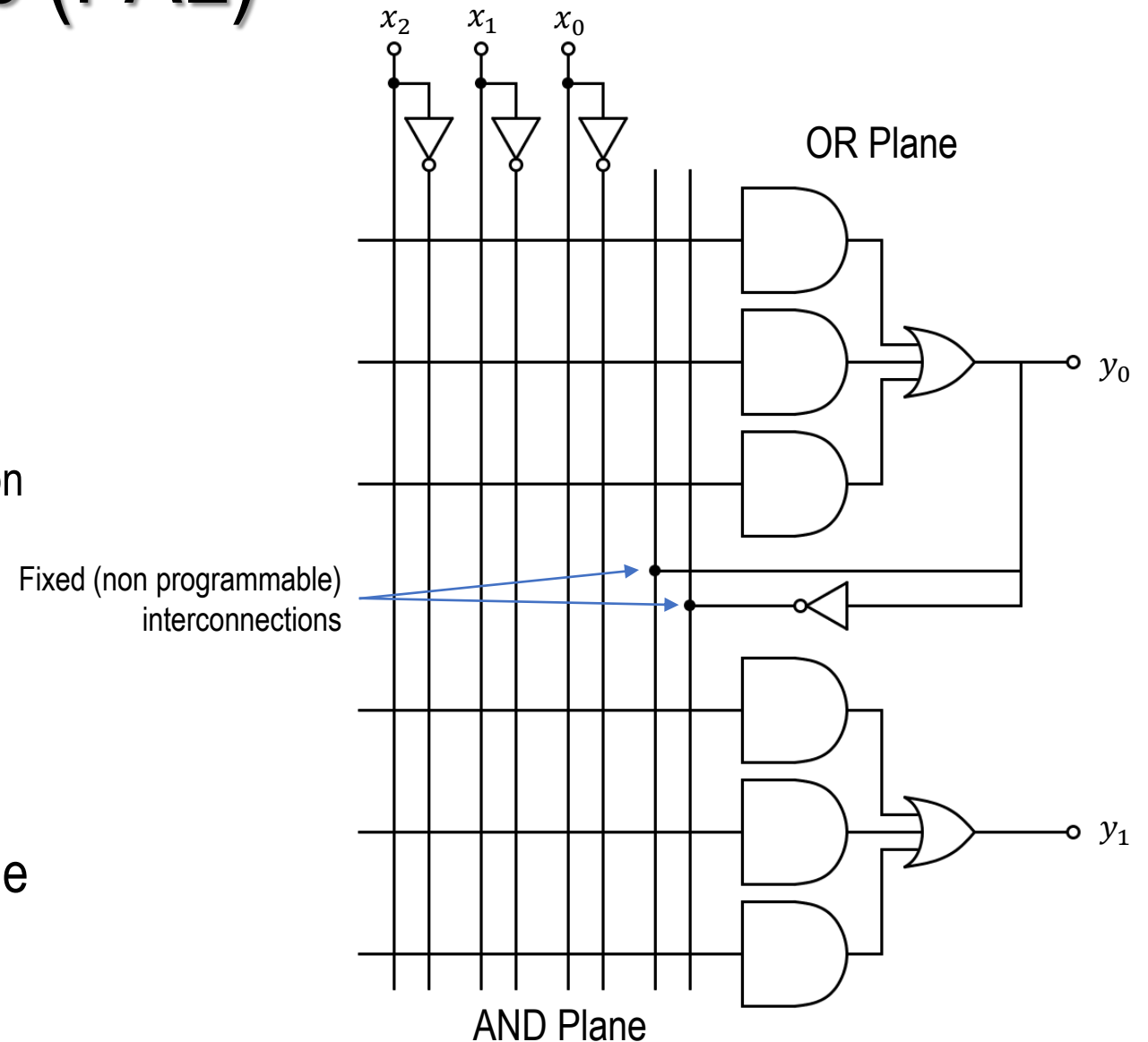
- PLA problems
 - Speed
 - Due to the several programmable interconnections PLAs are pretty slow
 - High parasitic resistances and capacitances
 - Slower than ROMs
- Implementing sequential functions
 - Output in feedback to the input
 - Outside the integrated circuit
 - High capacitances (pads)
 - Slow
- Increasing the number of input and output pins
 - Gates with many inputs are slow

Programmable Array Logic (PAL)

- Programmable Array Logic (PAL)
 - Increase speed by reducing flexibility
 - Reduce the number of programmable connections
 - Reduce parasitic resistances and capacitances
 - Device is faster than ROMs and PLAs
- Programmable AND, fixed OR
 - AND plane identical to PLA
 - Fixed OR plane
 - Limits the number of implicants for each function
 - Regardless of the total number of AND gates in the AND plane
 - Restricted set of combinational functions that can be implemented
 - Compared to ROM and PLA

Programmable Array Logic (PAL)

- PAL structure example
 - 3 inputs
 - Each input affirmed and complemented
 - 6 AND gates
 - Grouped in 2 sets of 3 AND gates
 - The number of AND gates does not depend on the number of inputs
 - 2 outputs
 - Connections in the OR plane are fixed
 - All intersections are programmable interconnections
 - Bring outputs in feedback in the AND plane
 - Implement functions using multi-level logic



Programmable Array Logic (PAL)

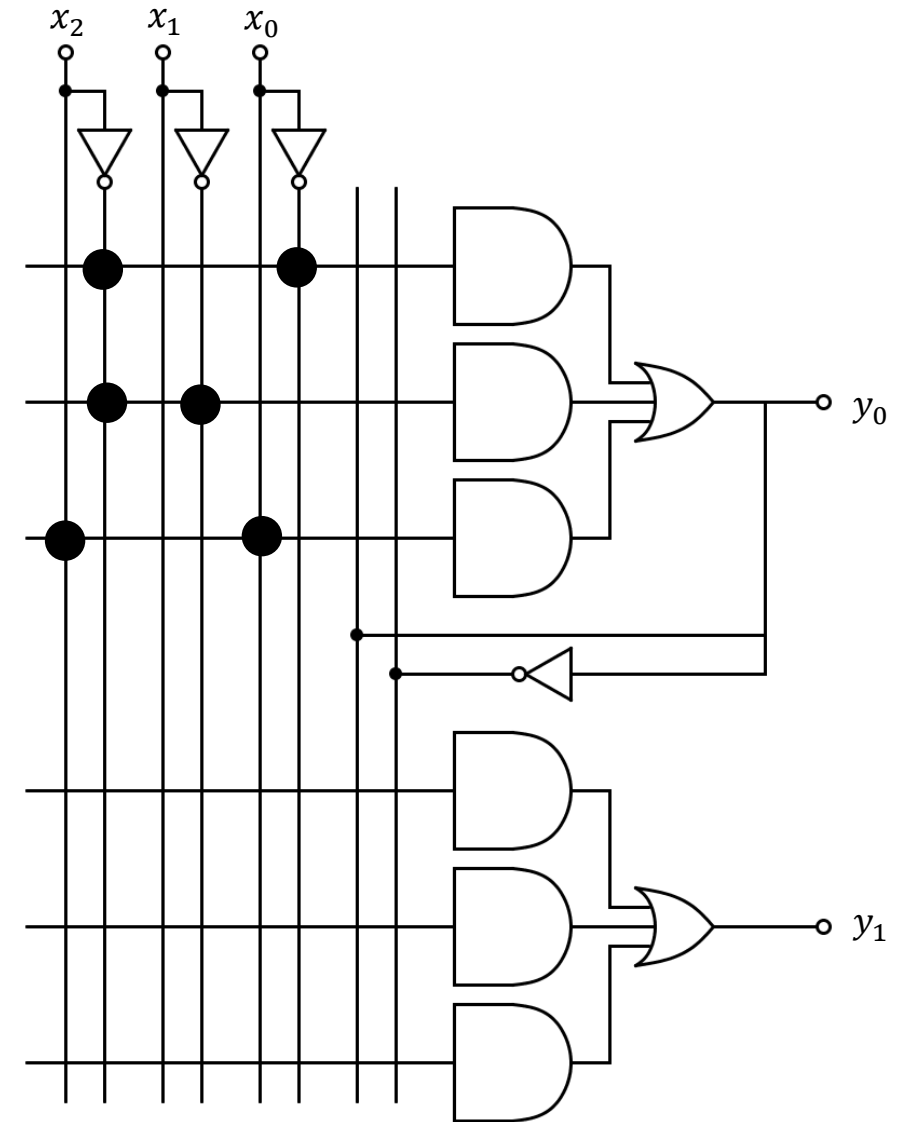
- PAL Example 1
 - 1 function, 5 minterms, 3 implicants
 - It fits, wastes one output and its 3 AND gates

Truth Table

x_2	x_1	x_0	y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$x_1 x_0$		00	01	11	10
x_2	0	1	1	0	1
	1	0	1	1	0

$$y = \overline{x_2} \overline{x_0} + \overline{x_2} \overline{x_1} + x_2 x_0$$



Programmable Array Logic (PAL)

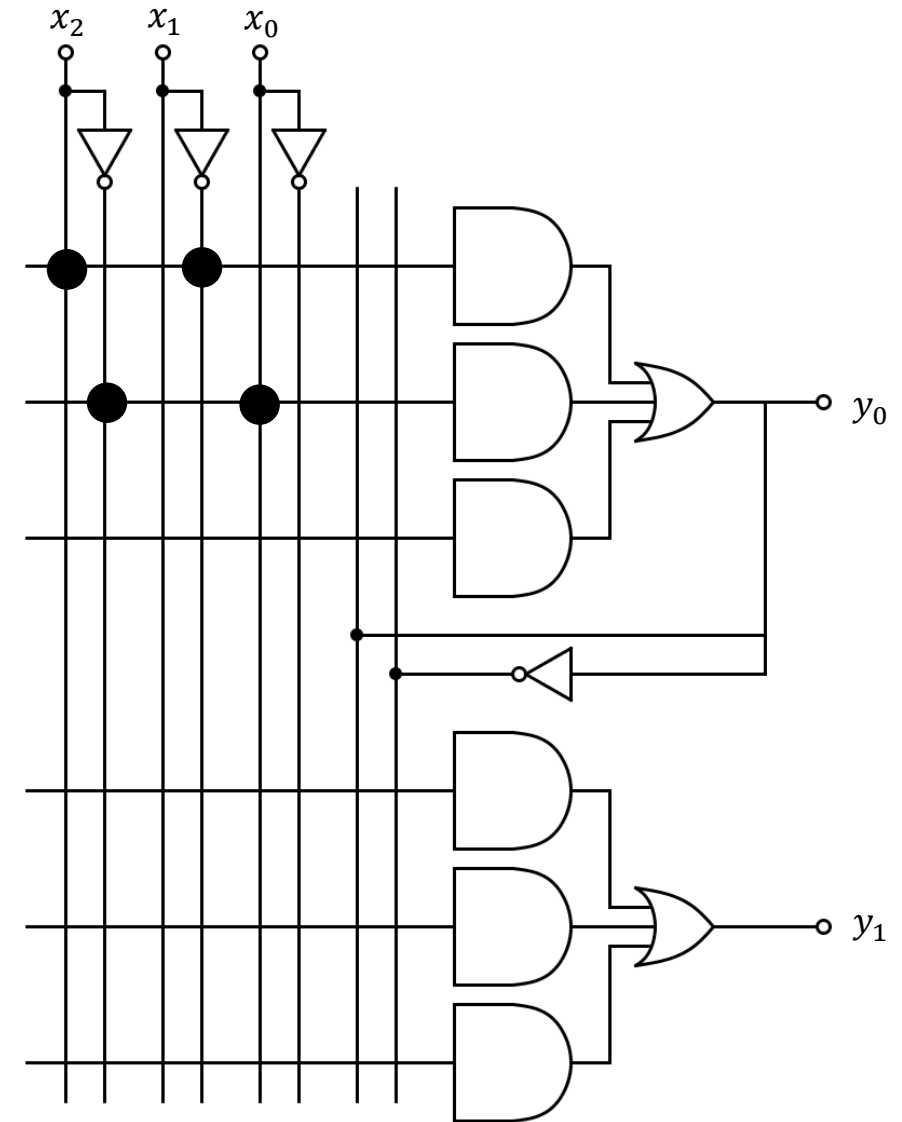
- PAL Example 2
 - 1 function, 4 minterms, 2 implicants
 - It fits, wastes one output (and the 3 AND gates) and one AND gate

Truth Table

x_2	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$x_1 x_0$		00	01	11	10
x_2	0	0	1	1	0
	1	1	1	0	0

$$y = x_2 \bar{x}_1 + \bar{x}_2 x_0$$



Programmable Array Logic (PAL)

• PAL Example 3

- 1 function, 4 minterms, 4 implicants
 - It doesn't fit in one set of AND gates
 - Need to use the feedback connection
 - Wastes one output (y_0) and one AND gate
 - Uses multiple logic levels (slower)

Truth Table

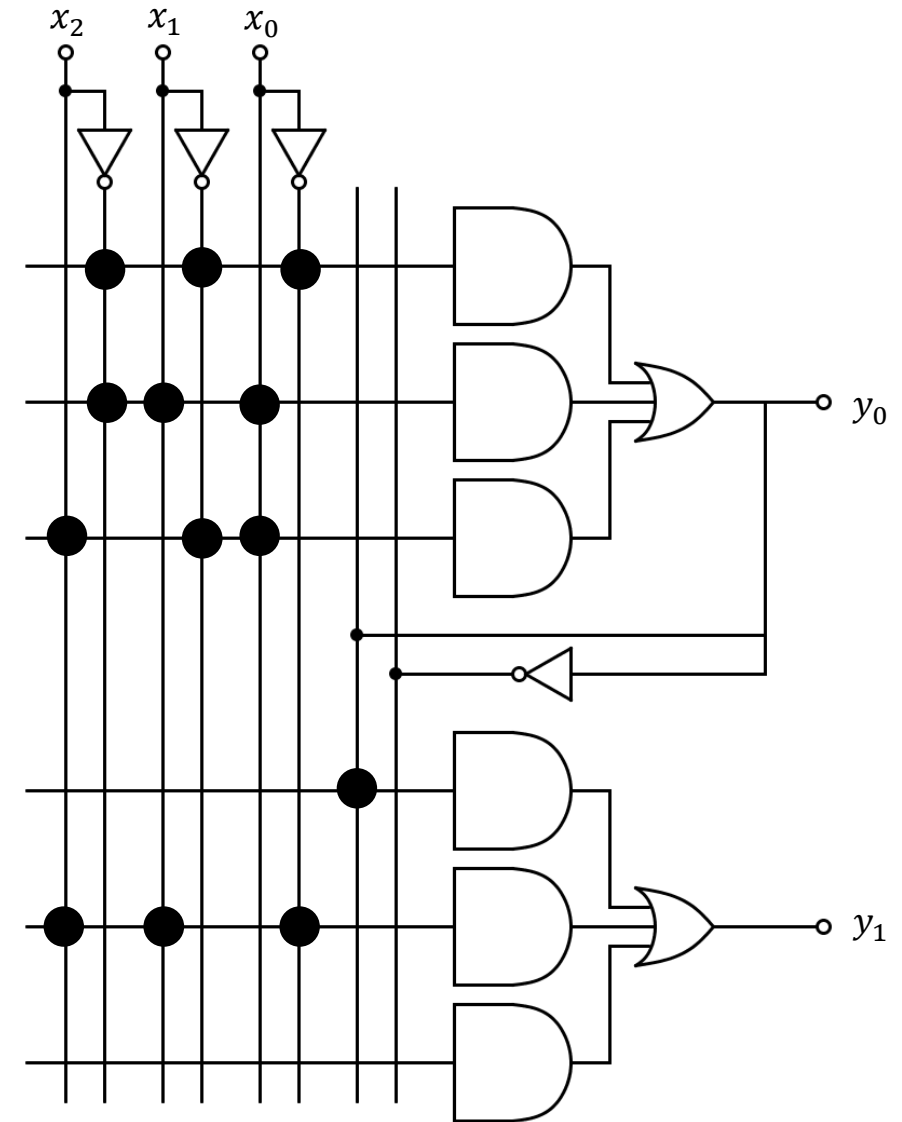
x_2	x_1	x_0	y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

		$x_1 x_0$			
		00	01	11	10
x_2	0	1	0	1	0
	1	0	1	0	1

$$y = \overline{x_2} \overline{x_1} \overline{x_0} + \overline{x_2} x_1 x_0 + x_2 \overline{x_1} x_0 + x_2 x_1 \overline{x_0}$$

$$y_0 = \overline{x_2} \overline{x_1} \overline{x_0} + \overline{x_2} x_1 x_0 + x_2 \overline{x_1} x_0$$

$$y_1 = y_0 + x_2 x_1 \overline{x_0}$$



Programmable Array Logic (PAL)

- PAL Example 4
 - 2 functions, 7 minterms, first two examples
 - They fit

Truth Table

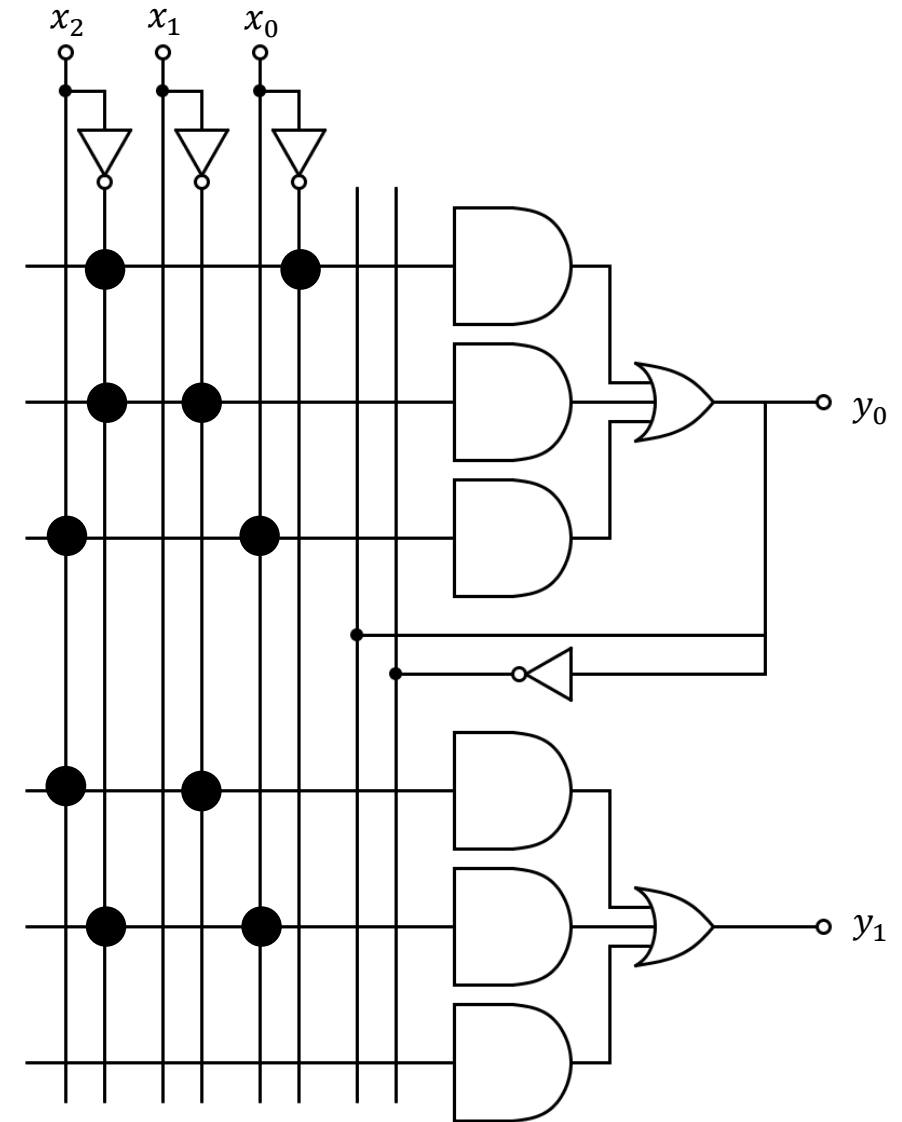
x_2	x_1	x_0	y_1	y_0
0	0	0	0	1
0	0	1	1	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	0
1	1	1	0	1

x_2	$x_1 x_0$			
	00	01	11	10
0	1	1	0	1
1	0	1	1	0

$$y_0 = \overline{x_2} \overline{x_0} + \overline{x_2} \overline{x_1} + x_2 x_0$$

x_2	$x_1 x_0$			
	00	01	11	10
0	0	1	1	0
1	1	1	0	0

$$y_1 = x_2 \overline{x_1} + \overline{x_2} x_0$$



Programmable Array Logic (PAL)

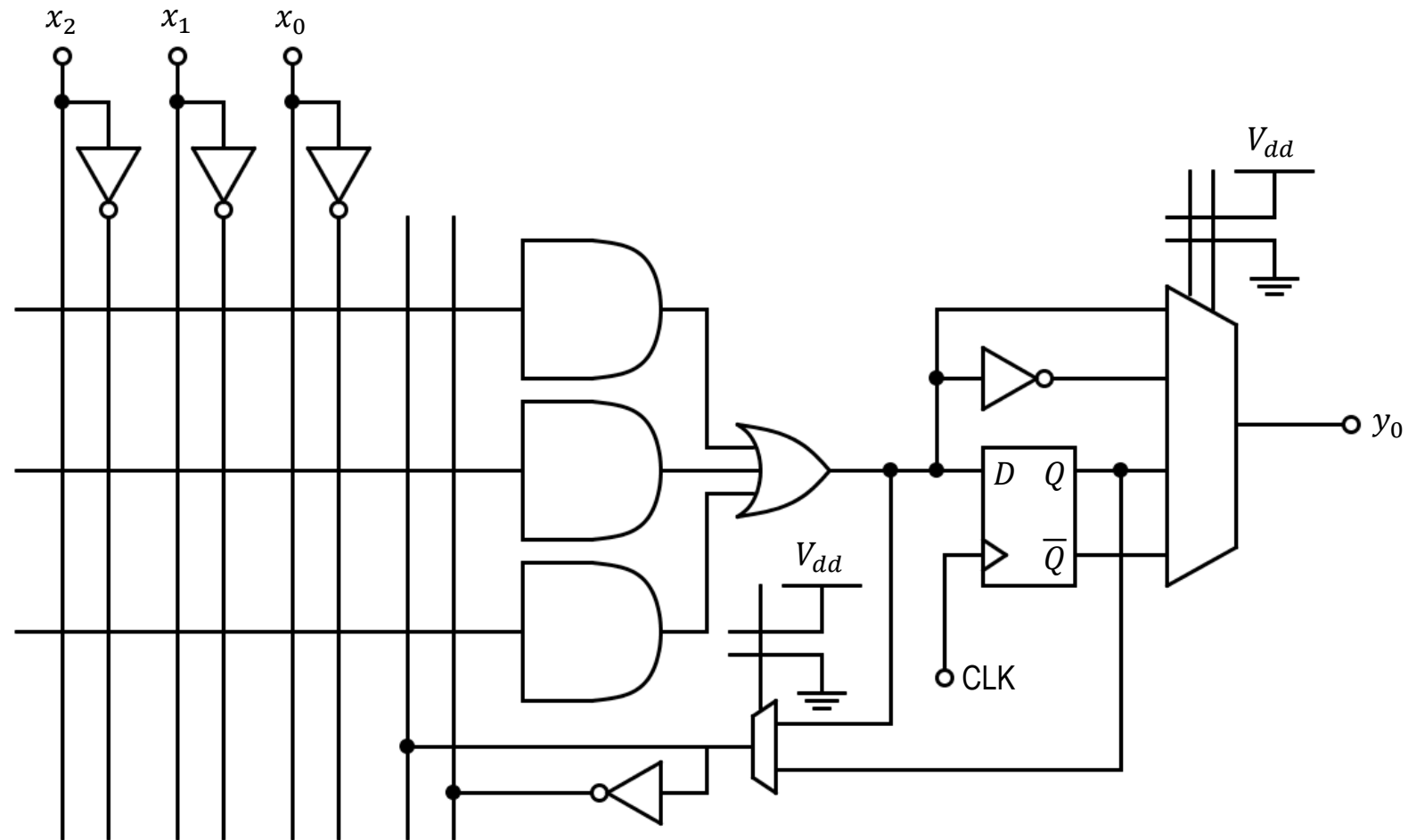
- PAL problems
 - Fewer functions can be implemented using a PAL with respect to a PLA
 - Cannot share implicants among functions
 - Feedback connections may help, but they use more resources
 - More expensive
 - And introduce higher delays
 - Lower performance
- Sequential logic cannot be implemented
 - Feedbacks can be used in principle to implement sequential logic
 - It works, but not used in practice
 - Slow
- Cannot increase the number of inputs and outputs
 - Same problem as in PLA

Macrocell and Macroblock

- Macrocell
 - A configurable logic circuit made of
 - A PLA or a PAL
 - A flip-flop added to each output
 - Multiplexers to select
 - Combinational bypass (direct and inverted)
 - Latched output (direct and inverted)
 - Only selection inputs of the multiplexers can be configured
 - Not the data inputs or outputs
 - Configured using the same technology as the combinational logic
 - Additional clock input
 - Feedback connection for PAL

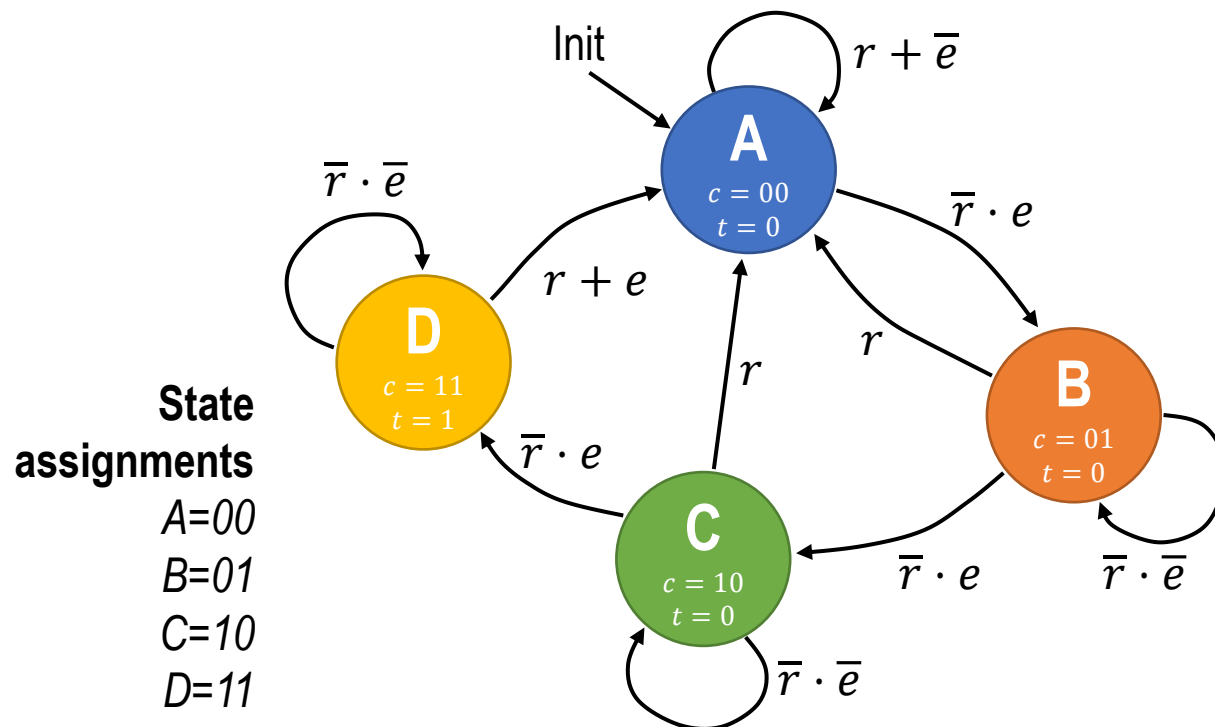
Macrocell and Macroblock

- Macrocell
 - Configure logic
 - Configure MUXes
- Macroblock
 - A collection of macrocells
 - Example: 16 macrocells
 - They share all inputs
 - They share the feedback connections
 - One distinct feedback for each macrocell in the macroblock
 - They share the clock
 - They have individual outputs



Macrocell and Macroblock

- Example
 - FSM
 - 2 bits counter with terminal count
 - Synchronous reset and enable signals



r	e	cs_1	cs_0		ns_1	ns_0	t	c_1	c_0
0	0	0	0	A	A	0	0	0	0
0	0	0	1	B	B	0	1	0	1
0	0	1	0	C	C	1	0	1	0
0	0	1	1	D	D	1	1	1	1
0	1	0	0	A	B	0	1	0	0
0	1	0	1	B	C	1	0	0	1
0	1	1	0	C	D	1	1	0	1
0	1	1	1	D	A	0	0	1	1
1	0	0	0	A	A	0	0	0	0
1	0	0	1	B	A	0	0	0	1
1	0	1	0	C	A	0	0	1	0
1	0	1	1	D	A	0	0	1	1
1	1	0	0	A	A	0	0	0	0
1	1	0	1	B	A	0	0	0	1
1	1	1	0	C	A	0	0	1	0
1	1	1	1	D	A	0	0	1	1

Macrocell and Macroblock

- Example
 - FSM
 - 2 bits counter with terminal count
 - Synchronous reset and enable signals

$cs_1 cs_0$					
re		00	01	11	10
		00	01	11	10
00		0	0	1	1
01		0	1	0	1
11		0	0	0	0
10		0	0	0	0

$$ns_1 = \bar{r} \bar{e} cs_1 + \bar{r} cs_1 \bar{cs}_0 + \bar{r} e \bar{cs}_1 cs_0$$

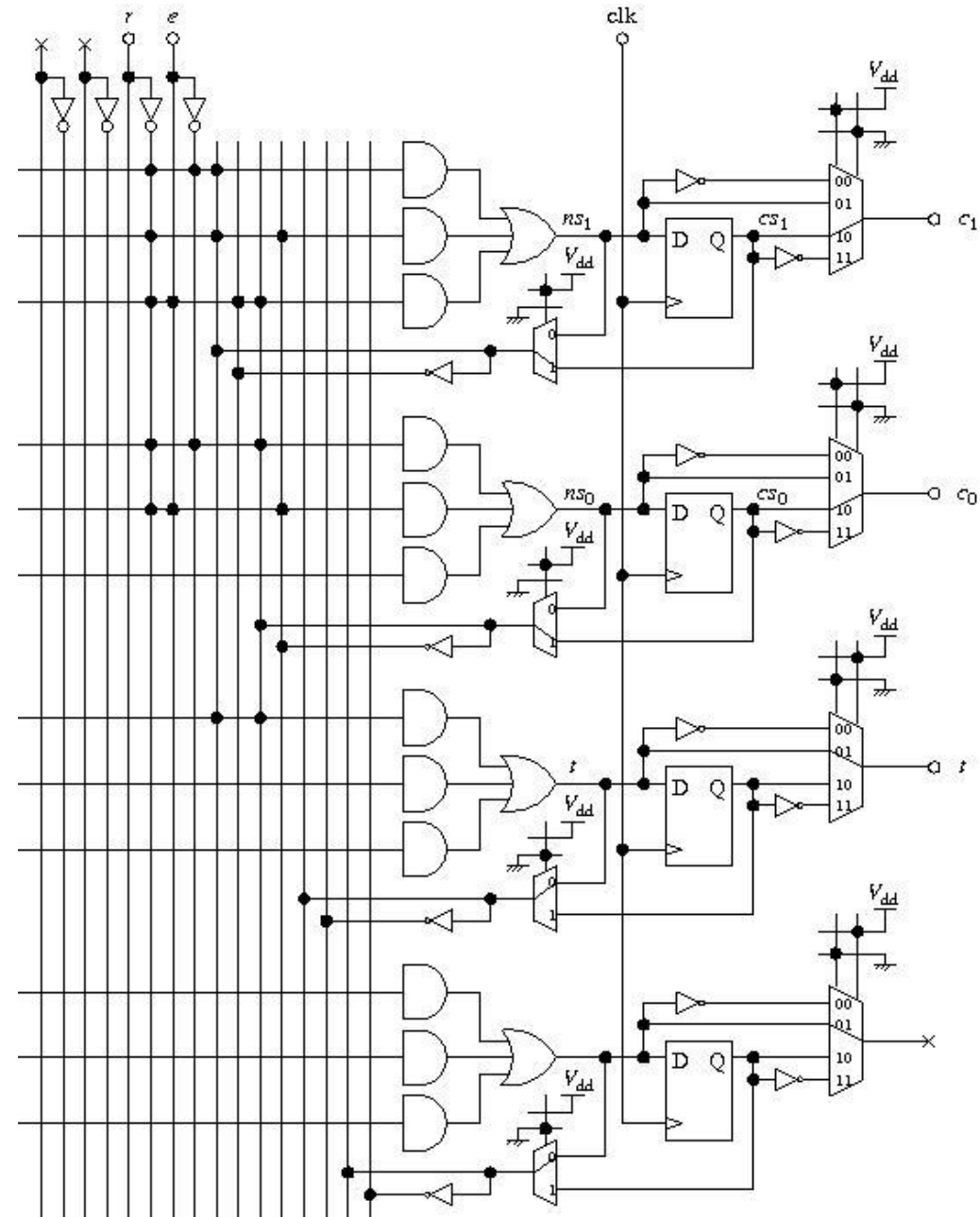
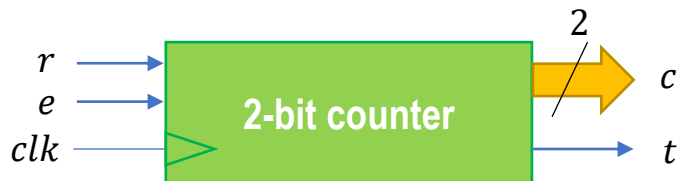
$$t = cs_1 cs_0$$

$$c_0 = cs_0$$

$$c_1 = cs_1$$

$cs_1 cs_0$					
re		00	01	11	10
		00	01	11	10
00		0	1	1	0
01		1	0	0	1
11		0	0	0	0
10		0	0	0	0

$$ns_0 = \bar{r} \bar{e} cs_0 + \bar{r} e \bar{cs}_0$$



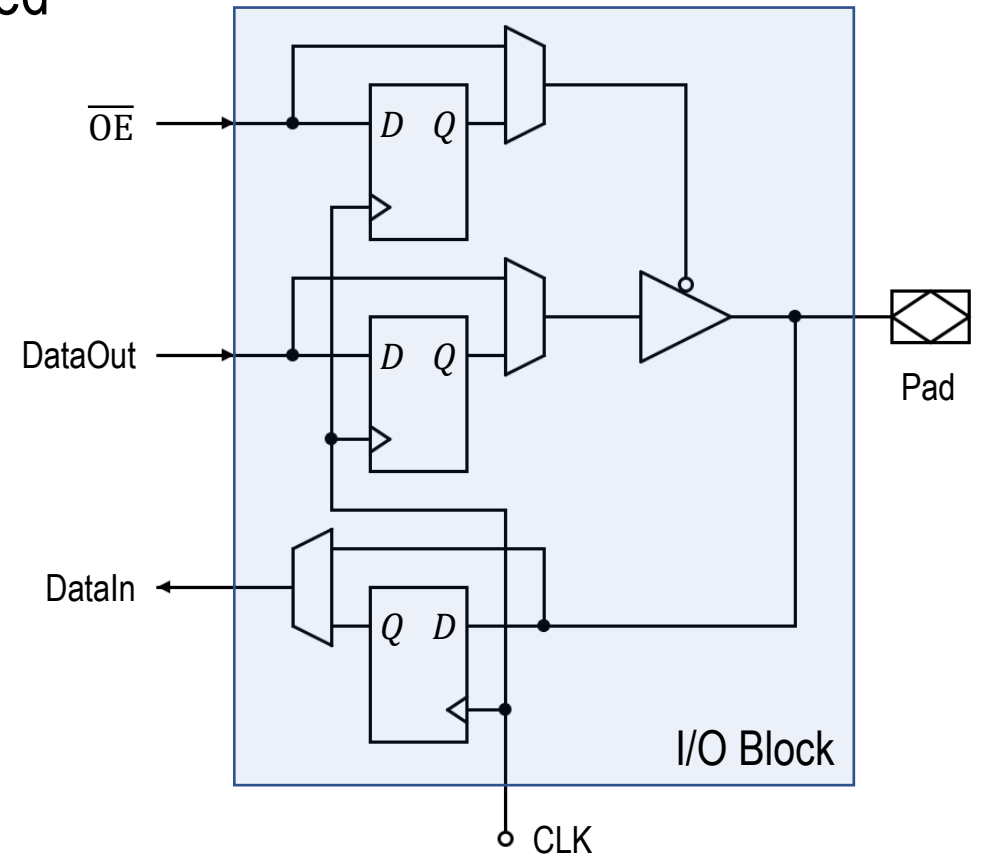
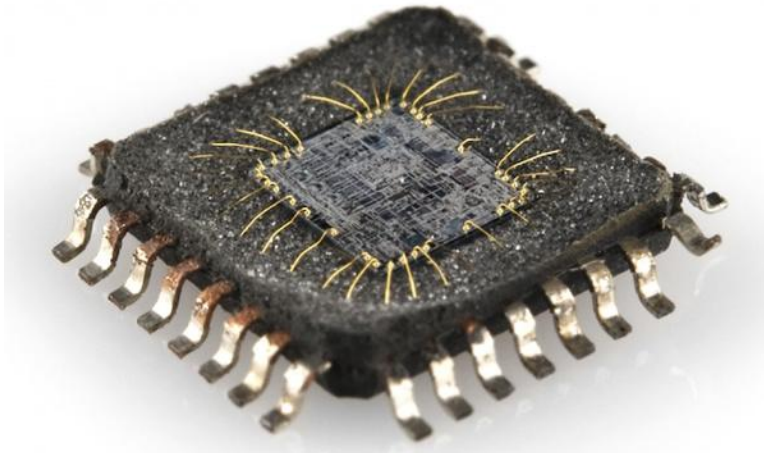
Complex Programmable Logic Device (CPLD)

- Complex circuits
 - The number of inputs and outputs increases
 - Arithmetic operations usually require many signals
 - A 32-bit adder requires 65 inputs and 33 outputs
 - Delays increase as the number of inputs grows
 - Functions operate on a subset of the total number of inputs
 - It is not necessary to have a big PAL (or PLA) with all functions implemented
 - Group together several Macrocells to form a logic unit
 - Group of Macrocells → Macroblock
 - Group together several Macroblocks to form a Complex Programmable Logic Device (CPLD)
 - Programmable interconnections to route signal from one Macrocell to another
 - Within the same logic unit
 - Between different logic units

Complex Programmable Logic Device (CPLD)

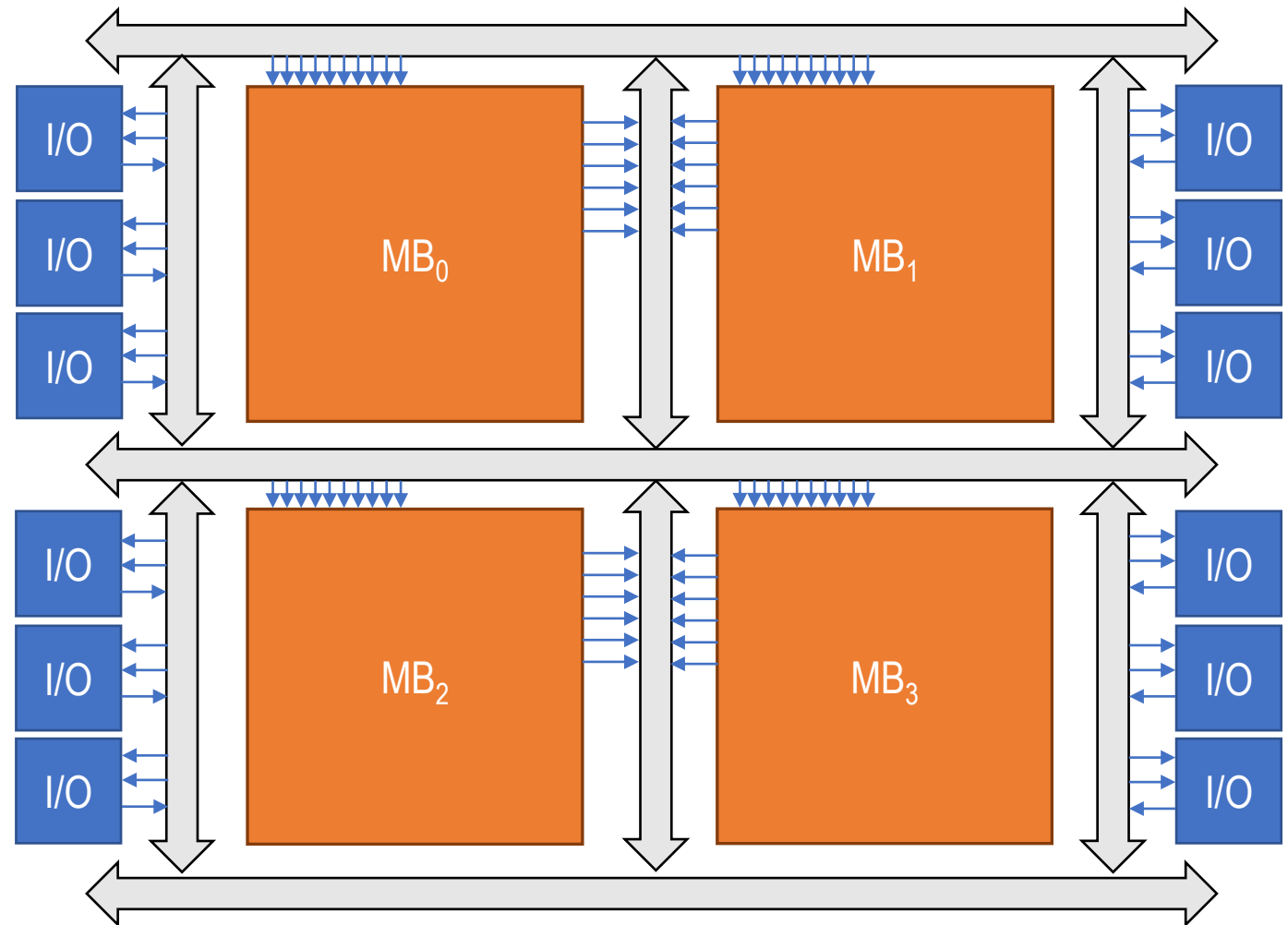
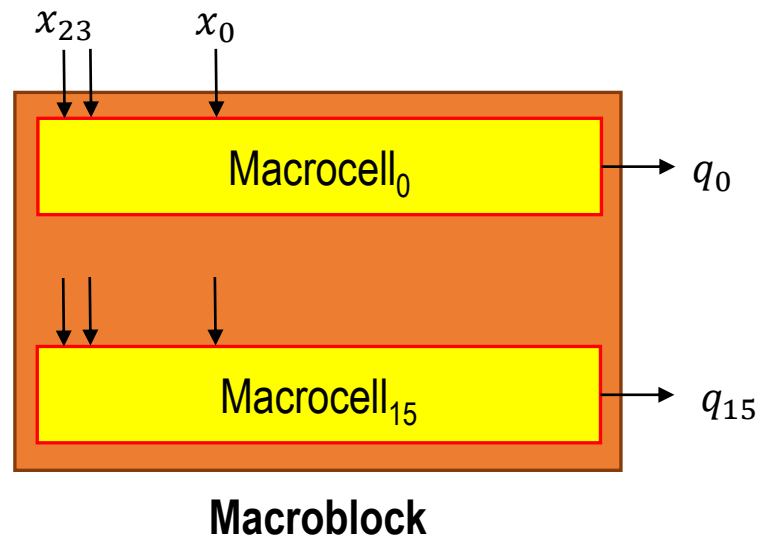
- CPLD

- Inputs and outputs need to be defined and connected
 - I/O Block to configure a pin (input, output, bidir)
 - Programmable interconnections to route signal
- I/O block structure
 - Buffer with enable signal
 - MUXes controlled by programmable connections



Complex Programmable Logic Device (CPLD)

- CPLD overall structure
 - Programmable routing
 - Several wire segments
 - Programmable interconnections between segments



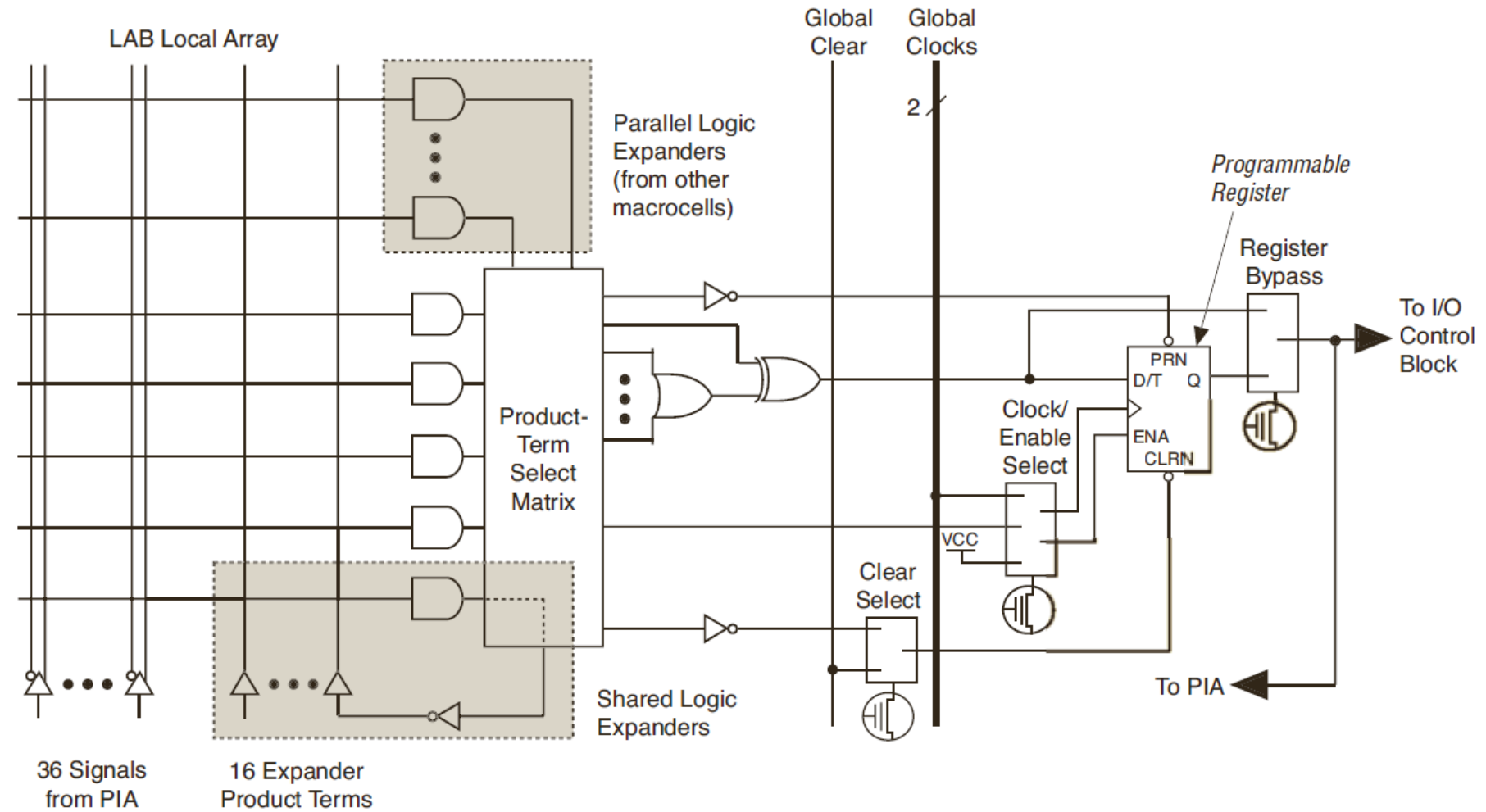
Complex Programmable Logic Device (CPLD)

- Altera MAX 3000A
 - Family of different sizes
 - Number of Macrocells, number of user I/O pins
 - Choose the smallest device in which your circuit fits

Table 1. MAX 3000A Device Features					
Feature	EPM3032A	EPM3064A	EPM3128A	EPM3256A	EPM3512A
Usable gates	600	1,250	2,500	5,000	10,000
Macrocells	32	64	128	256	512
Logic array blocks	2	4	8	16	32
Maximum user I/O pins	34	66	98	161	208
t_{PD} (ns)	4.5	4.5	5.0	7.5	7.5
t_{SU} (ns)	2.9	2.8	3.3	5.2	5.6
t_{CO1} (ns)	3.0	3.1	3.4	4.8	4.7
f_{CNT} (MHz)	227.3	222.2	192.3	126.6	116.3

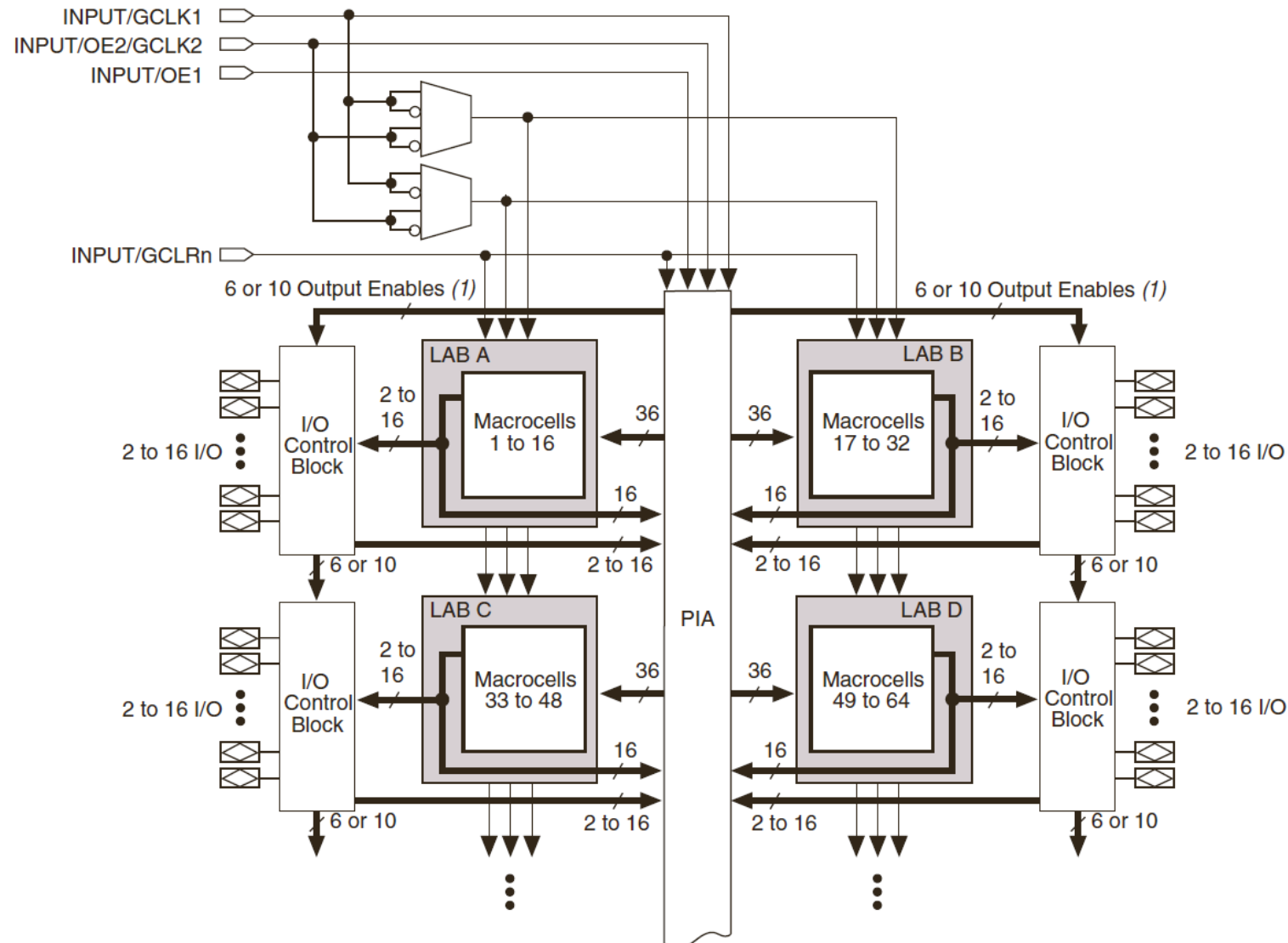
Complex Programmable Logic Device (CPLD)

- Altera MAX 3000A
 - Single Macrocell



Complex Programmable Logic Device (CPLD)

- Altera MAX 3000A
 - General architecture
 - Number of macroblocks varies with the family instance of the device
 - Number of I/O Control Blocks varies with the number of pins of the package
 - PIA: Programmable Interconnect Array



Complex Programmable Logic Device (CPLD)

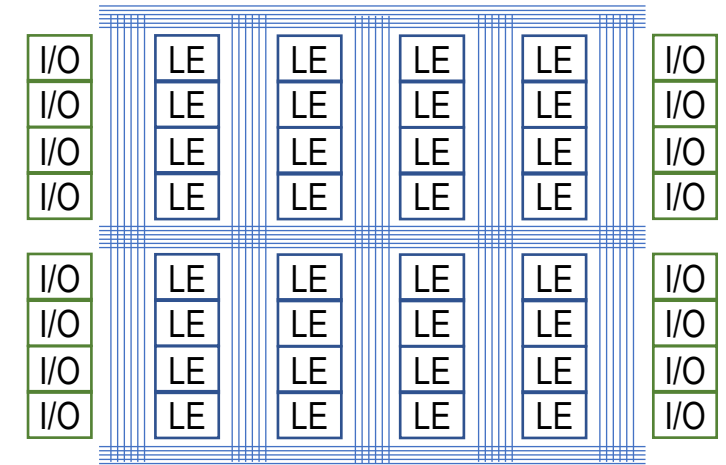
- CPLD problems
 - Number of flip-flops
 - Cannot implement complex finite state machines
 - Cannot implement complex sequential data paths
 - Cannot implement a processor!
 - Requires lots of Flip Flops for the FSM in the control unit
 - Requires lots of Flip Flops for registers in the data path
 - CPLDs used for glue logic
 - Configured at low granularity
 - Small circuits waste a lot of area

Field Programmable Gate Array (FPGA)

- Many digital circuits need large storage for temporary data
 - Pipeline registers, shift registers, counters, FSM registers, ...
 - Memories, FIFOs, register files, ...
 - CPLDs do not provide enough storage
- Field Programmable Gate Array (FPGA)
 - Programmable Logic Device based on Look-Up Tables (LUT)
 - A LUT is a small memory used to implement combinational logic functions
 - From 3 to 6 inputs and 1 or 2 outputs
 - Limited number of inputs to avoid the exponential explosion
 - Often implemented using SRAM technology
 - Each LUT can have an associated FF
 - LUTs and FFs are grouped in a Configurable Logic Block (CLB) or Logic Element (LE)
 - 1÷ 6 LUTs, 1÷ 6 FFs

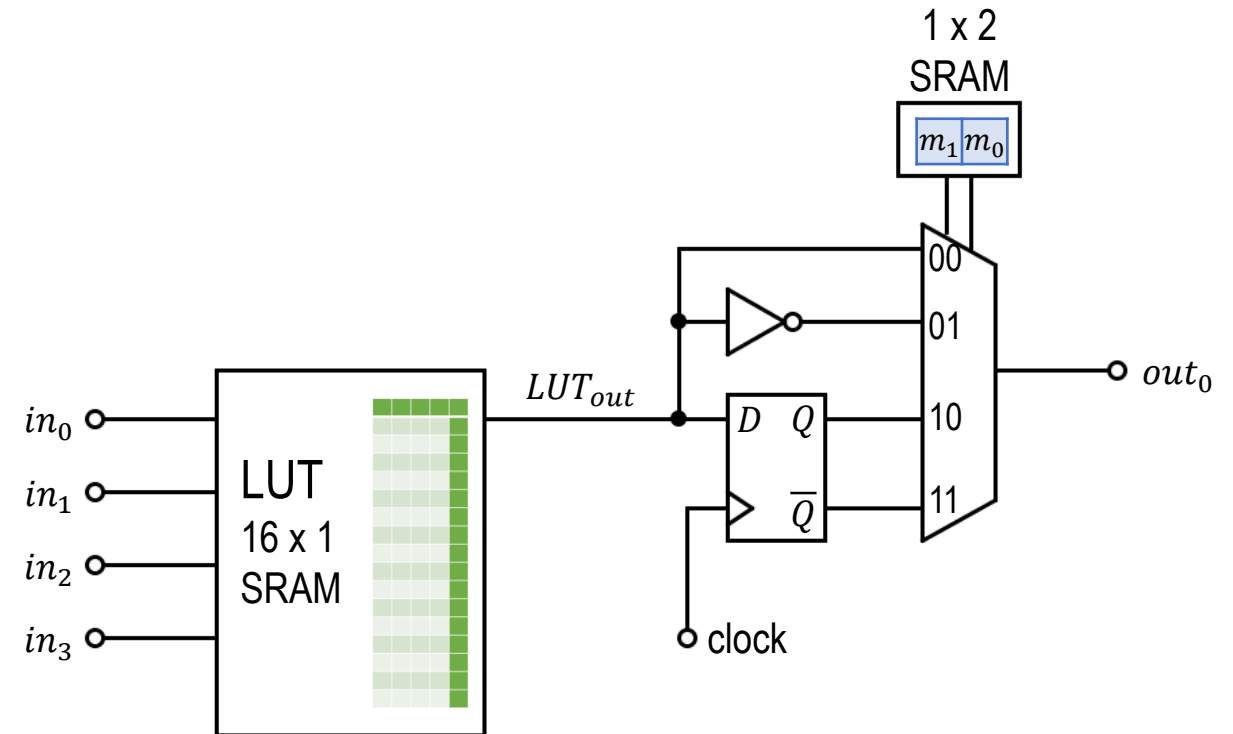
Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - A collection of a large number of CLBs or LEs
 - CLB or LE smaller than a macrocell
 - Organized as a matrix array of rows and columns
 - Programmable interconnections allow to route signals from one CLB to another
 - Wires running between rows and columns
 - Configurable I/O blocks allow to connect to the external world
 - Similar to CPLD example



Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - Logic Element
 - Many different designs
 - The figure shows a simple example
 - One 4-input 1-output LUT
 - 16 SRAM bit cells
 - Normally in reading
 - Written only during configuration
 - One Flip Flop
 - Multiplexer to select the output value
 - Combinational or sequential output
 - MUX configuration based on the same technology as LUT (SRAM in most cases)

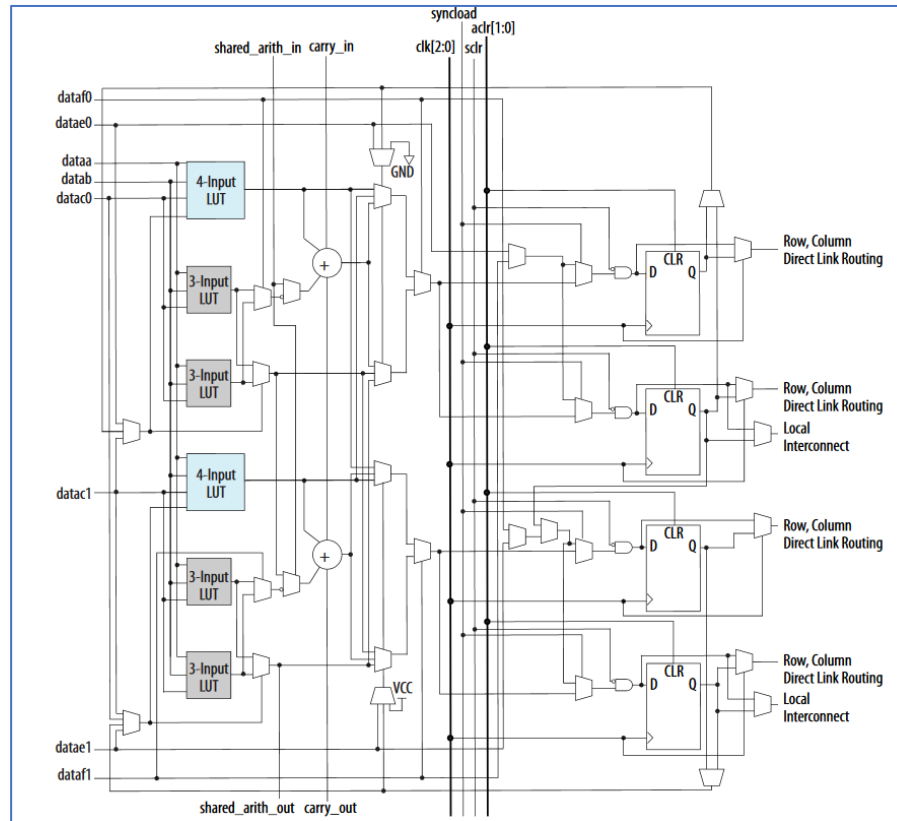


Field Programmable Gate Array (FPGA)

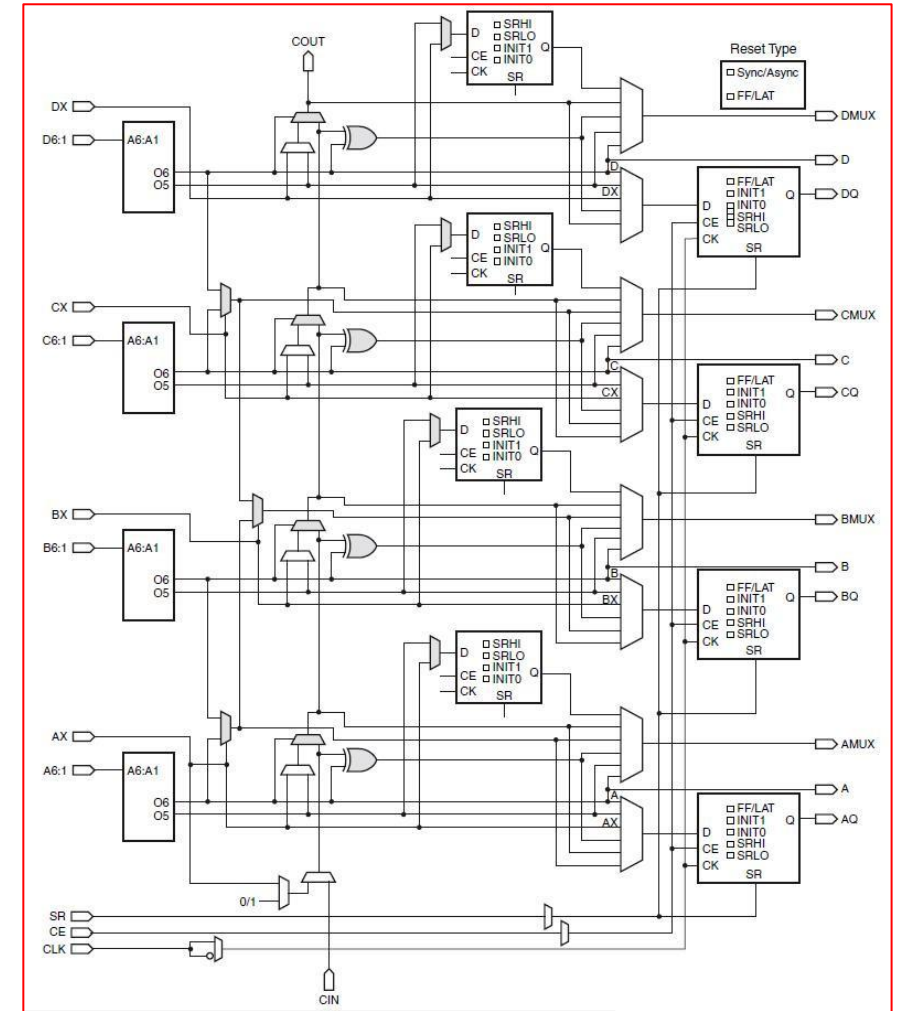
- Field Programmable Gate Array (FPGA)

- Logic Element

- Advanced LEs and CLBs from Intel/Altera and AMD/Xilinx



Intel/Altera ALM



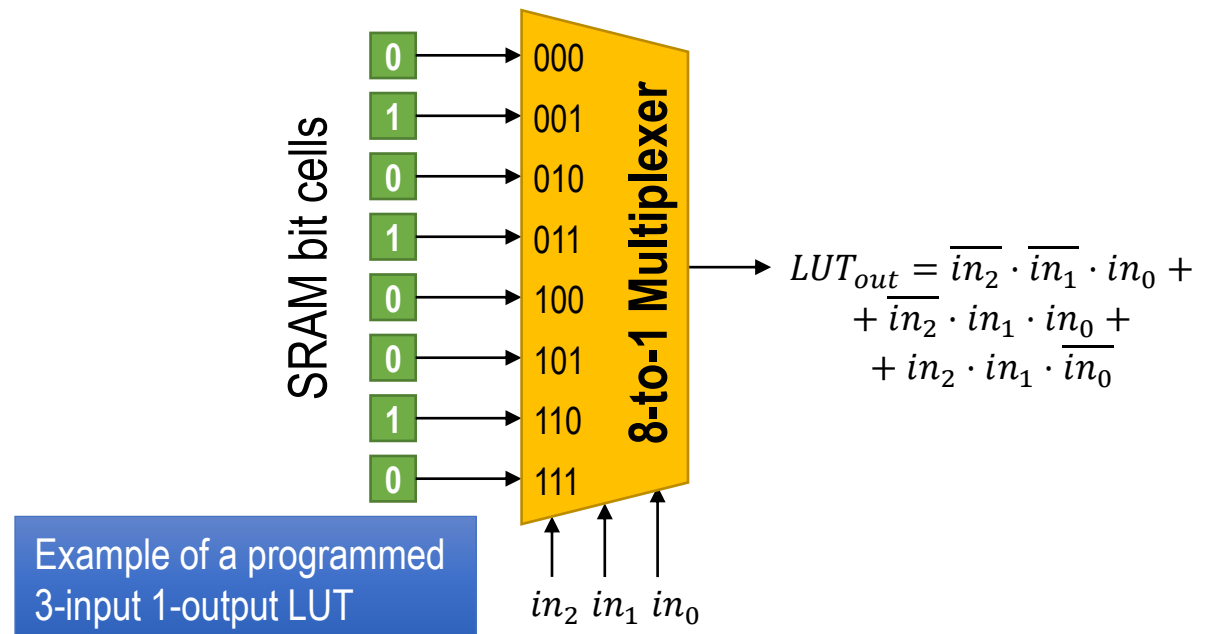
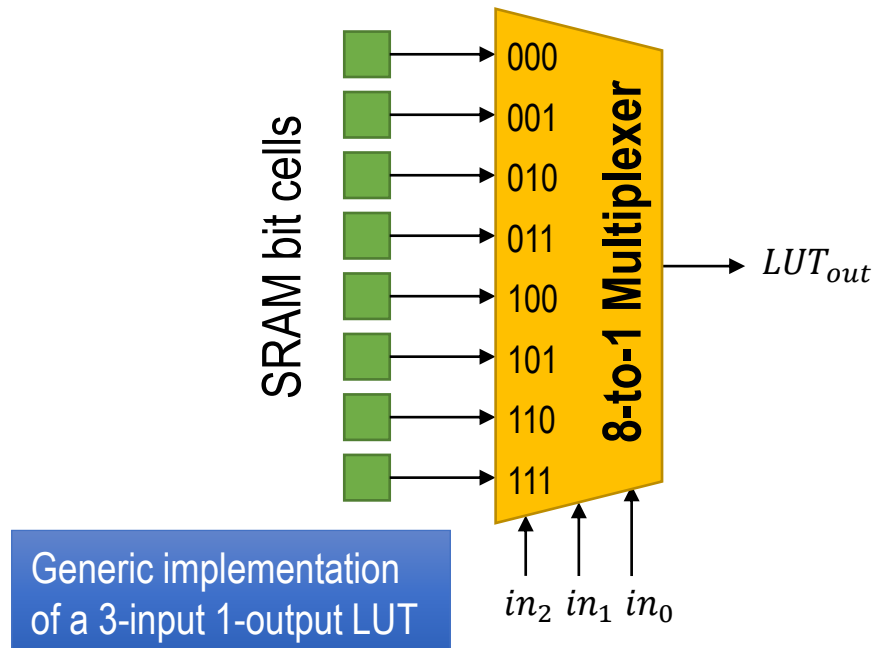
AMD/Xilinx CLB

Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - Look-Up Table
 - Implements an n inputs 1 output combinational logic circuit
 - 2^n SRAM bit cells
 - $2^n \rightarrow 1$ multiplexer, with n selection inputs

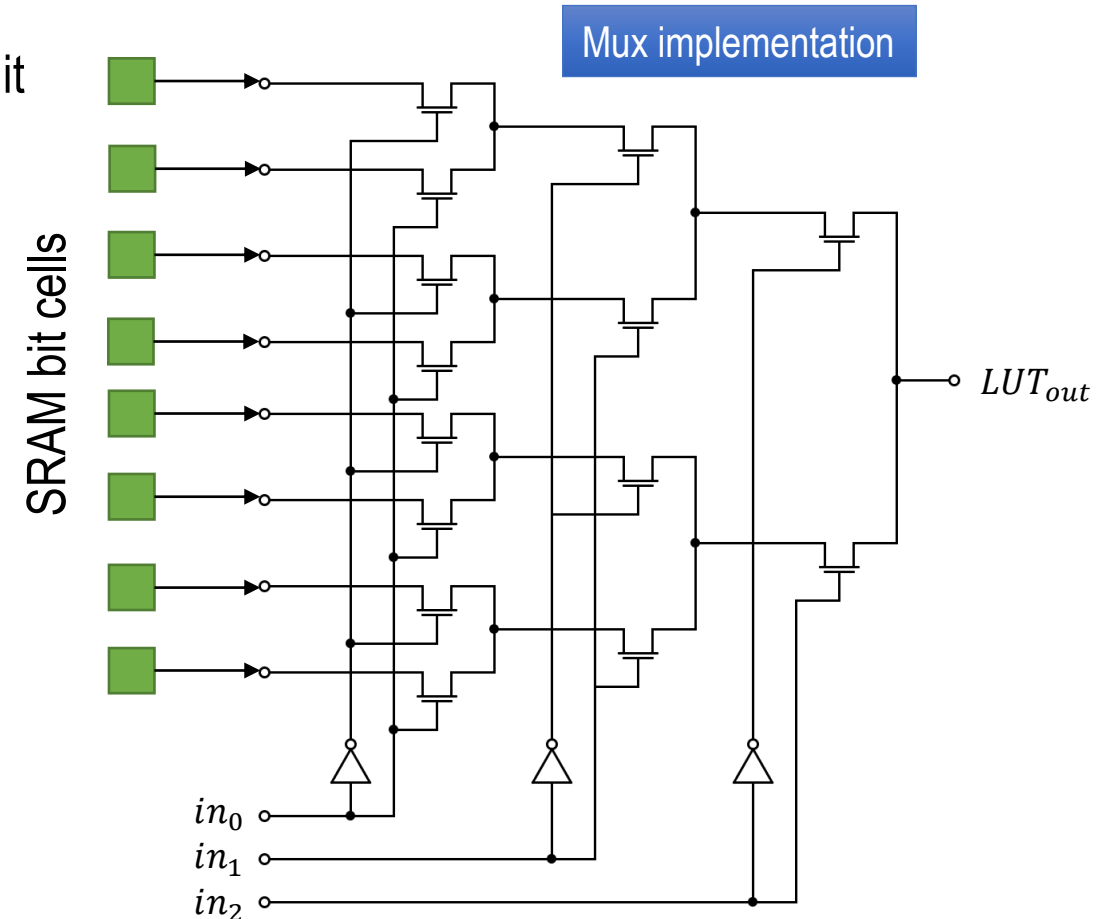
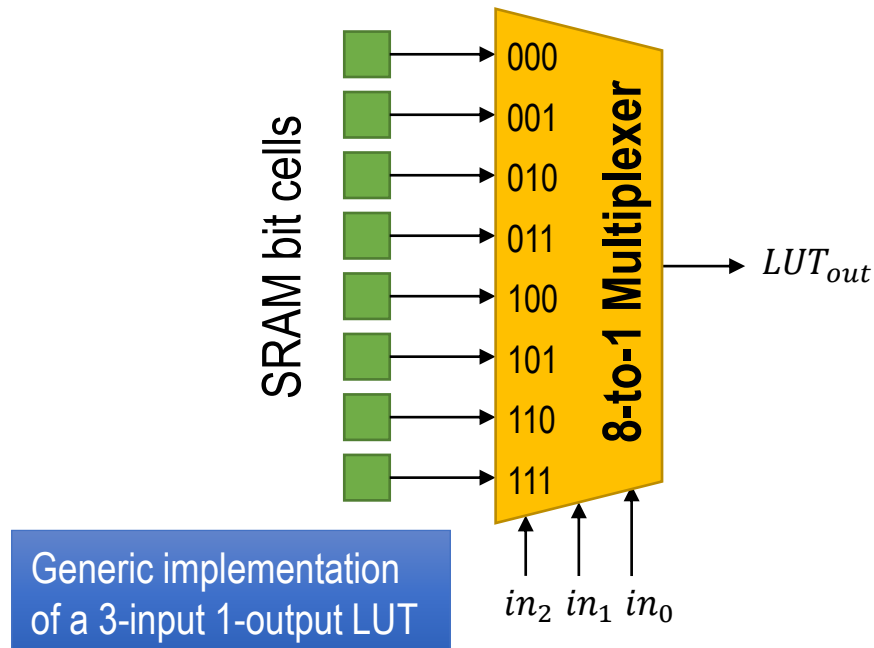
An n -input 1-output LUT is like a $2^n \times 1$ memory with:

- A single row
 - No need of a row decoder
 - The row is always selected
- 2^n columns
 - The MUX is the column decoder and column multiplexer of the memory



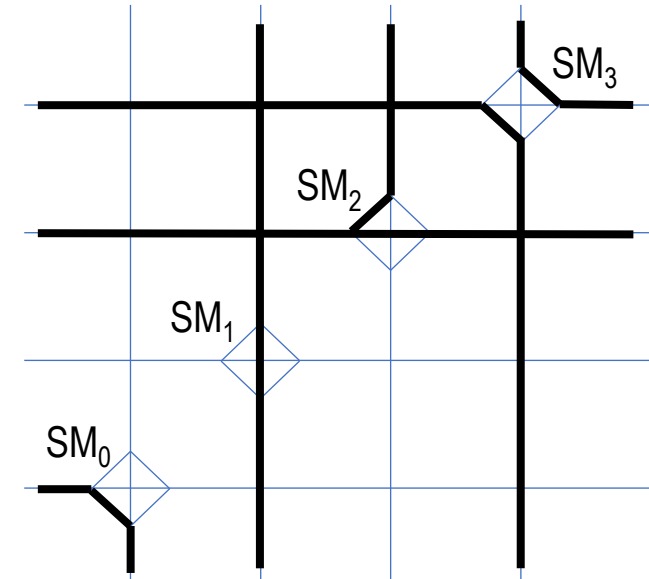
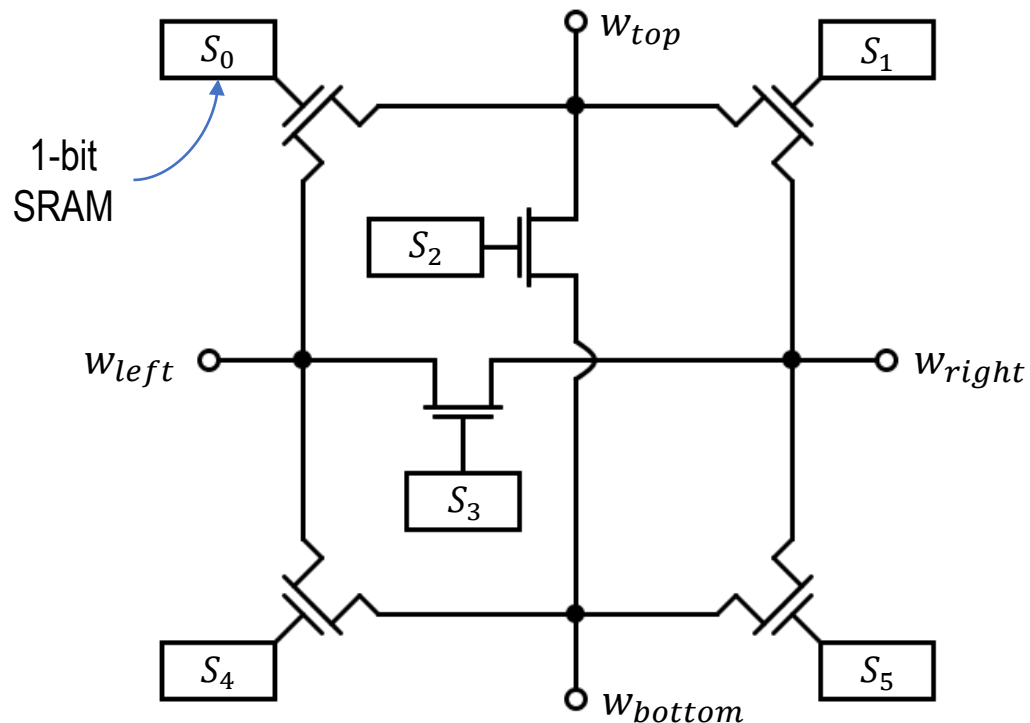
Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - Look-Up Table
 - Implements an n inputs 1 output combinational logic circuit
 - 2^n SRAM bit cells
 - $2^n \rightarrow 1$ multiplexer, with n selection inputs



Field Programmable Gate Array (FPGA)

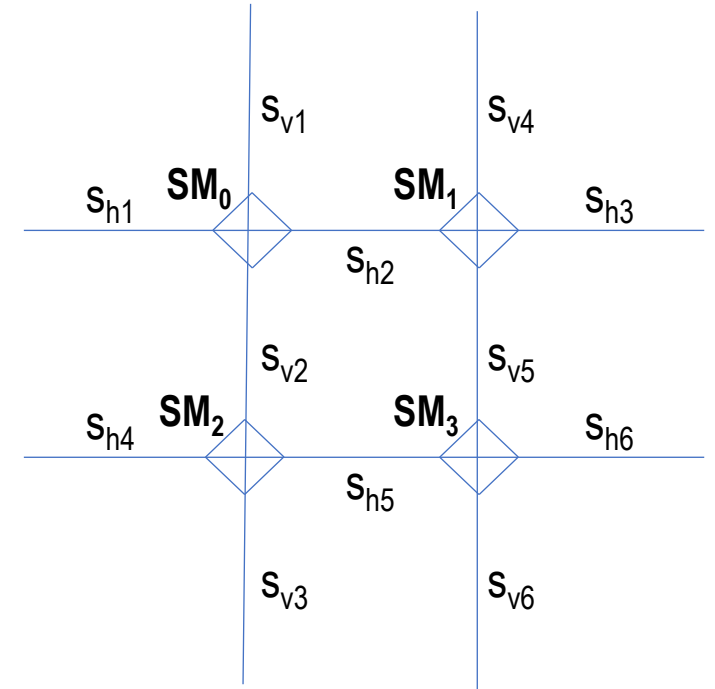
- Field Programmable Gate Array (FPGA)
 - Switch Matrix
 - Example of programmable interconnection
 - 6 pass transistors



	SM_0	SM_1	SM_2	SM_3
S_0	0	0	1	0
S_1	0	0	0/1	1
S_2	0	1	0	0
S_3	0	0	1	0
S_4	1	0	0	1
S_5	0	0	0	0

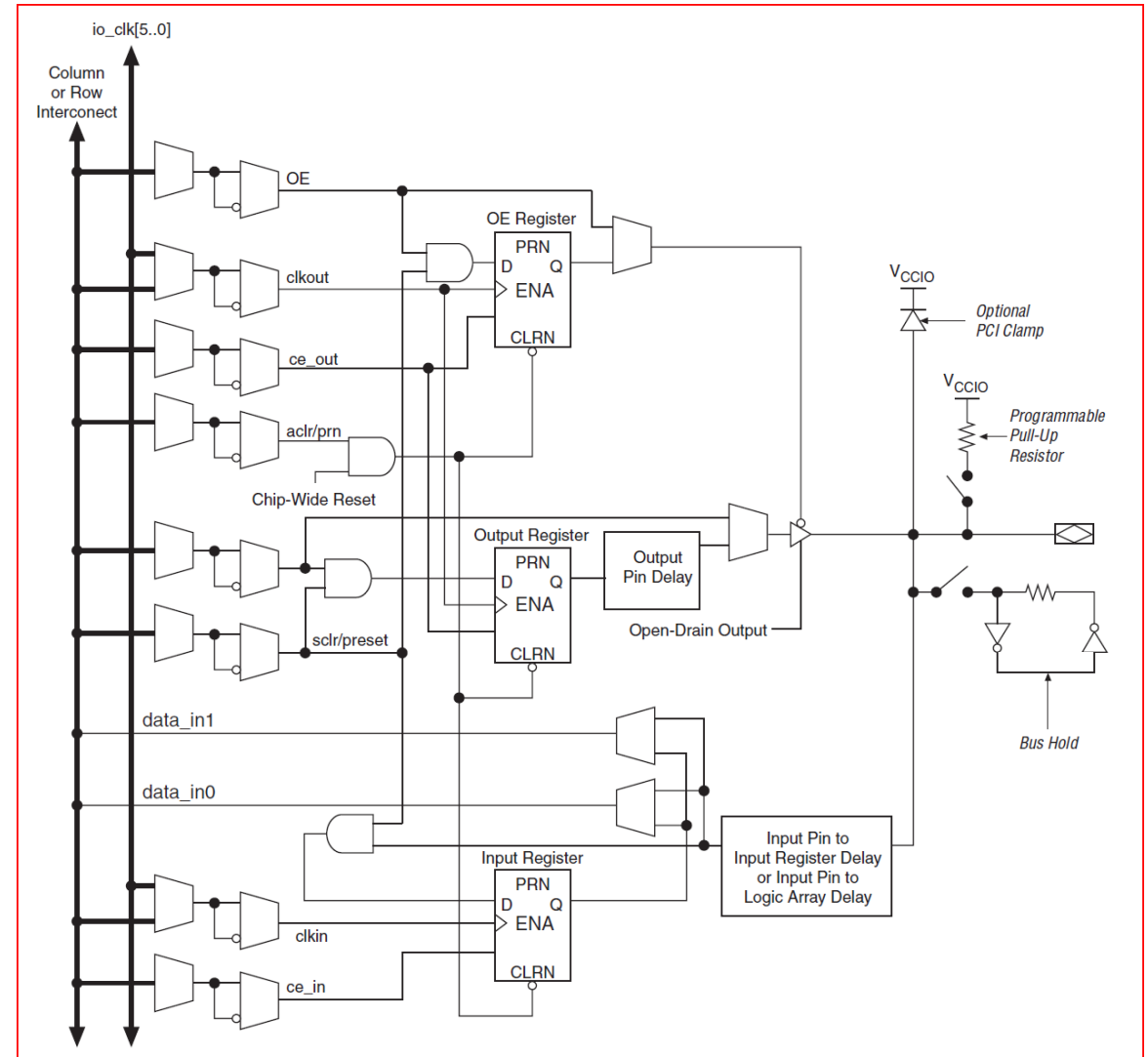
Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - Switch Matrix
 - Divides wires into independent segments
 - Segments are connected by properly activating transistors in the switch matrix
 - If not connected, adjacent segments may belong to different interconnections
 - No conflicts because inactive transistors separate them
 - Performance
 - A switch matrix introduces delays and increases the area
 - Limit the number of switch matrices to improve performance and decrease costs
 - A switch matrix increases the flexibility
 - More switch matrices make routing easier
 - Find a trade-off between performance, costs and flexibility



Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)
 - Input/Output Block
 - Output buffer with Enable signal
 - Combinational or sequential inputs and outputs
 - Optional Pull-Up, Clamp and Bus Hold circuits
 - Configurable by controlling the selection signals of multiplexers

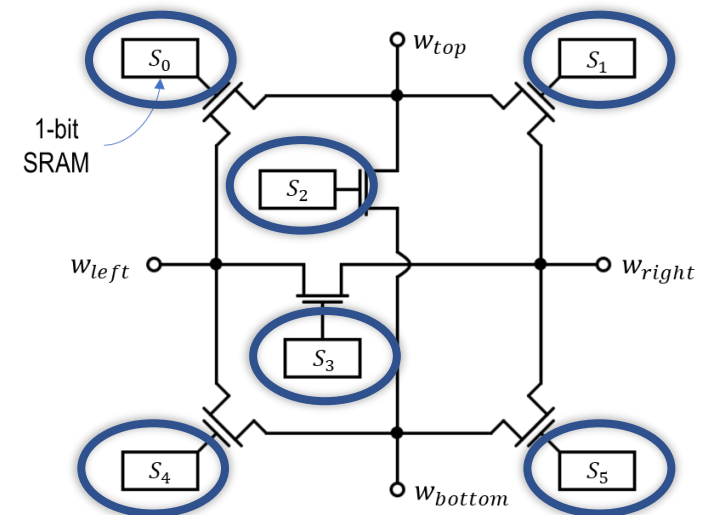
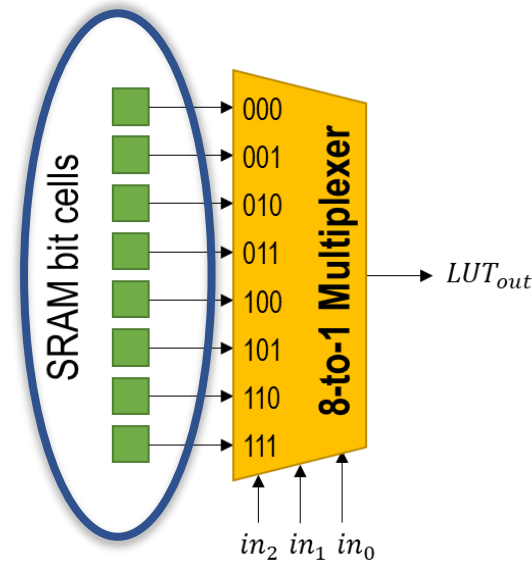
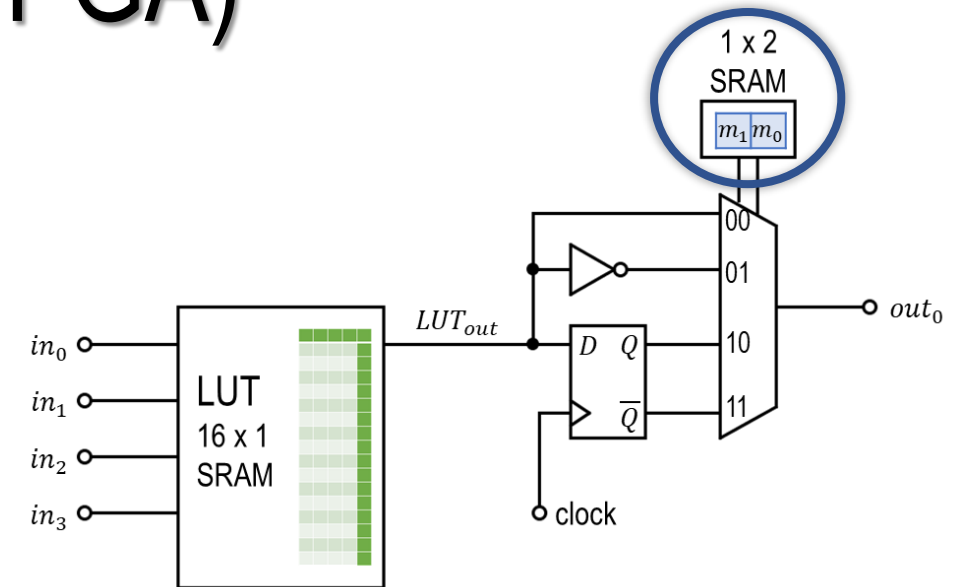


Field Programmable Gate Array (FPGA)

- Field Programmable Gate Array (FPGA)

- SRAM cells

- They are present in
 - Look-Up Tables (LUTs)
 - Selection signals of the multiplexers in the LE
 - Selection signals of the multiplexers in the Input/Output blocks
 - Switch matrices
- Written only during configuration
- Otherwise they are only read
- Each of them has a single row
 - No need to specify a row address
 - No need of row decoders
 - Need column decoders only in LUTs
 - Simplified circuit compared to a standard SRAM



Field Programmable Gate Array (FPGA)

• Examples

• Multiplexer

- A 4-input 1-output LUT is a 16 x 1 memory
- LUT content identical to the one presented for the ROM based PLD
 - Multiple different pin assignments
 - Different content
 - Same function
- A multiplexer is combinational
 - Need to configure the output multiplexer of the Logic Element
 - Route the combinational output LUT_{out} of the LUT to the Logic Element output out_0
 - $m_1m_0 = 00$
 - The output multiplexer of the Logic Element cannot be used as a general multiplexer

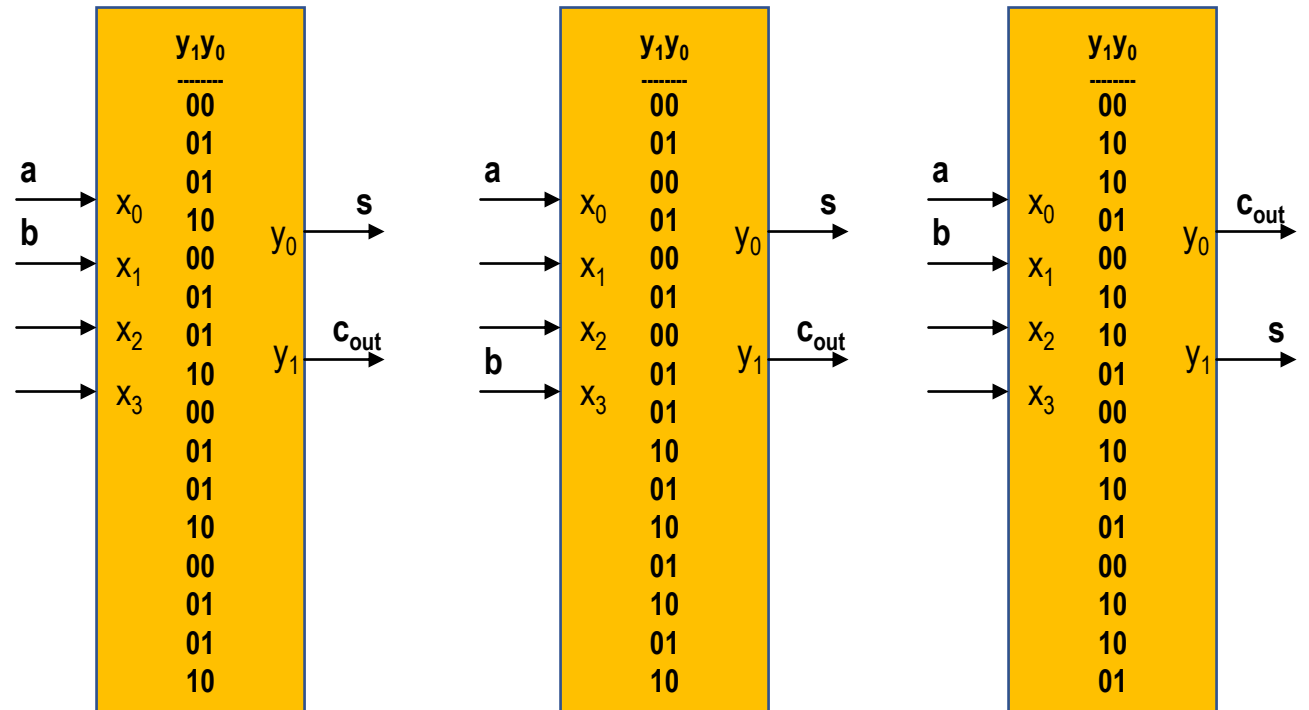
LUT content

—	<i>s</i>	<i>b</i>	<i>a</i>	<i>y</i>
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Field Programmable Gate Array (FPGA)

- Example
 - Half Adder
 - Using a 4-input 2-outputs LUT
 - The LUT is larger than necessary
 - Make unused inputs *don't care*
 - Content depends on the pin assignment

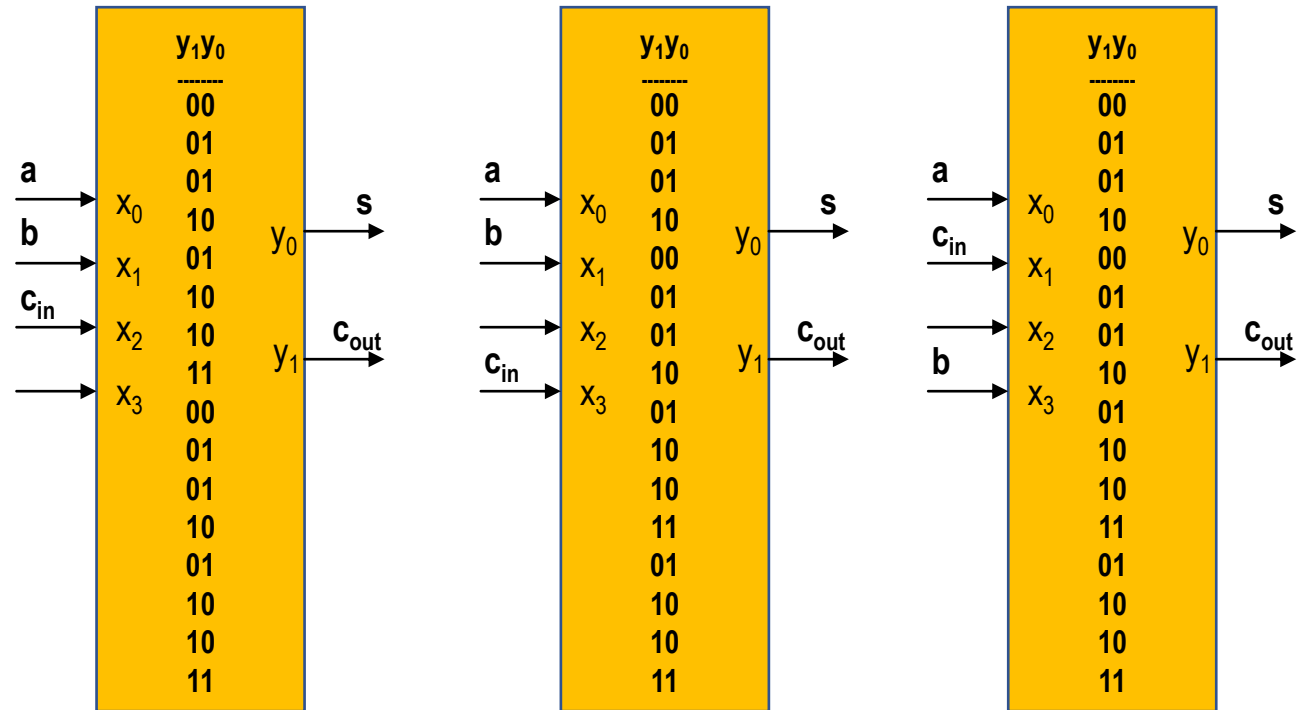
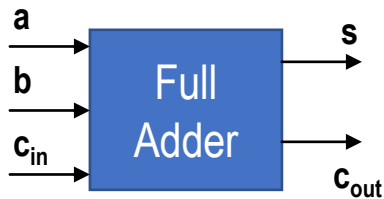
b	a	s	c _{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Field Programmable Gate Array (FPGA)

- Example
 - Full Adder
 - Using a 4-input 2-outputs LUT
 - The LUT is larger than necessary
 - Make unused inputs *don't care*
 - Content depends on the pin assignment

c _{in}	b	a	s	c _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Field Programmable Gate Array (FPGA)

- Example

- Combinational function with 5 inputs on a 4-input 1-output LUT
 - $y = f(x_4, x_3, x_2, x_1, x_0)$
- Can be rewritten as
 - $y = x_4 \cdot f(1, x_3, x_2, x_1, x_0) + \overline{x_4} \cdot f(0, x_3, x_2, x_1, x_0)$
 - $y' = f(1, x_3, x_2, x_1, x_0) = f'(x_3, x_2, x_1, x_0)$
 - $y'' = f(0, x_3, x_2, x_1, x_0) = f''(x_3, x_2, x_1, x_0)$
 - $\rightarrow y = x_4 \cdot y' + \overline{x_4} \cdot y''$
- Functions f' and f'' have only 4 inputs and 1 output
 - They fit into a LUT each
- The last function is a multiplexer with x_4 as the selection signal
 - Need an additional LUT
- The process can be repeatedly applied to functions with even more inputs

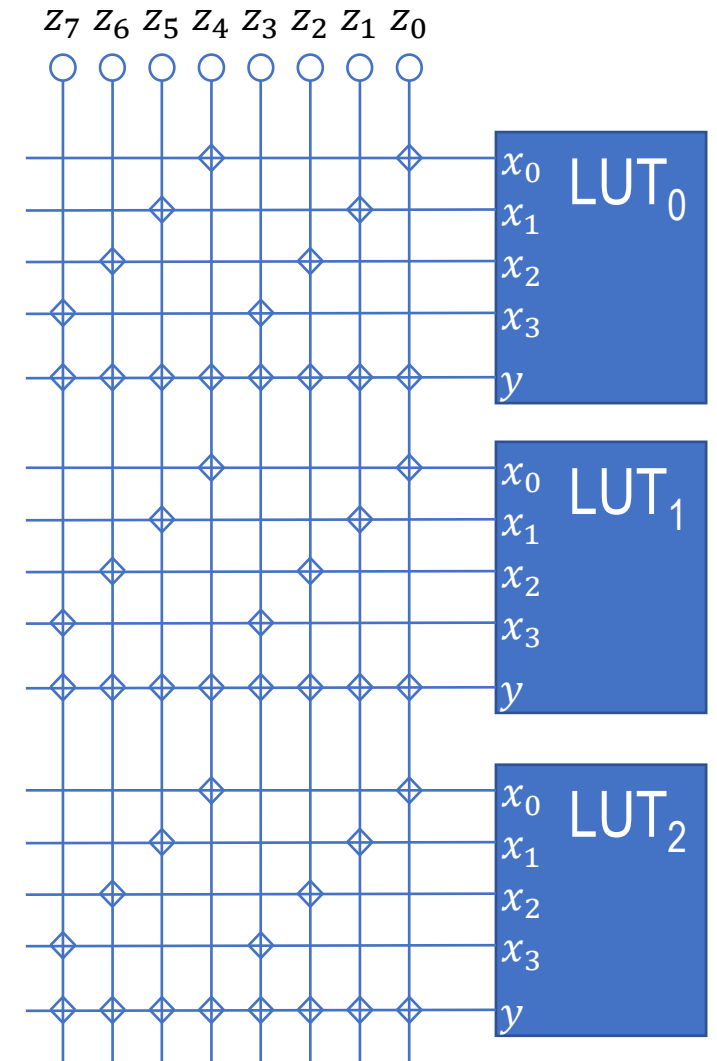
Field Programmable Gate Array (FPGA)

- Example

- Combinational function with 5 inputs on a 4-input 1-output LUT
 - $y = f(x_4, x_3, x_2, x_1, x_0)$
- Using a different input with respect to x_4 of the previous slide, it can be rewritten as
 - $y = x_3 \cdot f(x_4, 1, x_2, x_1, x_0) + \overline{x_3} \cdot f(x_4, 0, x_2, x_1, x_0)$
 - $y' = f(x_4, 1, x_2, x_1, x_0) = f'(x_4, x_2, x_1, x_0)$
 - $y'' = f(x_4, 0, x_2, x_1, x_0) = f''(x_4, x_2, x_1, x_0)$
 - $\rightarrow y = x_3 \cdot y' + \overline{x_3} \cdot y''$
- Functions f' and f'' have only 4 inputs and 1 output
 - They fit into a LUT each
 - They are different from those of the previous slide, but the final result is the same
- The last function is a multiplexer with x_3 as the selection signal
 - Need an additional LUT

Field Programmable Gate Array (FPGA)

- Example
 - You need to implement the following Boolean expression
 - $f = sca + \bar{s}\bar{b}\bar{a} + \bar{s}c\bar{b} + s\bar{d}b\bar{a}$
 - $f = s(ca + \bar{d}b\bar{a}) + \bar{s}(\bar{b}\bar{a} + c\bar{b})$
 - You have an FPGA with 3 LUTs
 - Each LUT has 4 inputs and 1 output
 - Interconnection fabric uses Switch Matrices
 - Switch matrices are present only where there is a diamond shape
 - The other wire crossing are not connections
 - Implement the given function
 1. Assign the pins
 2. Give the connections in the switch matrices
 3. Give the content of the LUTs



Field Programmable Gate Array (FPGA)

• Example

• Functions to be implemented

$$f' = ca + \bar{d}b\bar{a}$$

$$f'' = \bar{b}\bar{a} + c\bar{b}$$

$$f = sf' + \bar{s}f''$$

LUT₀

LUT₁

LUT₂

1. Assign the pins

• Configure external pins

- I/O elements not shown in the picture

• Inputs: a, b, c, d, s

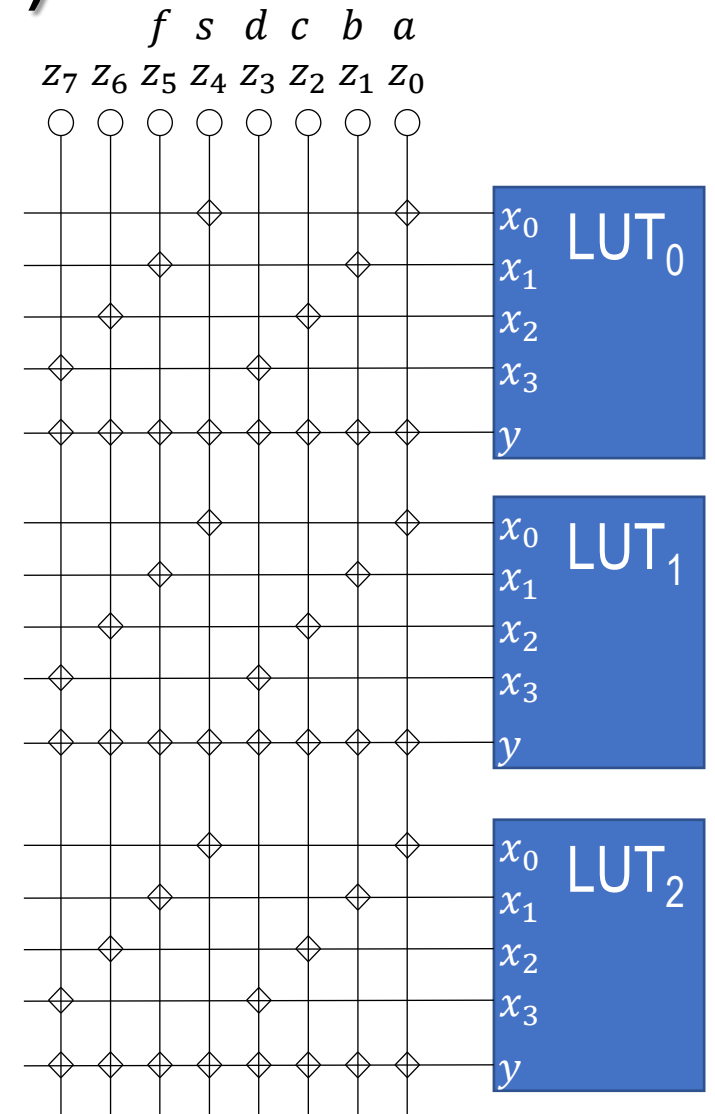
• Outputs: f

• Almost arbitrary

• Affects the complexity of the interconnections

• Affects the content of the LUTs

• Functions f' and f'' do not need external pins



Field Programmable Gate Array (FPGA)

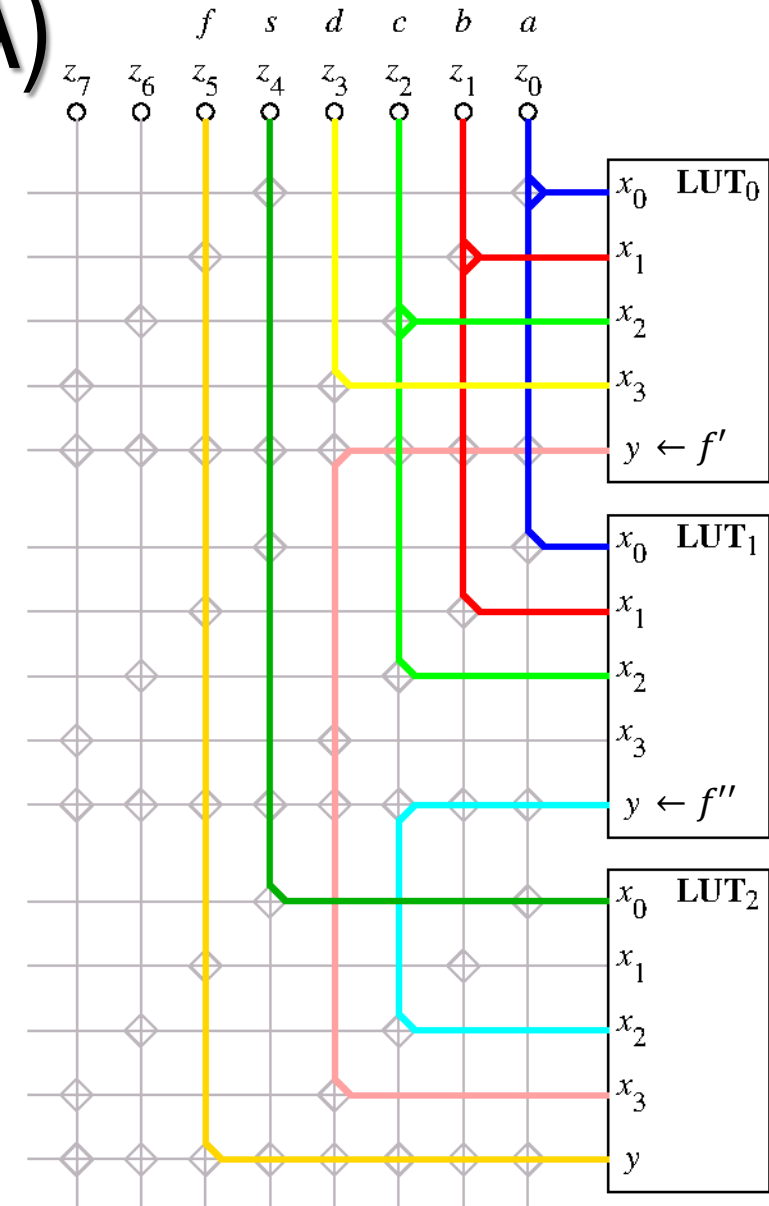
- Example

- Functions to be implemented

- $f' = ca + \overline{d}b\overline{a}$
- $f'' = \overline{b}\overline{a} + c\overline{b}$
- $f = sf' + \overline{s}f''$

2. Give the connections in the switch matrices

- Many different solutions
- Wires below z_2 and z_3 are segmented by means of the switch matrices
 - Different signals travel along them
 - Segments do not make contact
 - Some parts of the wires are floating
- Segmentation allows better use of the interconnection resources



Field Programmable Gate Array (FPGA)

• Example

• Functions to be implemented

- $f' = ca + \bar{d}b\bar{a}$
- $f'' = \bar{b}\bar{a} + c\bar{b}$
- $f = sf' + \bar{s}f''$

3. Give the content of the LUTs

- Depends on pin selection
- Make unused pins *don't care*
 - LUT1 and LUT2
 - Duplicate rows

d	c	b	a	f'
x ₃	x ₂	x ₁	x ₀	y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

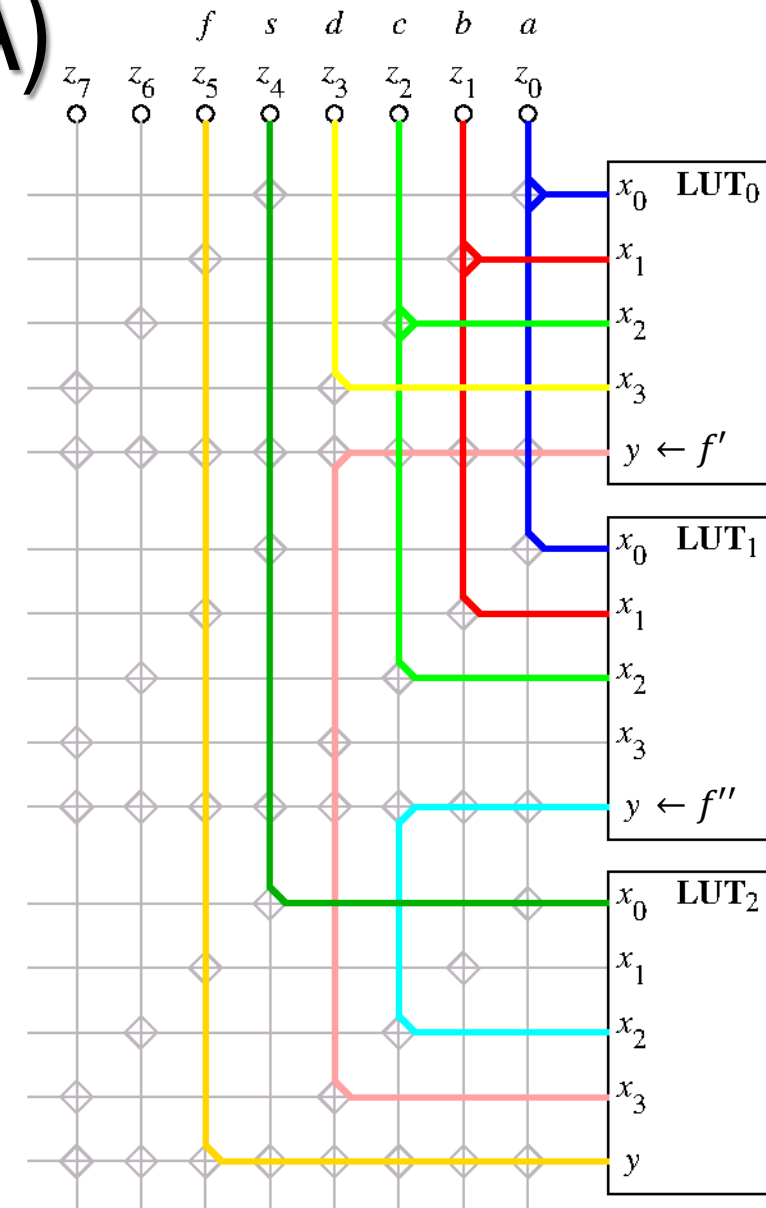
LUT₀

	c	b	a	f'
x_3	x_2	x_1	x_0	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

LUT₁

f'	f''		s	f
x_3	x_2	x_1	x_0	y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

LUT₂



Field Programmable Gate Array (FPGA)

• Example

• Functions to be implemented

- $f' = ca + \bar{d}b\bar{a}$
- $f'' = \bar{b}\bar{a} + c\bar{b}$
- $f = sf' + \bar{s}f''$

• Different interconnections

- Longer paths
 - Longer delays
- Greater number of switch matrices traversed
 - Longer delays
- Different LUT contents
 - Not a problem

d	c	b	a	f'
x_3	x_2	x_1	x_0	y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

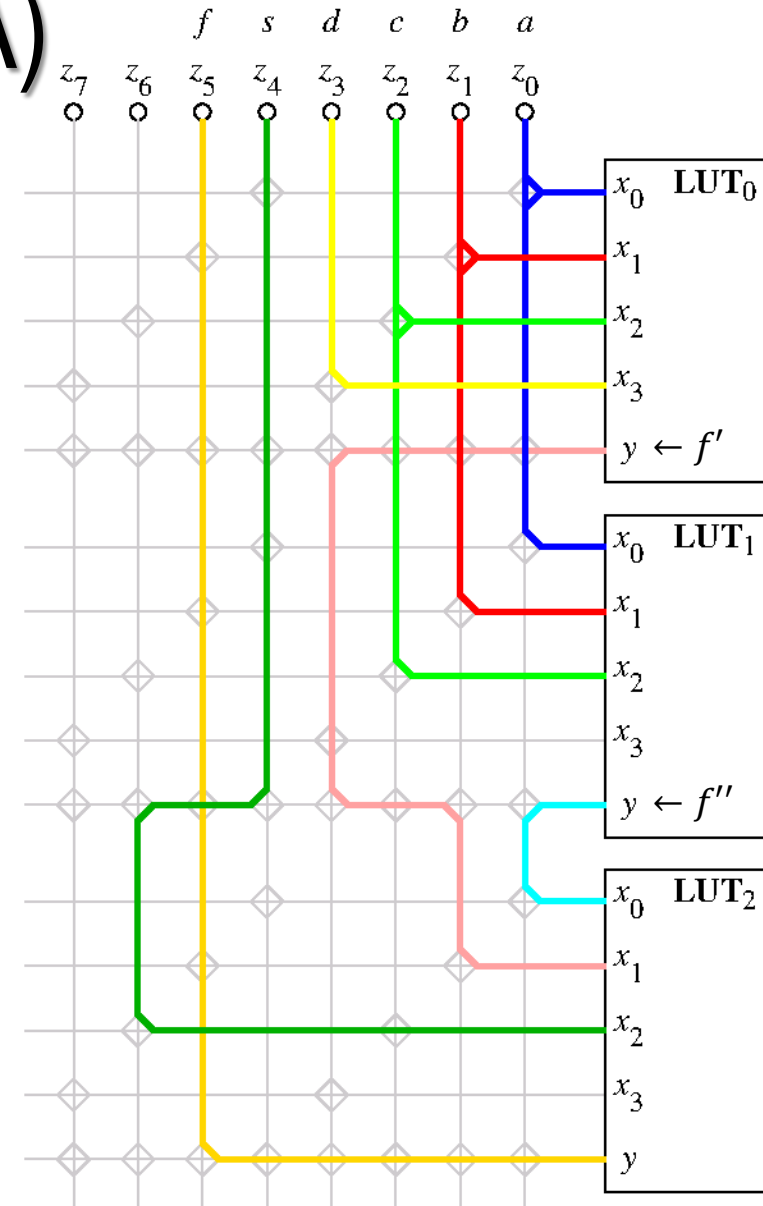
LUT₀

	c	b	a	f''
x_3	x_2	x_1	x_0	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

LUT₁

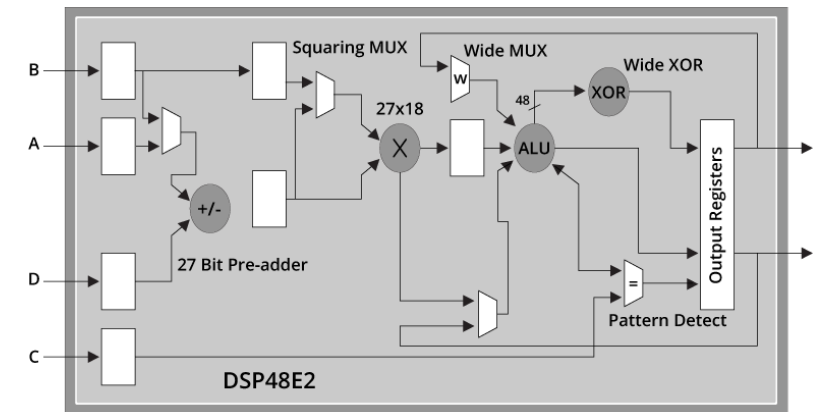
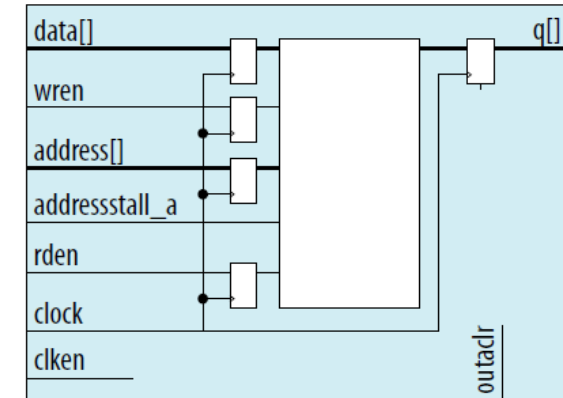
	s	f'	f''	f
x_3	x_2	x_1	x_0	y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

LUT₂



Field Programmable Gate Array (FPGA)

- Additional FPGA blocks
 - Memories and FIFOs
 - Internal SRAMs
 - Multiple dual port small memories
 - Can be combined to form a single large memory or FIFO
 - Interfaces for external dynamic memories
 - Carry chains between different CLBs or LEs
 - Allow implementing efficient adders
 - Embedded hardware multipliers or DSP blocks
 - Clock management
 - High speed I/O
 - LVDS, Gigabit, ...
 - Processor cores
 - PowerPC, ARM, NiosII, MicroBlaze



Field Programmable Gate Array (FPGA)

- Technologies
 - SRAM is dominant
 - Used for LUTs, but also to control MUXes or pass transistors
 - Standard CMOS technology, can be reprogrammed on the fly
 - Standard technology allows to embed other blocks
 - Large size, consumes power, not permanent
 - Fuse and Antifuse
 - Small, permanent
 - Cannot be reprogrammed
 - Cannot be tested
 - E²PROM, Flash
 - Can be reprogrammed
 - Non standard process

Field Programmable Gate Array (FPGA)

- Comparison with CPLDs
 - FPGAs are configured at a finer level
 - CPLDs have few large configurable blocks
 - FPGAs have many small configurable blocks
 - FPGAs are volatile
 - Typically implemented using SRAM technology
 - Need to reprogram at every power-up
 - CPLDs are permanent (using E²PROM technology)
 - FPGAs use standard technology
 - Easier to include additional resources
 - FPGAs are more expensive
 - They are cheaper considering the gate count, but usually much larger
 - CPLDs are more predictable
 - Delays in FPGAs can strongly change based on the implementation

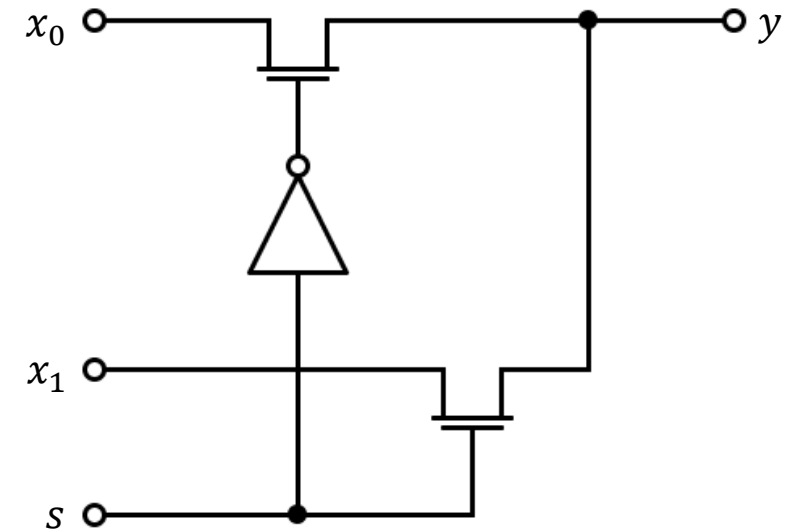
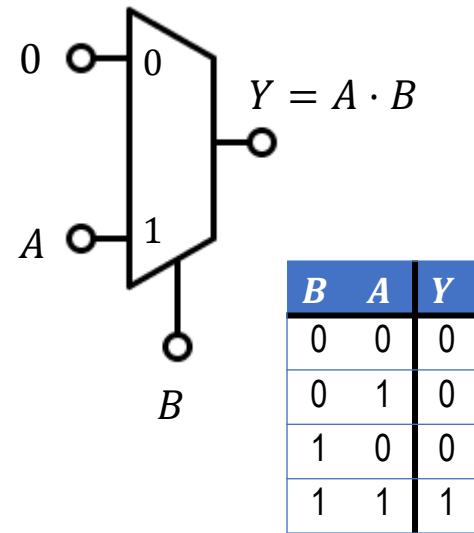
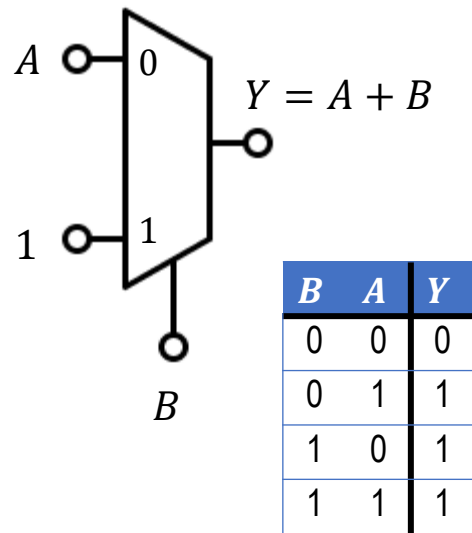
Field Programmable Gate Array (FPGA)

- Multiplexer based FPGAs
 - A 2^n to 1 multiplexer can be used to implement a combinational logic function of n variables
 - The n variables are the selection of the multiplexer
 - The 2^n inputs of the multiplexer are tied to logic 0 or logic 1, depending on the function
 - It is a Look Up Table (LUT)
 - A 2^{n-1} to 1 multiplexer can be used to implement a combinational logic function of n variables
 - $n - 1$ variables are used as multiplexer selection
 - The remaining variable may be passed, affirmed or complemented, in the 2^{n-1} inputs of the multiplexer
 - Some of the 2^{n-1} inputs of the multiplexer are still tied to logic 0 or logic 1

x_{n-2}			x_0	x_{n-1}	Y			
—	—	—	—	0	0	0	1	1
—	—	—	—	1	0	1	0	1
					0	x_{n-1}	$\overline{x_{n-1}}$	1

Field Programmable Gate Array (FPGA)

- Multiplexer based FPGAs
 - Example of implementing AND and OR functions
 - Implementation using a small number of transistors (pass transistors or transmission gates)
 - Small size, cheap

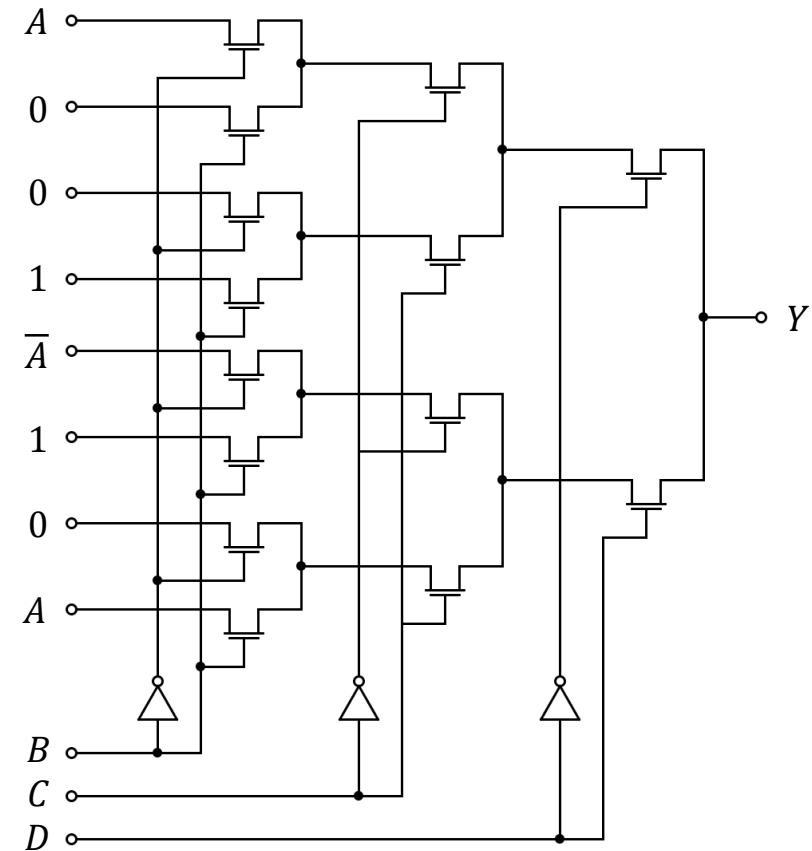
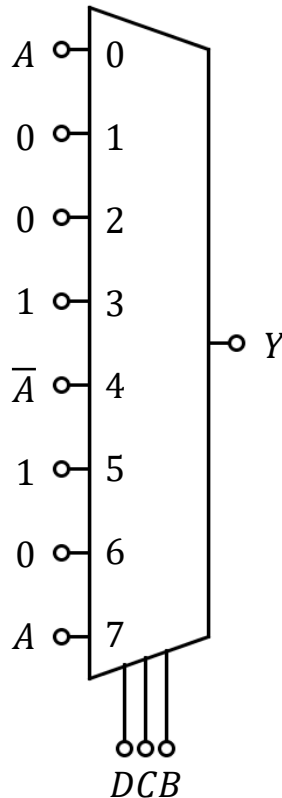


Field Programmable Gate Array (FPGA)

- Multiplexer based FPGAs
 - Example of implementing a complex function (4 inputs, 1 output)

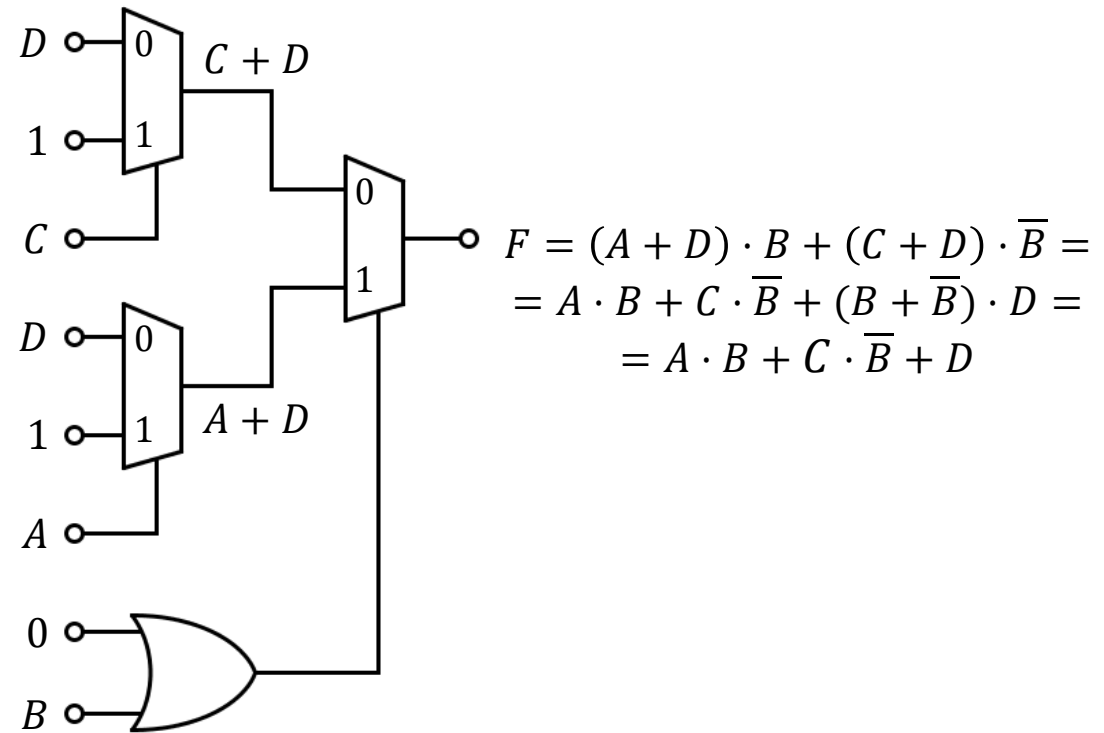
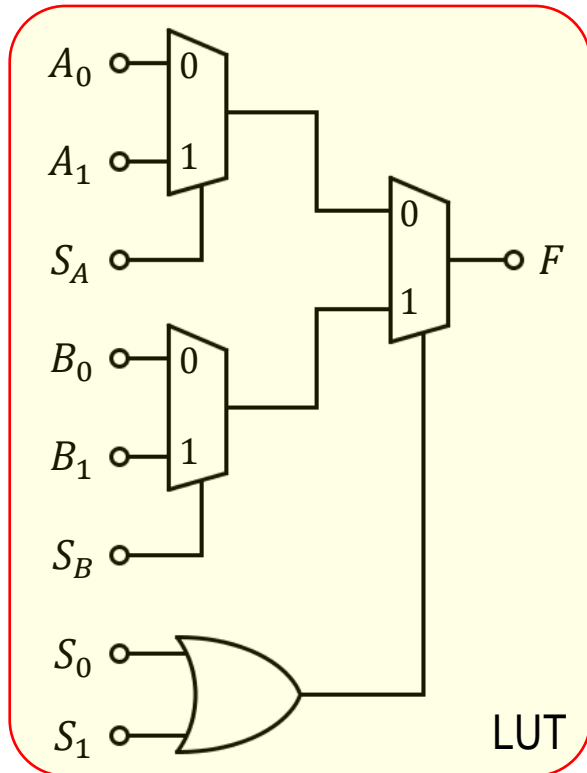
D	C	B	A	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

$$Y = \bar{D}\bar{C}\bar{B}A + \bar{D}CB + DBA + D\bar{C}\bar{A}$$



Field Programmable Gate Array (FPGA)

- Multiplexer based FPGAs
 - Example (Actel FPGA)
 - $F = (A \cdot B) + (\overline{B} \cdot C) + D$



Field Programmable Gate Array (FPGA)

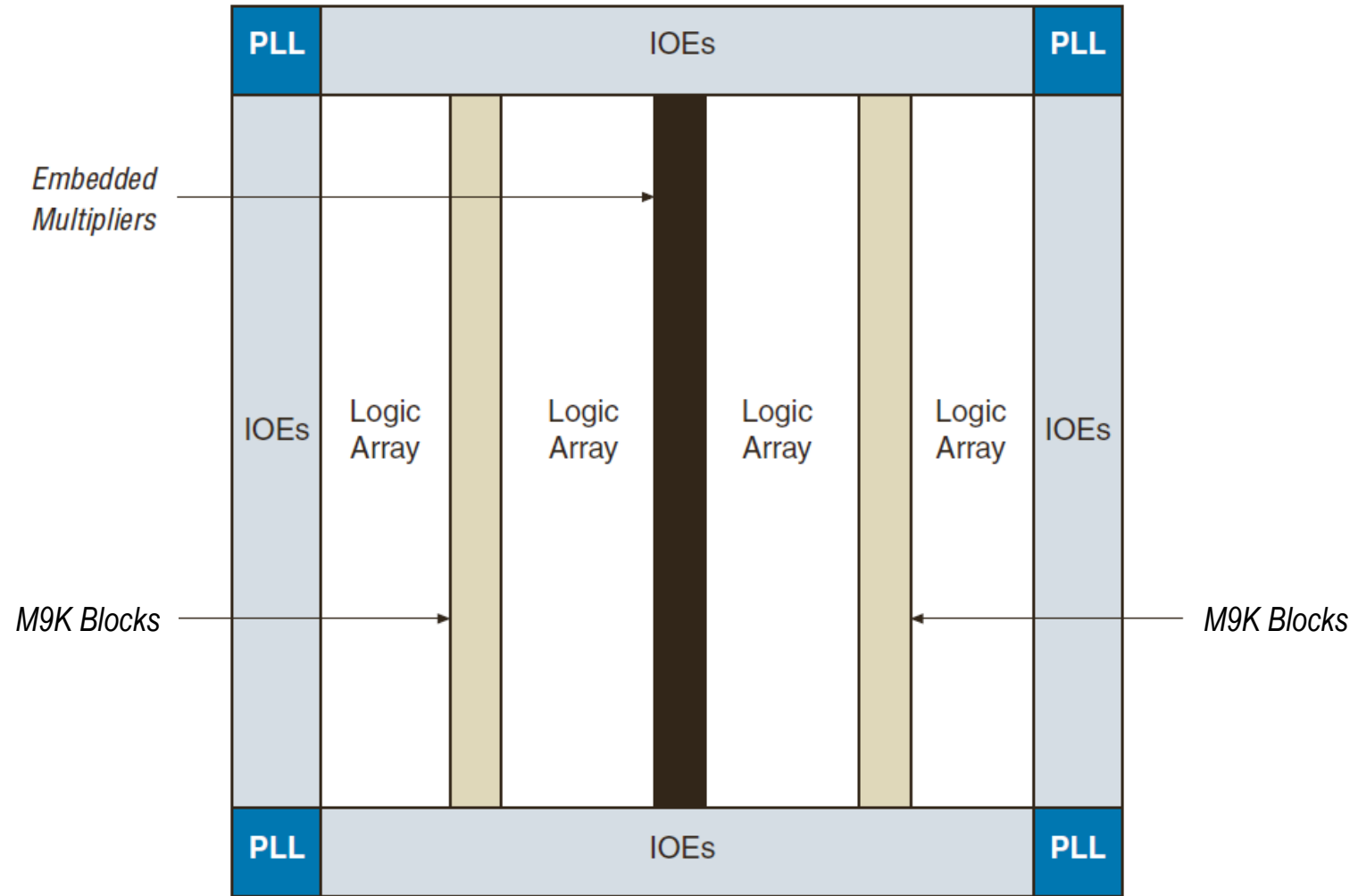
- Altera Cyclone 10 LP
 - Family of devices
 - Different number of resources
 - LEs
 - Memories
 - HW Multipliers
 - Pins

Table 2. Maximum Resource Counts for Intel Cyclone 10 LP Devices

Resource		Device							
		10CL006	10CL010	10CL016	10CL025	10CL040	10CL055	10CL080	10CL120
Logic Elements (LE)		6,272	10,320	15,408	24,624	39,600	55,856	81,264	119,088
M9K Memory	Block	30	46	56	66	126	260	305	432
	Capacity (Kb)	270	414	504	594	1,134	2,340	2,745	3,888
18 × 18 Multiplier		15	23	56	66	126	156	244	288
PLL		2	2	4	4	4	4	4	4
Clock		20	20	20	20	20	20	20	20
Maximum I/O		176	176	340	150	325	321	423	525
Maximum LVDS		65	65	137	52	124	132	178	230

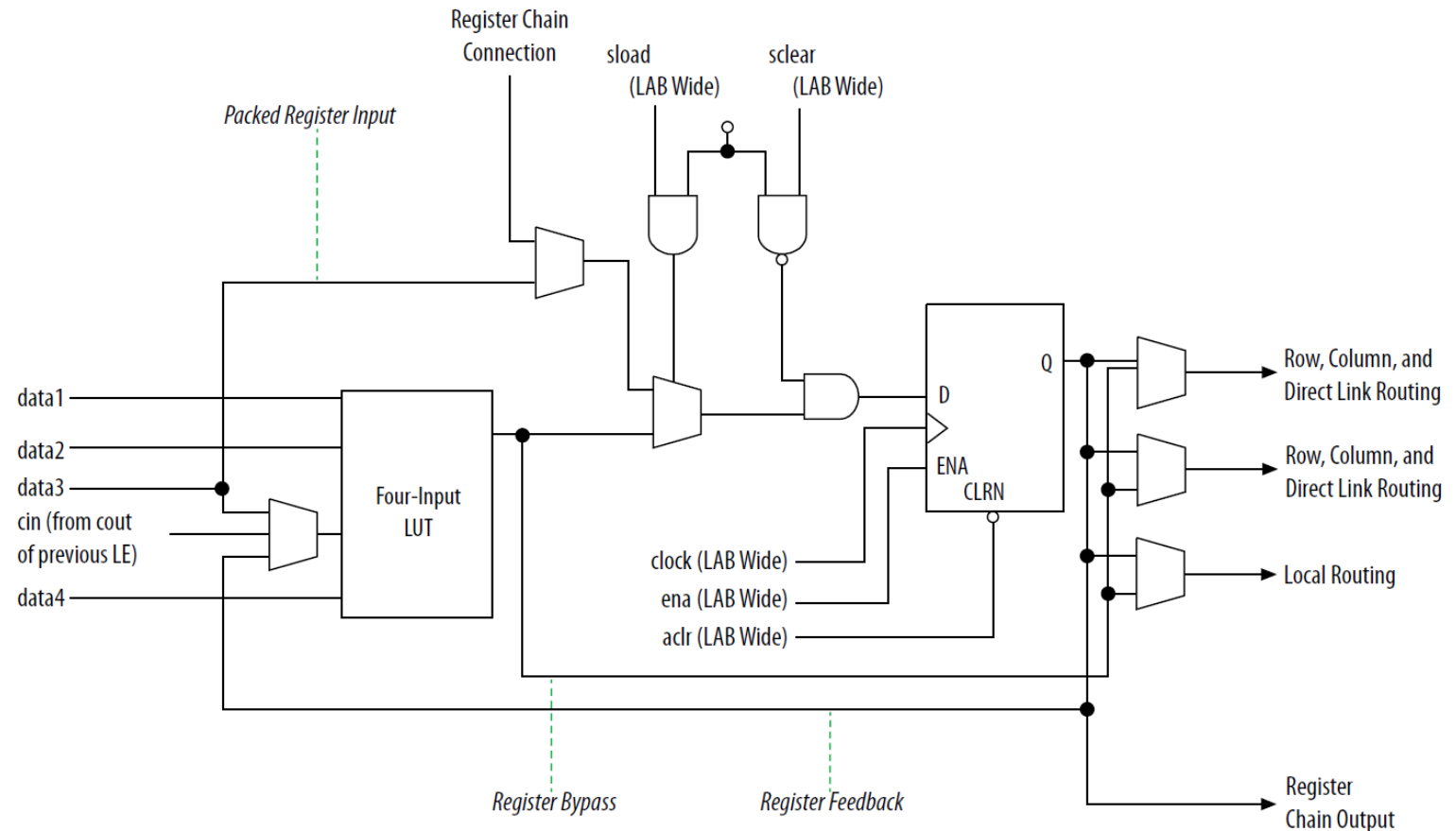
Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - High level block diagram
 - M9K is a 9 Kbit memory
 - 8K x 1, 4K x 2, 2K x 4
 - 1K x 8, 1K x 9
 - 512 x 16, 512 x 18
 - 256 x 32, 256 x 36
 - Dual Port SRAM
 - With possibly different widths
 - IOE is for Input/Output



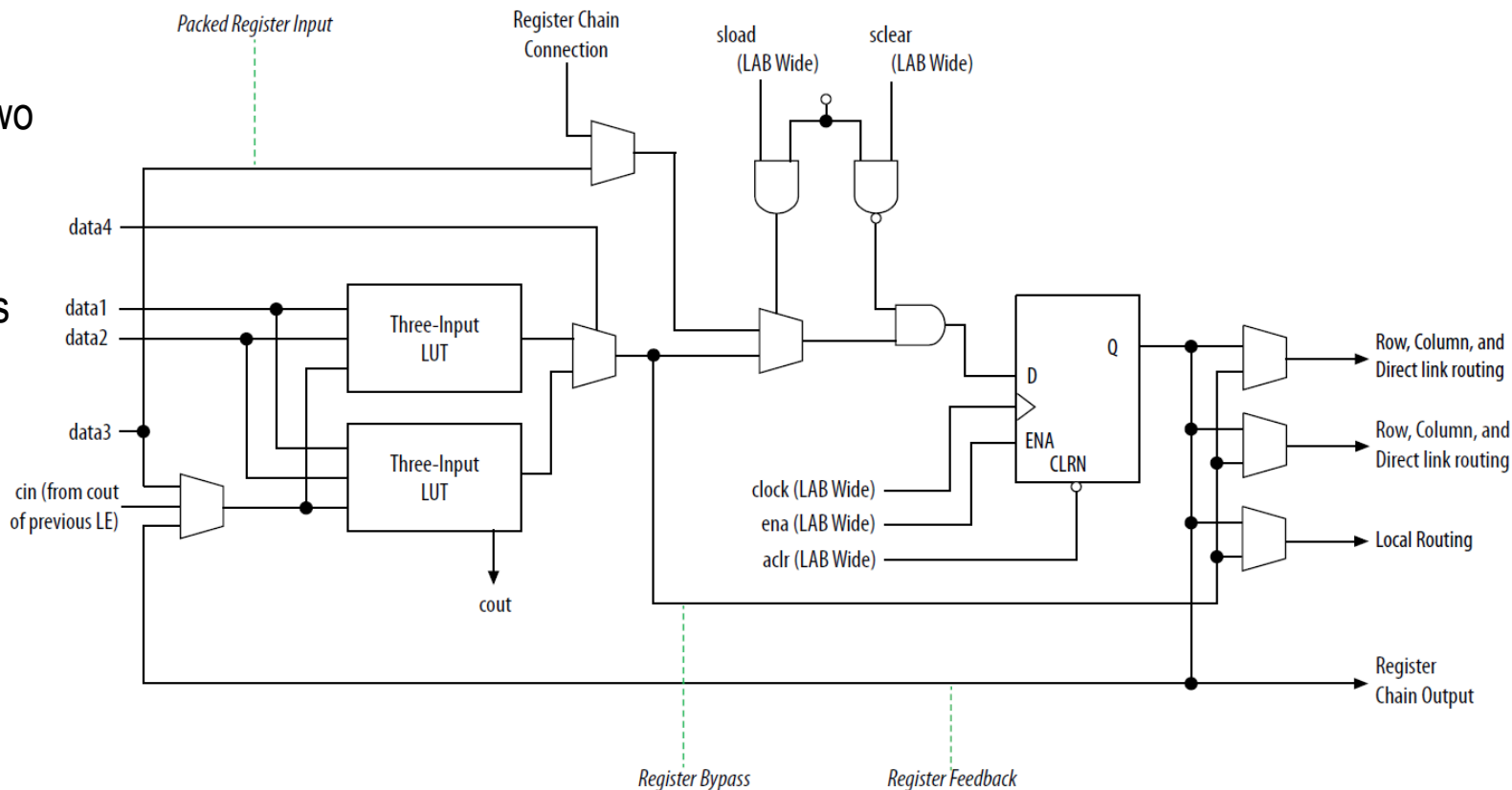
Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - Logic Element (normal mode)
 - 4-input 1-output LUT
 - 16 x 1 memory
 - Flip-flop
 - Clock enable
 - Asynchronous clear
 - Synchronous set and clear
 - Three outputs
 - Combinational
 - Sequential
 - Register chain
 - To implement shift registers
 - No need of programmable interconnections



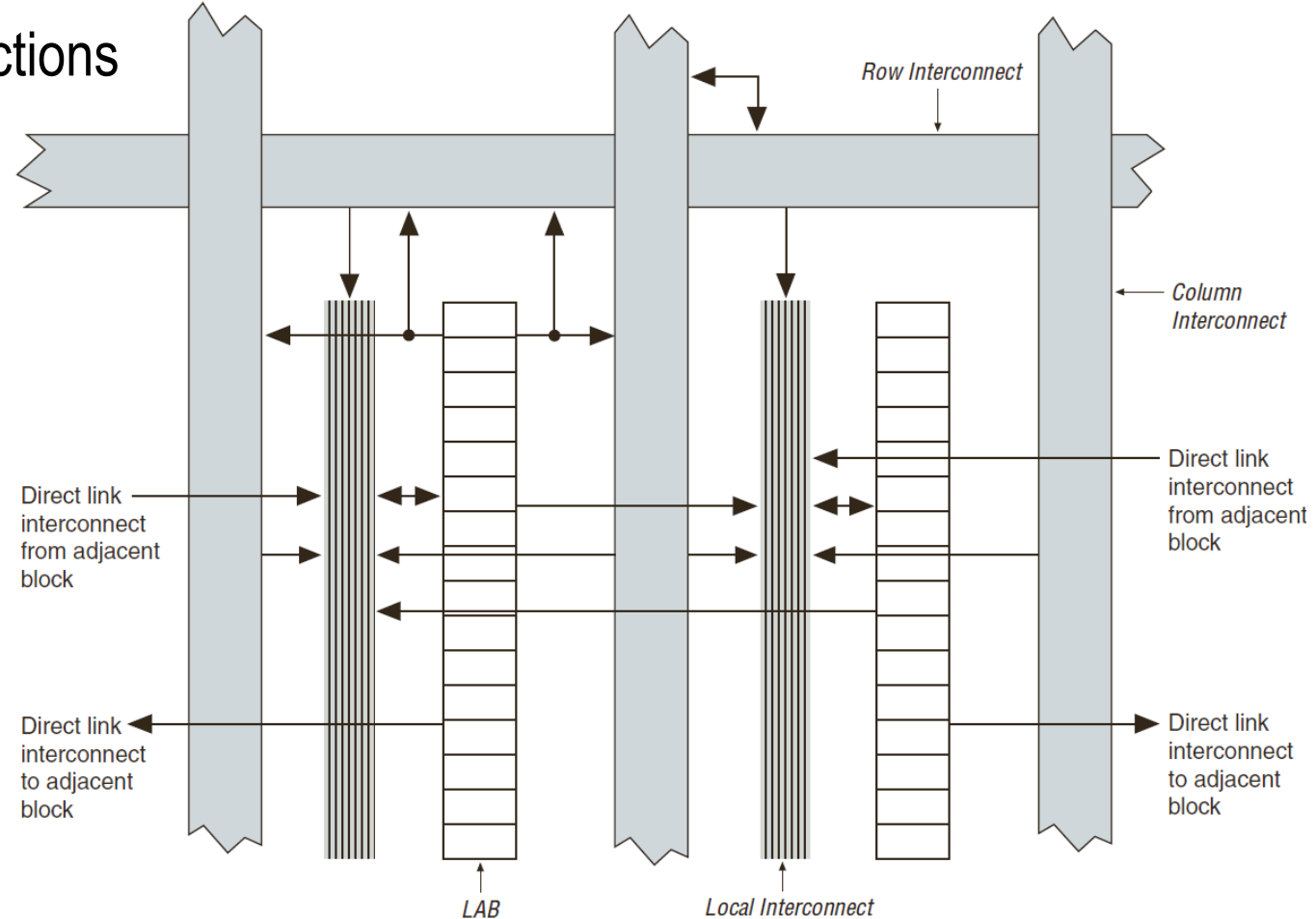
Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - Logic Element (arithmetic mode)
 - Two 3-input 1-output LUTs
 - 8 x 1 memories
 - Same LUT as before, split in two
 - Additional c_{out} output
 - Carry chain
 - Direct connection between LEs
 - No need of programmable interconnections
 - Fast carry propagation
 - Used to implement adders



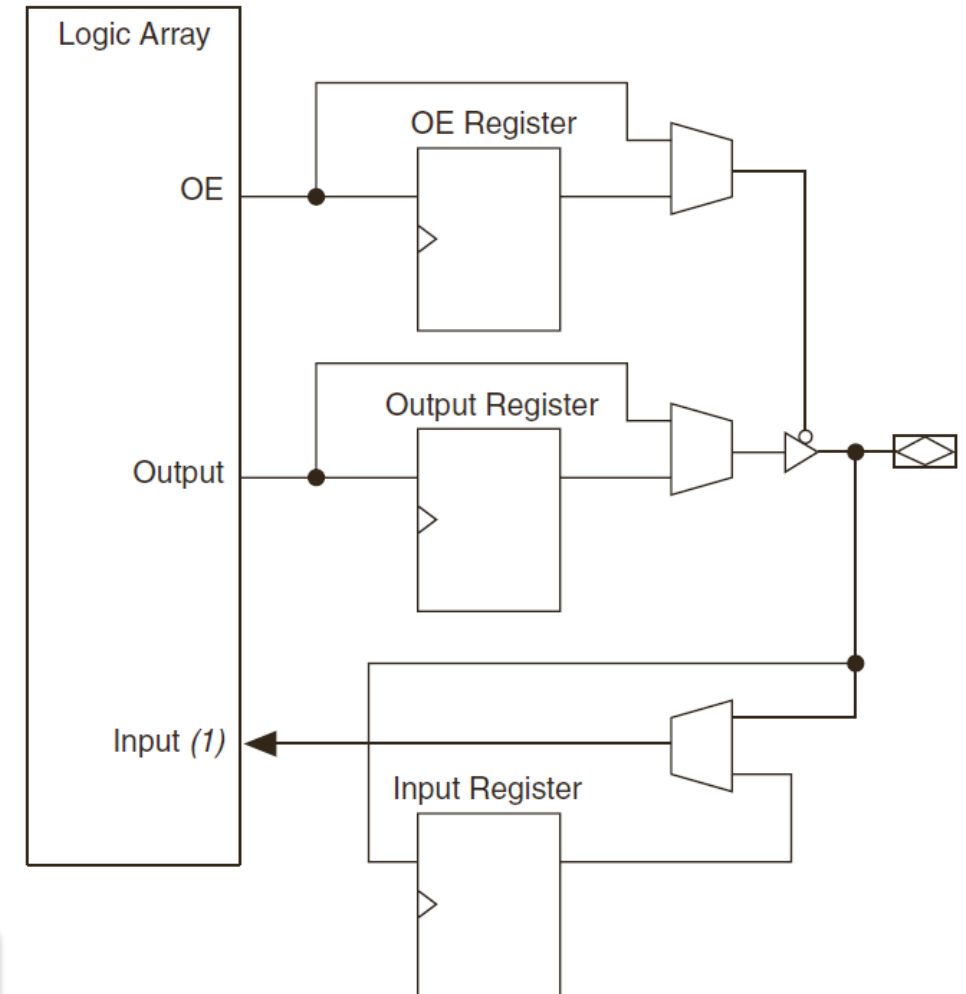
Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - Logic Array Block (LAB) and Interconnections
 - 16 LEs in a vertical array
 - Register chain
 - Carry chain
 - Local interconnections
 - Between LEs of the same LAB
 - Programmable
 - Row and Column Interconnections
 - Between LEs of different LABs
 - Programmable



Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - Input/Output Element
 - Can be configured as
 - Input only
 - Output only
 - Bidirectional
 - Output buffer can be configured as *open drain*
 - Internal selectable pull-up resistor (25 k Ω)
 - Configurable drive strength
 - From 4 mA to 24 mA
 - Configurable output voltages

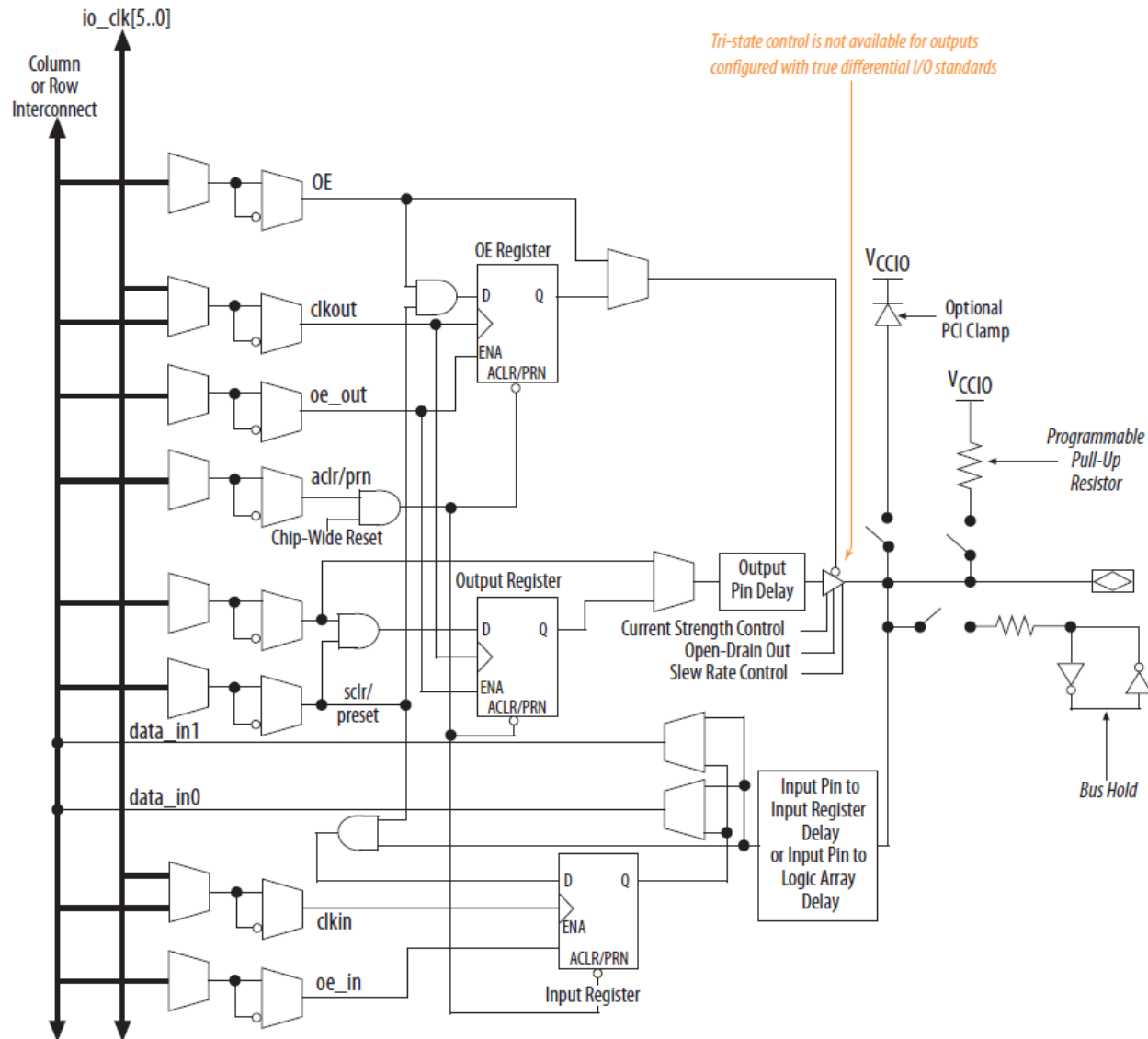


Simplified view

Field Programmable Gate Array (FPGA)

- Altera Cyclone 10 LP
 - Input/Output Element
 - Can be configured as
 - Input only
 - Output only
 - Bidirectional
 - Output buffer can be configured as *open drain*
 - Internal selectable pull-up resistor (25 k Ω)
 - Configurable drive strength
 - From 4 mA to 24 mA
 - Configurable output voltages

Detailed view



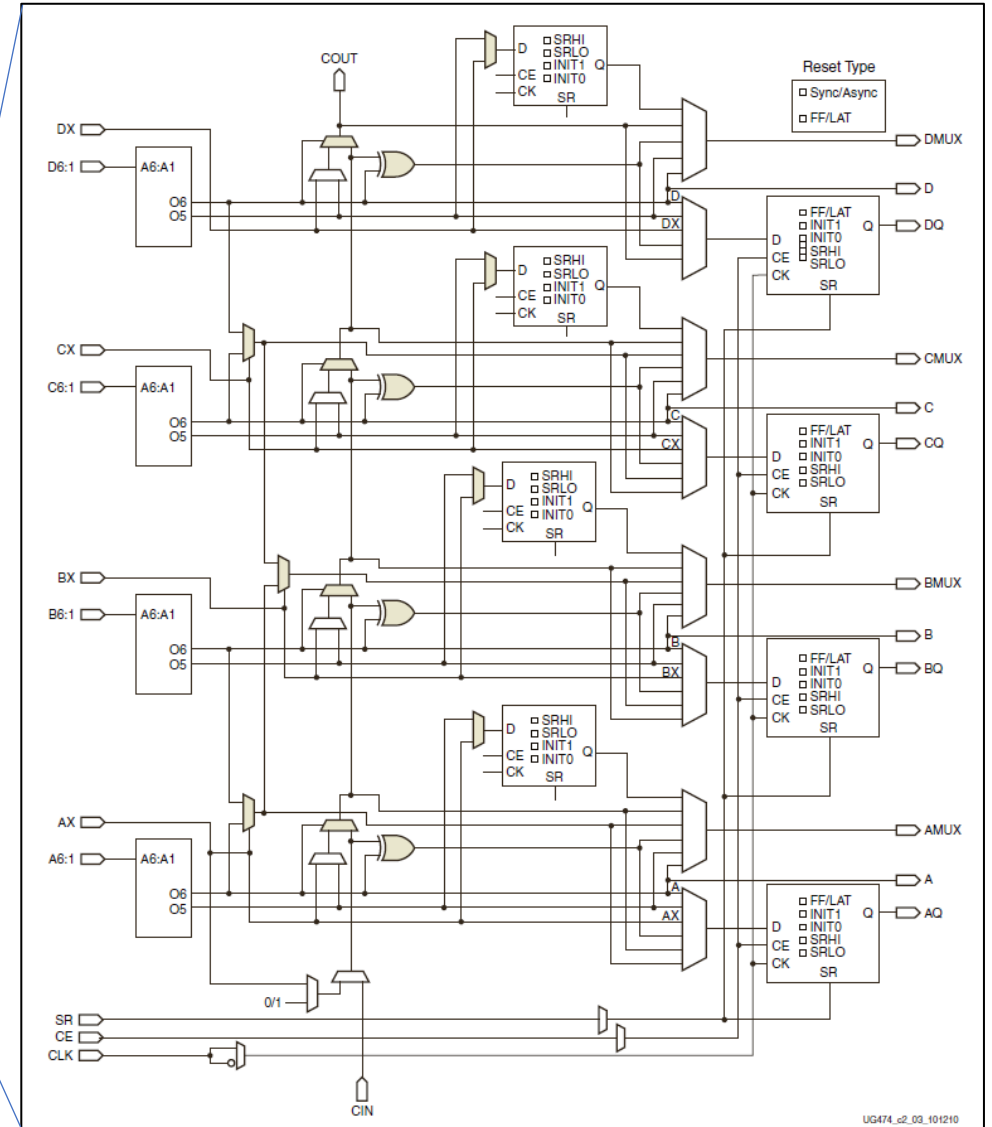
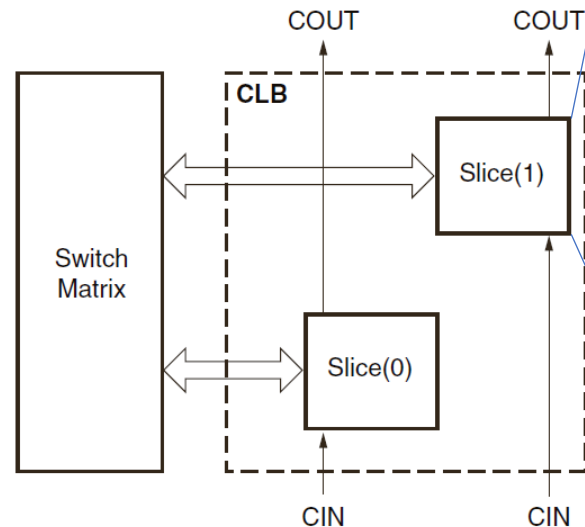
Field Programmable Gate Array (FPGA)

- Xilinx Virtex 7
 - Family of devices

Device ⁽¹⁾	Logic Cells	Configurable Logic Blocks (CLBs)		DSP Slices ⁽³⁾	Block RAM Blocks ⁽⁴⁾			CMTs ⁽⁵⁾	PCIe ⁽⁶⁾	GTX	GTH	GTZ	XADC Blocks	Total I/O Banks ⁽⁷⁾	Max User I/O ⁽⁸⁾	SLRs ⁽⁹⁾
		Slices ⁽²⁾	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)									
XC7V585T	582,720	91,050	6,938	1,260	1,590	795	28,620	18	3	36	0	0	1	17	850	N/A
XC7V2000T	1,954,560	305,400	21,550	2,160	2,584	1,292	46,512	24	4	36	0	0	1	24	1,200	4
XC7VX330T	326,400	51,000	4,388	1,120	1,500	750	27,000	14	2	0	28	0	1	14	700	N/A
XC7VX415T	412,160	64,400	6,525	2,160	1,760	880	31,680	12	2	0	48	0	1	12	600	N/A
XC7VX485T	485,760	75,900	8,175	2,800	2,060	1,030	37,080	14	4	56	0	0	1	14	700	N/A
XC7VX550T	554,240	86,600	8,725	2,880	2,360	1,180	42,480	20	2	0	80	0	1	16	600	N/A
XC7VX690T	693,120	108,300	10,888	3,600	2,940	1,470	52,920	20	3	0	80	0	1	20	1,000	N/A
XC7VX980T	979,200	153,000	13,838	3,600	3,000	1,500	54,000	18	3	0	72	0	1	18	880	N/A
XC7VX1140T	1,139,200	178,000	17,700	3,360	3,760	1,880	67,680	24	4	0	96	0	1	22	1,100	4
XC7VH580T	580,480	90,700	8,850	1,680	1,880	940	33,840	12	2	0	48	8	1	12	600	2
XC7VH870T	876,160	136,900	13,275	2,520	2,820	1,410	50,760	18	3	0	72	16	1	13	650	3

Field Programmable Gate Array (FPGA)

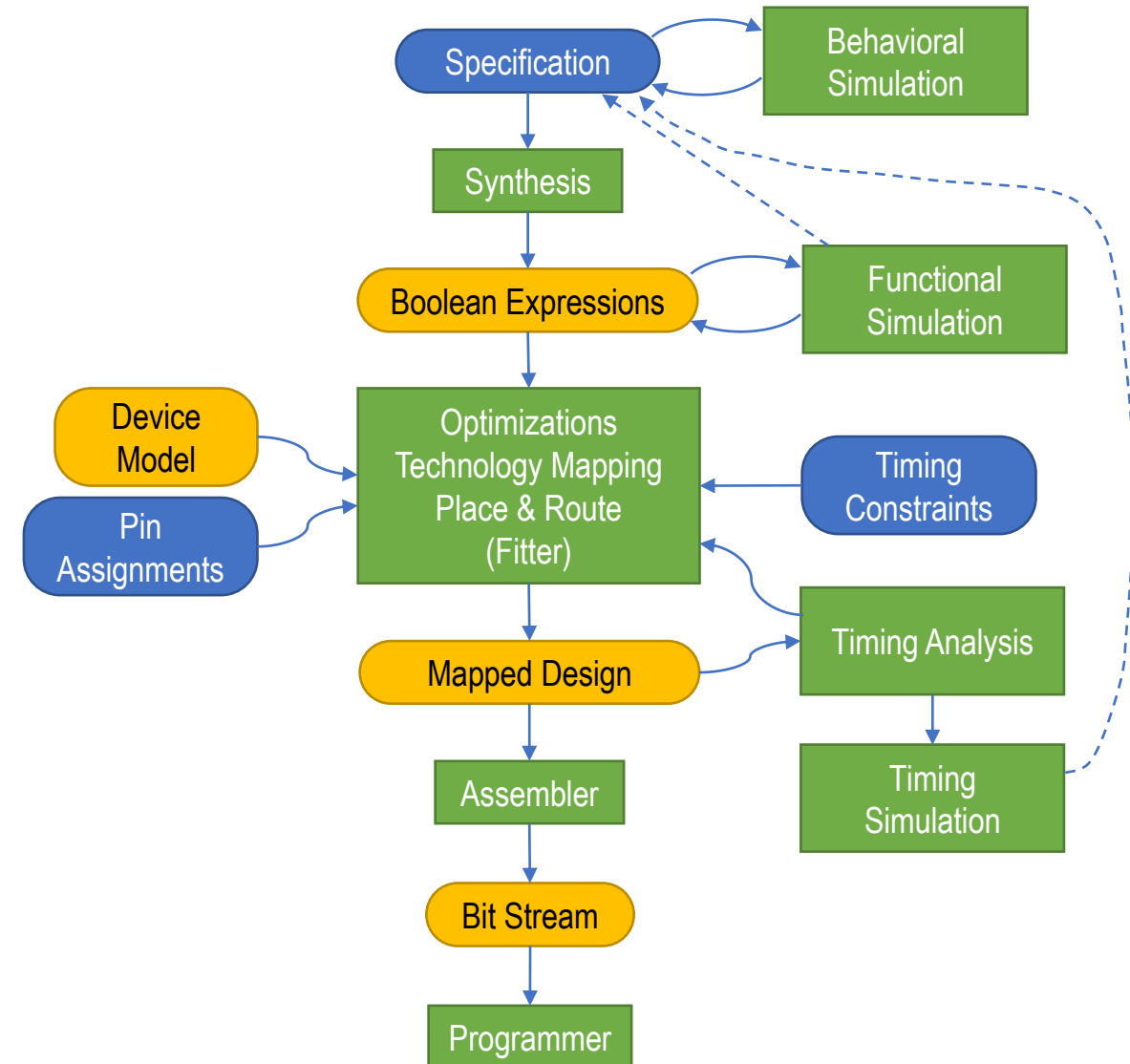
- Xilinx Virtex 7
 - Configurable Logic Block (CLB)
 - Two slices
 - Each slice
 - Four 6-input 2-output LUTs
 - 8 Storage Elements (Flip-Flops)
 - Carry chains
 - Multiplexers



Field Programmable Gate Array (FPGA)

• Design Flow

- Specification and Design Entry
 - VHDL, Verilog, SystemC, Chisel, HLS
 - Schematic and block diagrams
 - Other hardware description languages
 - Higher abstraction levels
- Behavioral Simulation
 - Execution of the specification
 - Code does not need to be synthesizable
 - No timing information
 - Verify algorithm
 - Very fast
 - Explore the design space



Field Programmable Gate Array (FPGA)

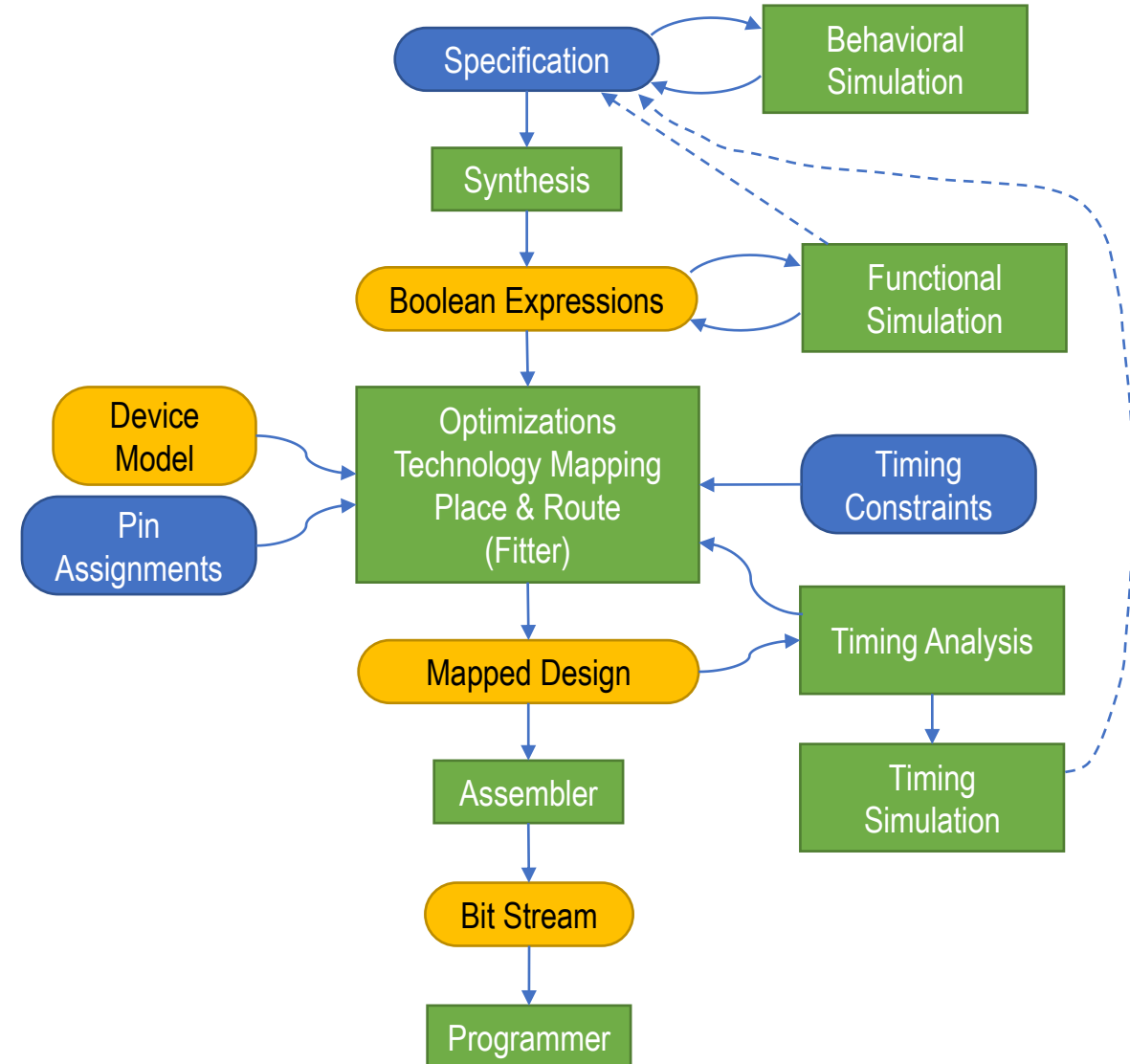
- Design Flow

- Initial Synthesis

- Generates boolean expressions
 - Independent of the target implementation
 - Code needs to be synthesizable

- Functional Simulation

- Similar to Behavioral Simulation
 - Also known as RTL simulation
 - No timing information
 - Very fast

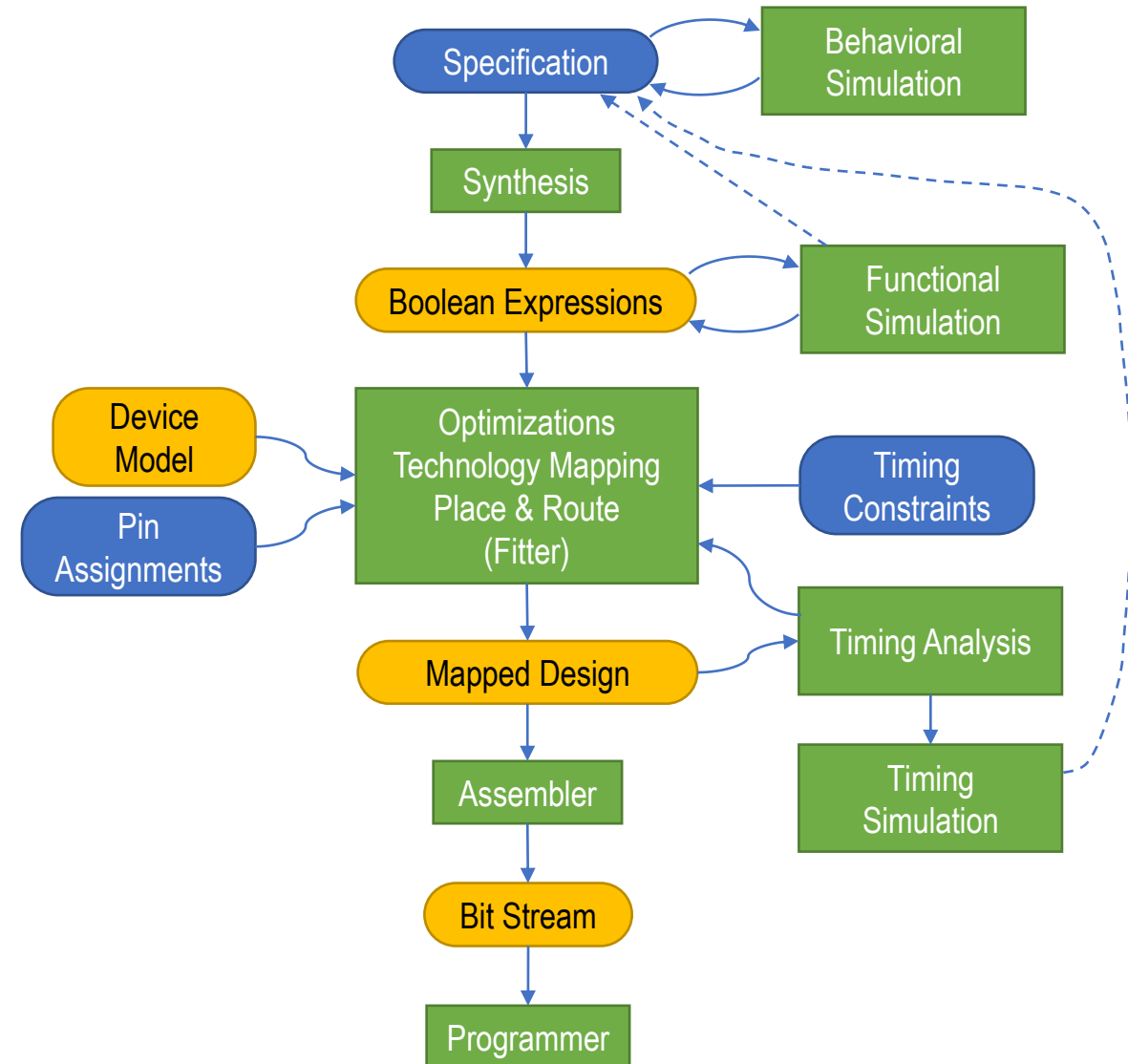


Field Programmable Gate Array (FPGA)

- Design Flow

- Logic Synthesis

- Technology independent optimizations
 - Technology mapping and Place & Route
 - Need a device model
 - Provided by the manufacturer
 - Need the pin assignments
 - Driven by timing constraints
 - Look for best implementation
 - Very complex problem
 - Simulated annealing
 - Genetic algorithms
 - ...
 - May get different results at any run



Field Programmable Gate Array (FPGA)

- Design Flow

- Timing analysis and Timing Simulation

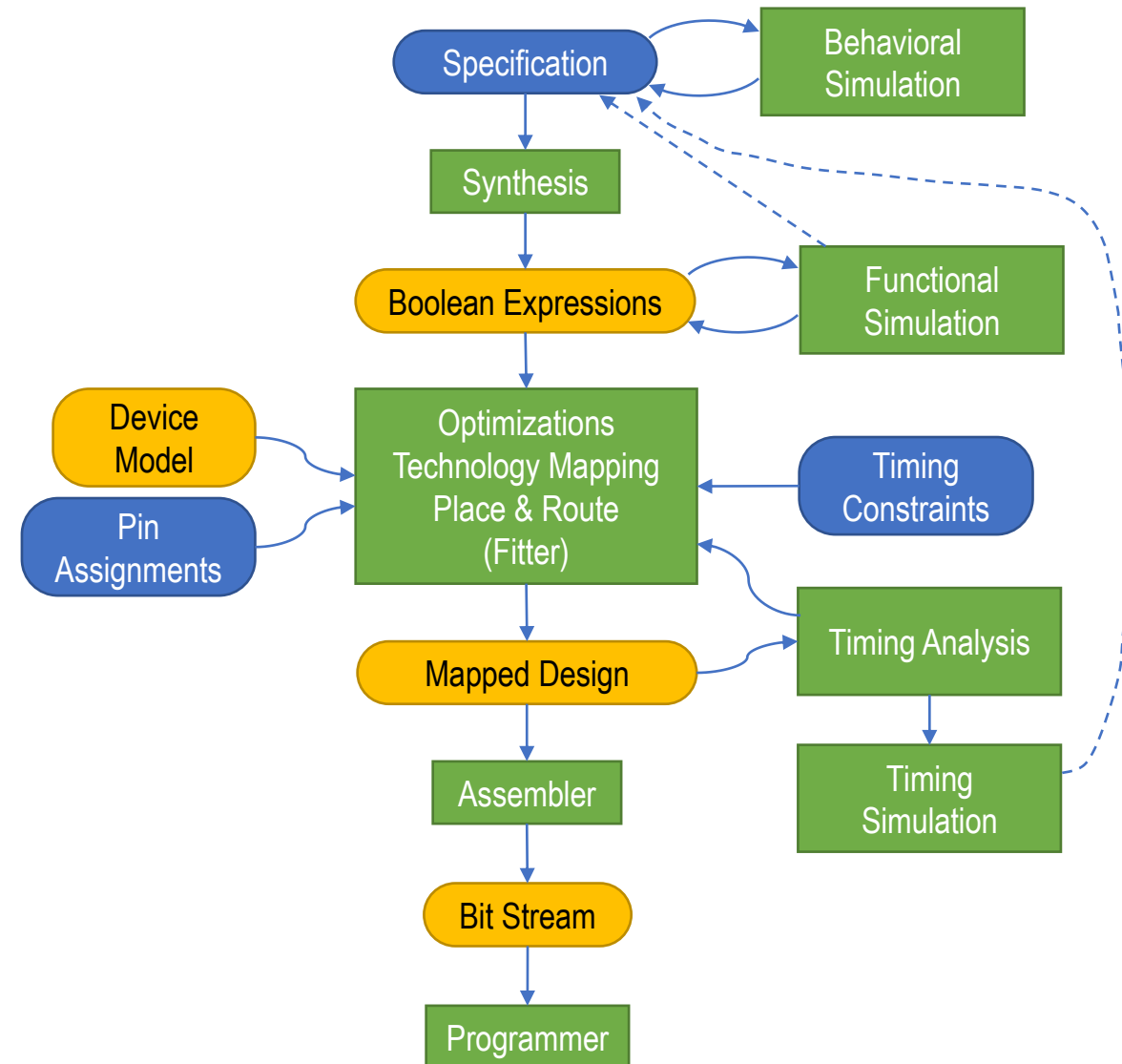
- Compared with timing constraints
 - Also known as gate level simulation
 - Slow

- Assembler

- Generates the Bit Stream for the target device
 - All SRAM configuration bits

- Programmer

- Writes the Bit Stream on the device
 - JTAG Interface
 - SRAM is like a very long shift register
 - Serially programmed
 - Reduced pins



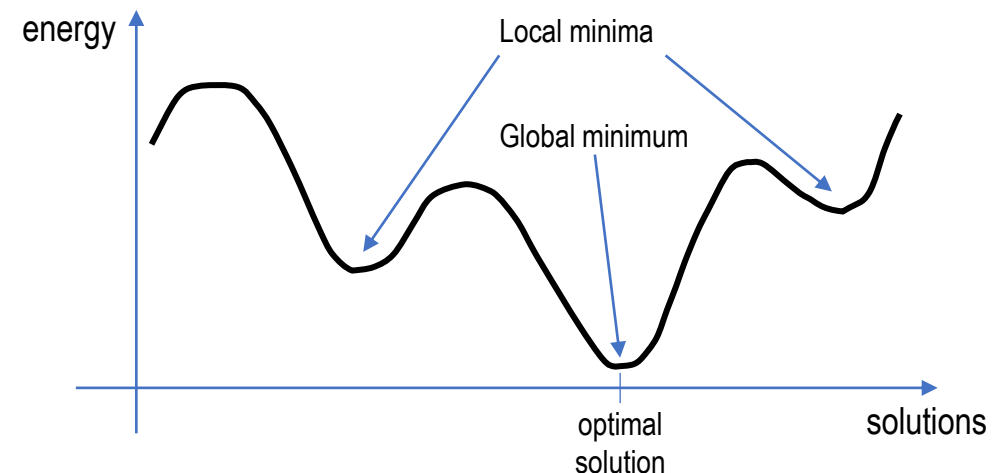
Field Programmable Gate Array (FPGA)

- Place & Route
 - Placement
 - Goal
 - Determine which logic block within an FPGA should implement each of the logic blocks required in the circuit
 - Objective
 - Minimize the required wiring
 - Balance the wiring density across the FPGA
 - Maximize circuit speed
 - Placement algorithms
 - Simulated annealing based placers
 - Min-cut (partitioning based) placers
 - Analytic placers

Field Programmable Gate Array (FPGA)

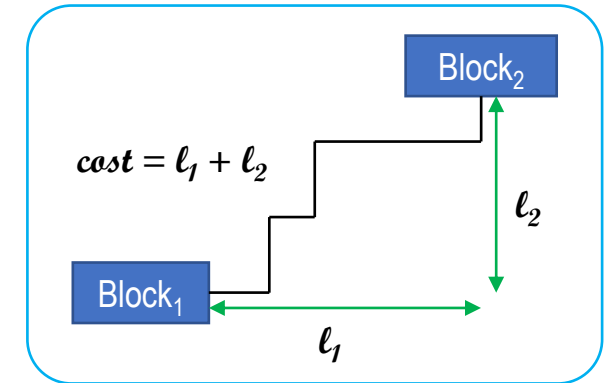
- Simulated Annealing (SA)

- It mimics the annealing process used to gradually cool molten metal
 - It produces a high quality crystal structure
- Start from an existing solution
- Make successive changes in a series of random moves
 - A move is accepted or rejected based on an *energy function* (or *cost function*)
- Allow “up-hill” moves to a less desirable solution to escape from *local minima*
 - Probability of accepting an “up-hill” move controlled by the expression $e^{-\frac{\Delta E}{T}}$
 - ΔE is the resulting increase in the energy function
 - T is the current temperature
- Temperature T is slowly decreased
 - The system comes to a rest at a low energy configuration



Field Programmable Gate Array (FPGA)

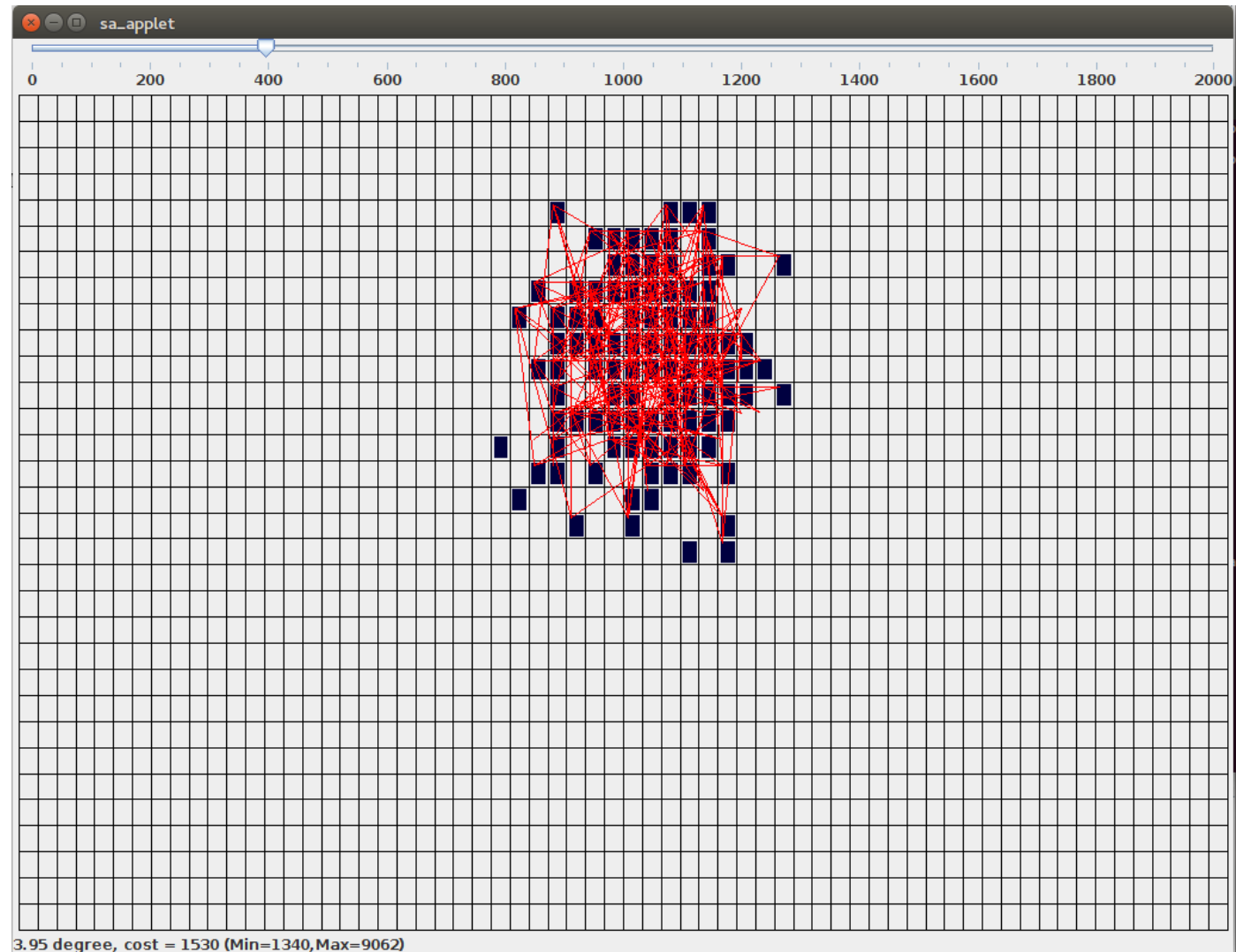
- Simulated Annealing (SA)
 - Initial placement
 - Assign logic blocks randomly to available locations on the FPGA
 - Cost function
 - Sum over all nets of the half perimeter of their bounding boxes
 - Manhattan distance
 - Move
 - Exchange the location of two randomly selected logic blocks
 - Initially the temperature T is very high
 - Almost all moves are accepted
 - Even those that lead to a worse solution
 - Temperature T gradually decreased
 - Probability of accepting a move to a worse solution becomes lower



```
begin
S = randomPlacement();
T = initialTemperature();
while (exitCriterion == FALSE) do
  while (innerLoopCriterion == FALSE) do
    S' = generateMove(S);
     $\Delta E = cost(S') - cost(S)$ ;
    if ( $\Delta E \leq 0$ ) then S = S';
    if ( $\Delta E > 0$ ) then S = S' with probability  $e^{-\frac{\Delta E}{T}}$ ;
  end while;
  T = updateTemp();
end while;
return S;
end
```

Field Programmable Gate Array (FPGA)

- Small java applet to show Simulated Annealing in action
 - Set array size, number of vertexes and number of edges from the command line
 - Defaults
 - Size: 64×32
 - 100 vertexes with 300 edges
 - Set temperature T manually using a slider
 - Status bar shows the current cost and the minimum and maximum found



Field Programmable Gate Array (FPGA)

- Small java applet to show Simulated Annealing in action
 - Try to identify the various parts of the algorithm

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import javax.swing.*;
import javax.swing.event.*;
import java.util.*;
import java.io.*;
import java.lang.Math.*;

class node {
    int id;
    int[] edges;
    int x;
    int y;
    int nmoves;
    int ntries;
}

public class sa_applet extends JApplet implements ChangeListener {

    public sa_painter painter;

    public static void main(String[] args) {
        JFrame f;
        sa_applet ap;
        JSlider tempSlider;
        int nv, ne, nx, ny;
        if (args.length != 0 && args.length != 4) {

            System.err.println("Usage: java sa_applet nv ne nx ny");
            System.err.println("      java sa_applet");
            System.err.println("      (defaults: nv=100, ne=300, nx=64, ny=32)");
            System.exit(1);
        }
        nv = 100;
        ne = 300;
        nx = 64;
        ny = 32;
        if (args.length > 0) {
            try {
                nv = Integer.parseInt(args[0]);
                ne = Integer.parseInt(args[1]);
                nx = Integer.parseInt(args[2]);
                ny = Integer.parseInt(args[3]);
            } catch (NumberFormatException e) {
                System.err.println("Arguments must be integers");
                System.exit(1);
            }
        }
        f = new JFrame("sa_applet");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ap = new sa_applet();

        tempSlider = new JSlider(0, 2000, 1500);
        tempSlider.setMinorTickSpacing(50);
        tempSlider.setMajorTickSpacing(200);
        tempSlider.setPaintTicks(true);
        tempSlider.setPaintLabels(true);

        tempSlider.addChangeListener(ap);
        ap.add("North", tempSlider);
        ap.painter = new sa_painter();
        ap.painter.nx = nx;
        ap.painter.ny = ny;
        ap.painter.nv = nv;
        ap.painter.ne = ne;
        ap.painter.myinit();
        ap.painter.cost_label = new JLabel("Cost = xxxxxxxx");
        ap.painter.setTemperature(1500);
        ap.add("Center", ap.painter);
        ap.add("South", ap.painter.cost_label);

        f.add("Center", ap);
        f.pack();
        f.setSize(new Dimension(1024,768));
        f.setVisible(true);

        while (true) {
            ap.painter.move();
        }

        public void stateChanged(ChangeEvent e) {
            JSlider slider = (JSlider) e.getSource();
            painter.setTemperature(slider.getValue());
        }
    }
}
```

Field Programmable Gate Array (FPGA)

- Class sa_painter

```
class sa_painter extends Component {

    JLabel cost_label;

    public int nx, ny, nv, ne;

    Random generator_init, generator_move;

    int[] matrix;
    node[] nodes;
    double temperature;
    int iterations;
    int max_cost, min_cost;

    public void setTemperature(int temp) {
        double t;

        t = (double) temp / 100.0;
        temperature = t;
    }

    public void myinit() {
        int i, j;
        int x, y;
        int v1, v2;

        generator_init = new Random();
        generator_move = new Random();
        nodes = new node[nv];
        iterations = 0;
        /* Initialize nodes */

        for (i = 0; i < nv; i++) {
            nodes[i] = new node();
            nodes[i].id = i;
            nodes[i].nmoves = 0;
            nodes[i].ntries = 0;
        }

        /* Initialize matrix */
        matrix = new int[nx][ny];
        for (i = 0; i < nx; i++) {
            for (j = 0; j < ny; j++) {
                matrix[i][j] = -1;
            }
        }

        /* Initialize connections */
        for (i = 0; i < ne; i++) {
            int[] dummy;
            do {
                v1 = generator_init.nextInt(nv);
                v2 = generator_init.nextInt(nv);
            } while (v1 == v2);
            if (nodes[v1].edges != null) {
                dummy = new int[nodes[v1].edges.length+1];
            } else {
                dummy = new int[1];
            }
            for (j = 0; j < dummy.length - 1; j++) {
                dummy[j] = nodes[v1].edges[j];
                dummy[dummy.length - 1] = v2;
                nodes[v1].edges = dummy;
            }
            if (nodes[v2].edges != null) {
                dummy = new int[nodes[v2].edges.length+1];
            } else {
                dummy = new int[1];
            }
            for (j = 0; j < dummy.length - 1; j++) {
                dummy[j] = nodes[v2].edges[j];
                dummy[dummy.length - 1] = v1;
                nodes[v2].edges = dummy;
            }
        }

        /* Create first random placement */
        for (i = 0; i < nv; i++) {
            do {
                x = generator_init.nextInt(nx);
                y = generator_init.nextInt(ny);
            } while (matrix[x][y] != -1);
            matrix[x][y] = i;
            nodes[i].x = x;
            nodes[i].y = y;
        }
        max_cost = min_cost = compute_cost();
    }

    public int compute_cost() {
        int i, j, x, y;
        int cost = 0;

        for (i = 0; i < nv; i++) {
            if (nodes[i].edges != null) {
                for (j = 0; j < nodes[i].edges.length; j++) {
                    if (nodes[i].edges[j] > i) {
                        x = nodes[i].x - nodes[nodes[i].edges[j]].x;
                        if (x < 0) x = -x;
                        y = nodes[i].y - nodes[nodes[i].edges[j]].y;
                        if (y < 0) y = -y;
                        delta = delta + x + y;
                    }
                }
            }
            return cost;
        }

        i = generator_move.nextInt(nv);
        nodes[i].ntries++;
        do {
            newx = generator_move.nextInt(nx);
            newy = generator_move.nextInt(ny);
        } while (matrix[newx][newy] >= 0);
        delta = 0;
        if (nodes[i].edges != null) {
            for (j = 0; j < nodes[i].edges.length; j++) {
                x = nodes[i].x - nodes[nodes[i].edges[j]].x;
                if (x < 0) x = -x;
                y = nodes[i].y - nodes[nodes[i].edges[j]].y;
                if (y < 0) y = -y;
                delta = delta - x - y;
            }
        }
        uphill = false;
        if (delta > 0) {
            p = Math.exp(-delta / temperature) * 1000.0;
            if (p < 1.0) p = 0.0;
            dice = generator_move.nextInt(1000);
            if (dice < p) uphill = true;
        }

        if (delta <= 0 || uphill) {
            matrix[nodes[i].x][nodes[i].y] = -1;
            nodes[i].x = newx;
            nodes[i].y = newy;
            nodes[i].nmoves++;
            matrix[newx][newy] = i;
            int tcost = compute_cost();
            if (tcost > max_cost) max_cost = tcost;
            if (tcost < min_cost) min_cost = tcost;
            repaint();
        }
        iterations++;
    }
}
```

Field Programmable Gate Array (FPGA)

- Class sa_painter (continued)

```
public Dimension getPreferredSize() {
    return new Dimension(640, 480);
}

public void paint(Graphics g) {
    int i, j;
    int x, y;
    double xx1, yy1, xs1, ys1;
    double xx2, yy2;
    Graphics2D g2;
    Dimension size;

    g2 = (Graphics2D) g;
    size = getSize();

    /* Draw grid */
    g2.setColor(Color.black);
    for (y = 0; y <= ny; y++) {
        yy1 = (((double) size.height - 10.0) / (double) ny) * (double) y + 5.0;
        g2.draw(new Line2D.Double(5, yy1, size.width - 5, yy1));
    }
    for (x = 0; x <= nx; x++) {
        xx1 = (((double) size.width - 10.0) / (double) nx) * (double) x + 5.0;
        g2.draw(new Line2D.Double(xx1, 5, xx1, size.height - 5));
    }

    /* Draw blocks */
    g2.setColor(new Color(0, 0, 64));
    for (i = 0; i < nv; i++) {
        xx1 = (((double) size.width - 10.0) / (double) nx) * (double) nodes[i].x + 7.0;
        yy1 = (((double) size.height - 10.0) / (double) ny) * (double) nodes[i].y + 7.0;
        xs1 = (((double) size.width - 10.0) / (double) nx) - 4.0;
        ys1 = (((double) size.height - 10.0) / (double) ny) - 4.0;
        g2.fill(new Rectangle2D.Double(xx1, yy1, xs1, ys1));
    }

    /* Draw connections */
    g2.setColor(Color.red);
    for (i = 0; i < nv; i++) {
        if (nodes[i].edges != null) {
            for (j = 0; j < nodes[i].edges.length; j++) {
                if (nodes[i].edges[j] > i) {
                    xx1 = (((double) size.width - 10.0) / (double) nx) * (double) nodes[i].x + 9.0;
                    yy1 = (((double) size.height - 10.0) / (double) ny) * (double) nodes[i].y + 9.0;
                    xx2 = (((double) size.width - 10.0) / (double) nx) * (double) nodes[nodes[i].edges[j]].x + 9.0;
                    yy2 = (((double) size.height - 10.0) / (double) ny) * (double) nodes[nodes[i].edges[j]].y + 9.0;
                    g2.draw(new Line2D.Double(xx1, yy1, xx2, yy2));
                }
            }
        }
    }

    g2.setColor(Color.black);
    String s1 = temperature + " degree, cost = " + compute_cost() + " (Min=" + min_cost + ",Max=" + max_cost + ")";
    cost_label.setText(s1);
}
```

Field Programmable Gate Array (FPGA)

- Min-Cut
 - The circuit is repeatedly partitioned into two sub-circuits
 - The chip area is alternatively partitioned in the horizontal and vertical directions into subsections
 - Each sub-circuit is assigned to a subsection
 - Process repeated until each sub-circuit consists of a single logic block
 - Has a unique location on the chip area
- Partitioning
 - Number of nets cut by the partition is minimized
 - Densely connected circuits are placed closely
 - Reduce delays
 - Simple partitioning algorithms lead to local minima and cannot escape
 - Kernighan-Lin (K-L) algorithm designed to be able to escape from local minima

Field Programmable Gate Array (FPGA)

- Routing
 - Global Routing
 - Find for each net a routing tree
 - Select a set of routing channels
 - Do not choose specific routing tracks and switches for each net
 - Minimize channel density to guarantee routability
 - Detailed routing
 - Assign to each net specific routing segments in the channels
 - Algorithm strongly depend on the FPGA routing architecture

Field Programmable Gate Array (FPGA)

- Timing analysis

- Estimate delays of all nets

- Lumped parameters

- Elmore delay if model is with R and C only

- Distributed parameters

- Transmission line

- Generate a *timing graph* of the entire circuit

- Longest combinational path determines the maximum clock frequency

- Or minimum clock period (cycle time)

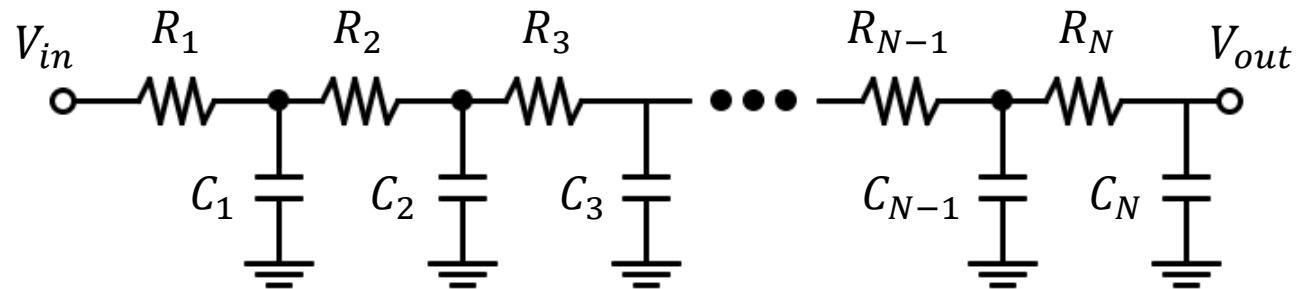
- Found by traversing the timing graph in topological order

- Complexity is $\mathcal{O}(n)$

- *slack*: delay that can be added to a connection without increasing the minimum cycle time of the circuit

- If a connection has slack = 0, then it is a critical path

- If a connection has slack < 0, then timing constraints have been violated



$$\tau = \sum_{k=1}^N \left(C_k \sum_{j=1}^k R_j \right)$$